

Introduction to Mobile Communication Systems & Security

IE3112 – Mobile Security

Next Generation All-IP Architecture

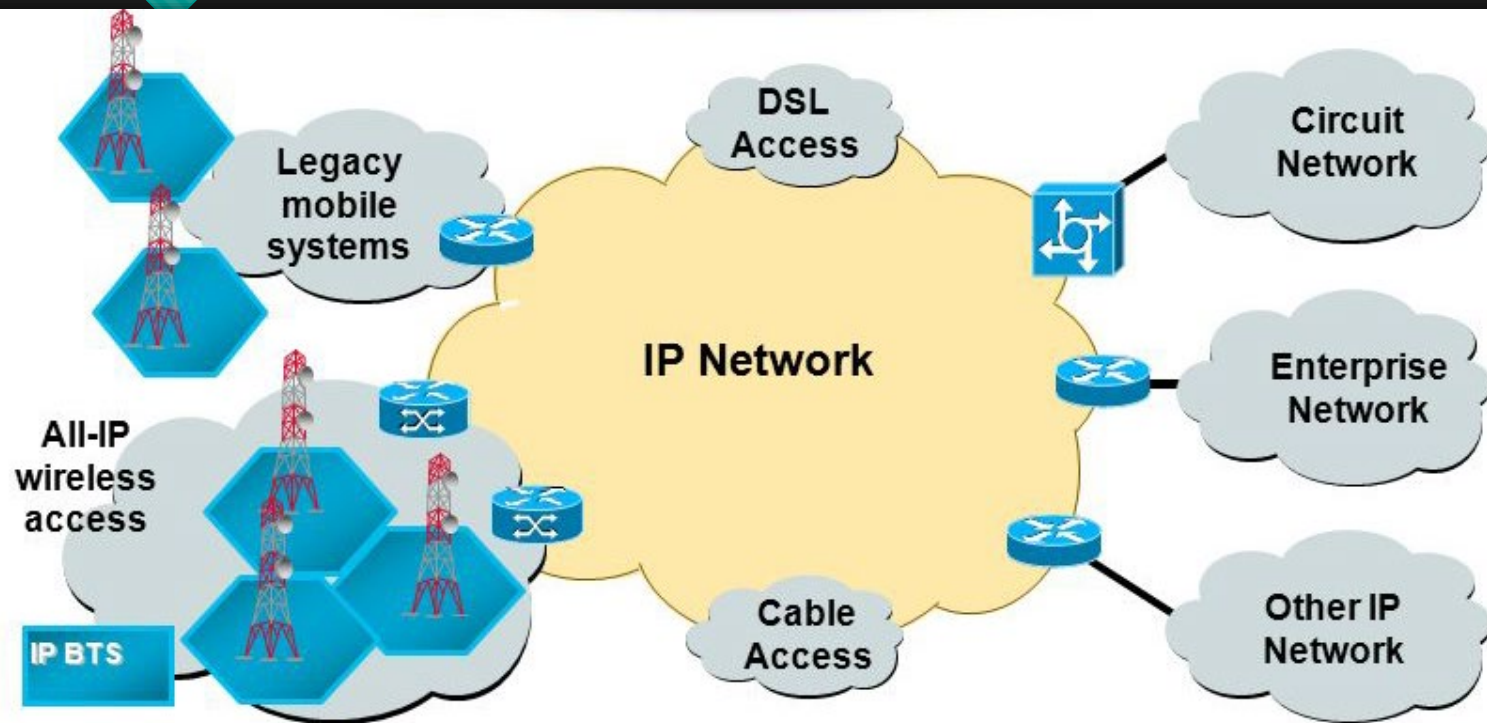
Existing Architecture



IP Architecture



Wireless & Mobile: All-IP Architecture

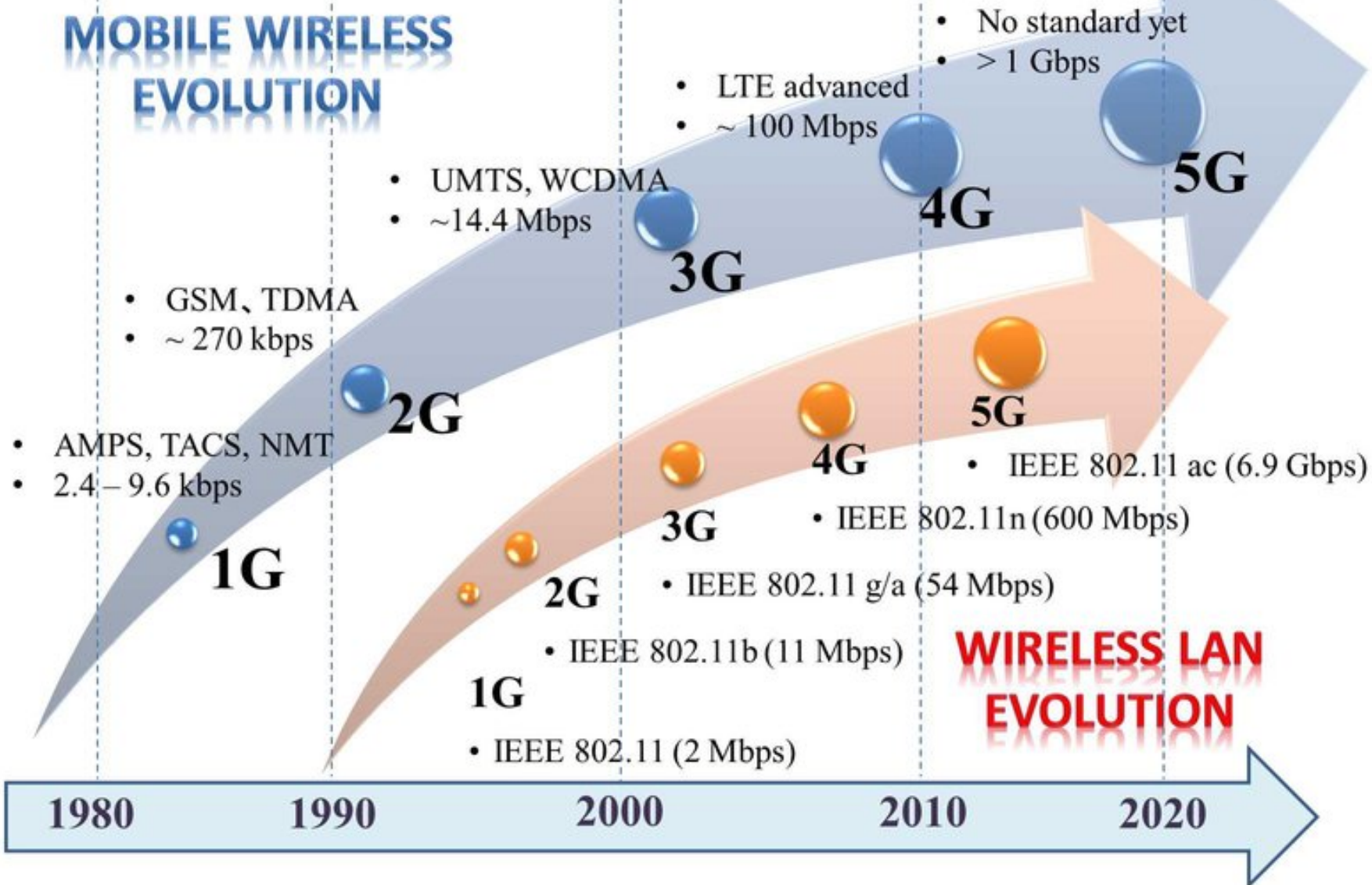


Wireless becomes just another access technology
Mobility mgmt & radios resource mgmt are the key

Wireless Comes of Age

- ❑ Guglielmo Marconi invented the wireless telegraph in 1896
 - ❑ Communication by encoding alphanumeric characters in analog signal
 - ❑ Sent telegraphic signals across the Atlantic Ocean
- ❑ Communications satellites launched in 1960s
- ❑ Advances in wireless technology
 - ❑ Radio, television, mobile telephone, communication satellites
- ❑ More recently
 - ❑ Satellite communications, wireless networking, cellular technology⁴

MOBILE WIRELESS EVOLUTION



Embracing a Wireless World

- ❑ The last two decades have witnessed a revolution of sorts in communication spearheaded by the rapidly evolving technologies in both software and hardware.

A mobile communication systems consists of:

- ❑ Mobile telephone, broadly construed here to include devices based on Code Division Multiple Access (CDMA), Time Division Multiple Access (TDMA), Global System for Mobile Communications (GSM)
- ❑ Wireless Personal Digital Assistants (WPDA) digital technologies and follow-ons, as well as satellite telephones and email appliances.

Broadband Wireless Technology

- ❑ Higher data rates obtainable with broadband wireless technology
 - ❑ Graphics, video, audio
- ❑ Shares same advantages of all wireless services: convenience and reduced cost
 - ❑ Service can be deployed faster than fixed service
 - ❑ No cost of cable plant
 - ❑ Service is mobile, deployed almost anywhere

Embracing a Wireless World

- Mobile communication systems are revolutionising and shrinking the world to between two or more small handheld mobile devices.
- Competition between the mobile telecommunication operators resulting in plummeting device prices, the quickly developing smartphone technology and growing number of undersea cables and cheaper satellite technologies are bringing Internet access to almost everyone

Mobile Operating Systems

- **Android** - developed by a small startup company that was purchased by Google Inc., is a Linux-derived OS backed by Google, along with major hardware and software developers (such as Intel, HTC, ARM, Samsung, Motorola and eBay, to name a few), that form the Open Handset Alliance.
- **iOS** - iOS is Apple's mobile operating system, originally developed for the iPhone, it has since been extended to support other Apple devices such as the iPod touch, iPad and Apple TV.
- **Windows Phone** - is the mobile operating system, by Microsoft. Although the OS internally identified itself as version 7.1 during pre-beta 2 releases, it is marketed as version 7.5 in all published materials intended for end-users.

Mobile Operating Systems

- Bada (Samsung) – (Korean meaning “ocean” and “seashore”. First Wave S8500 was unveiled in 2010 in Barcelona. Version 1.2 was released with the Samsung S8530 Wave II phone. The alpha-version of Bada 2.0 was introduced on February 15, 2011, with Samsung S8600 Wave III.
- BlackBerry OS (RIM) - The operating system provides multitasking and supports specialized input devices that have been adopted by RIM for use in its handhelds, particularly the trackwheel, trackball, trackpad and touchscreen. Best known for its native support for corporate email, through MIDP 1.0 and, more recently, a subset of MIDP 2.0, which allows complete wireless activation and synchronization with Microsoft Exchange, Lotus Domino, or Novell GroupWise email, calendar, tasks, notes, and contacts, when used with BlackBerry Enterprise Server.

Mobile Operating Systems

- Symbian – (Nokia, Sony Ericsson) was used on more phones and smartphones globally than any other mobile OS. Symbian's strengths include its longevity, widespread use, and maturity as an operating system. With its most recent release, Symbian 9, increased emphasis has been placed on improved e-mail functionality, enhanced capabilities to assist third-party developers, and additional security functions.

Security in the Mobile Ecosystems

- As mobile devices, more importantly smart devices, become ubiquitous, the risk for using them is increasing.
- They are increasingly holding and storing more private data like personal and business and they are roaming in public spaces on public networks with limited security and cryptographic protocols to protect the data.
- Major threats to mobile devices include:
 - Application-Based Threats
 - Web-based Threats
 - Network Threats
 - Physical Threats
 - Operating System Based Threats

General Mobile Device Attack Types

- Most mobile system attacks are launched against specific mobile devices or operating systems or applications.
- Most of these attack techniques are carried over from the computer and computer networks.
- The most common attack channels and techniques are:
 - Denial-of-service (DoS)
 - Phone Hacking
 - Mobile malware/virus
 - Spyware
 - Exploit
 - Phishing
 - SMishing

Mitigation of Mobile Device Attacks

- More and more people are now using a mobile device with either personal or work-related data.
- There is a growing number of employers are increasingly using unmanaged, personal devices to access sensitive enterprise resources and then connecting these devices to third party services outside of the enterprise security controls.
- This potentially expose the enterprise sensitive data to possible attackers.
- There are several security protocols and best practices that can come in handy to situations including:
 - Mobile Device Encryption
 - Mobile Remote Wiping
 - Mobile Passcode/Biometrics Policy

Users Role in Securing Mobile Devices

- Users must be aware that there are risks to the convenience afforded by mobile devices.
- It is important to know that mobile computing devices can store large amounts of personal and sometimes sensitive data whose loss may cause problems to the owner or user.
- It is also important to know that it is easy to steal or lose that data.

Users Role in Securing Mobile Devices

- Unless precautions are taken, an unauthorized person can gain access to the information stored on these mobile devices or gain accessed through these devices to other devices or data because these devices may provide access to other services that store or display non-public data.
- This access may be enabled because the mobile device contains passwords or security certificates and other information that may help to identify the device, its user or its content.
- So our role as users is to be vigilant and security aware.

GSM - Security

IE3112 – Mobile Security

Introduction

- **GSM (Global System for Mobile communication)** is a digital mobile telephony system and is the most popular standard for mobile phones in the world.
- Used by over 4 billion people across more than 212 countries
- Its ubiquity makes international roaming very common between mobile phone operators, enabling subscribers to use their phones in many parts of the world.
- GSM differs from its predecessors in that both signaling and speech channels are digital, and thus is considered a second generation (2G) mobile phone system.

Introduction

- **GSM (Global System for Mobile communication)** is being phased out around the world
- However it may outlive 3G(UMTS)
- Developing countries are unlikely to shutdown GSM yet

Switch-Off Date	Network Operator	Country
2025	Grameenphone	Bangladesh
2022	Telenor	Myanmar
2019	Airtel	India
2017	Telstra	Australia
2020	T Mobile	USA

Benefits

- GSM pioneered a **low-cost** (to the network carrier) alternative to voice calls, the **Short message service** (SMS, also called "text messaging")
- Standard includes **one worldwide Emergency telephone number, '112'**.
- This makes it easier for international travelers to connect to emergency services without knowing the local emergency number.

History

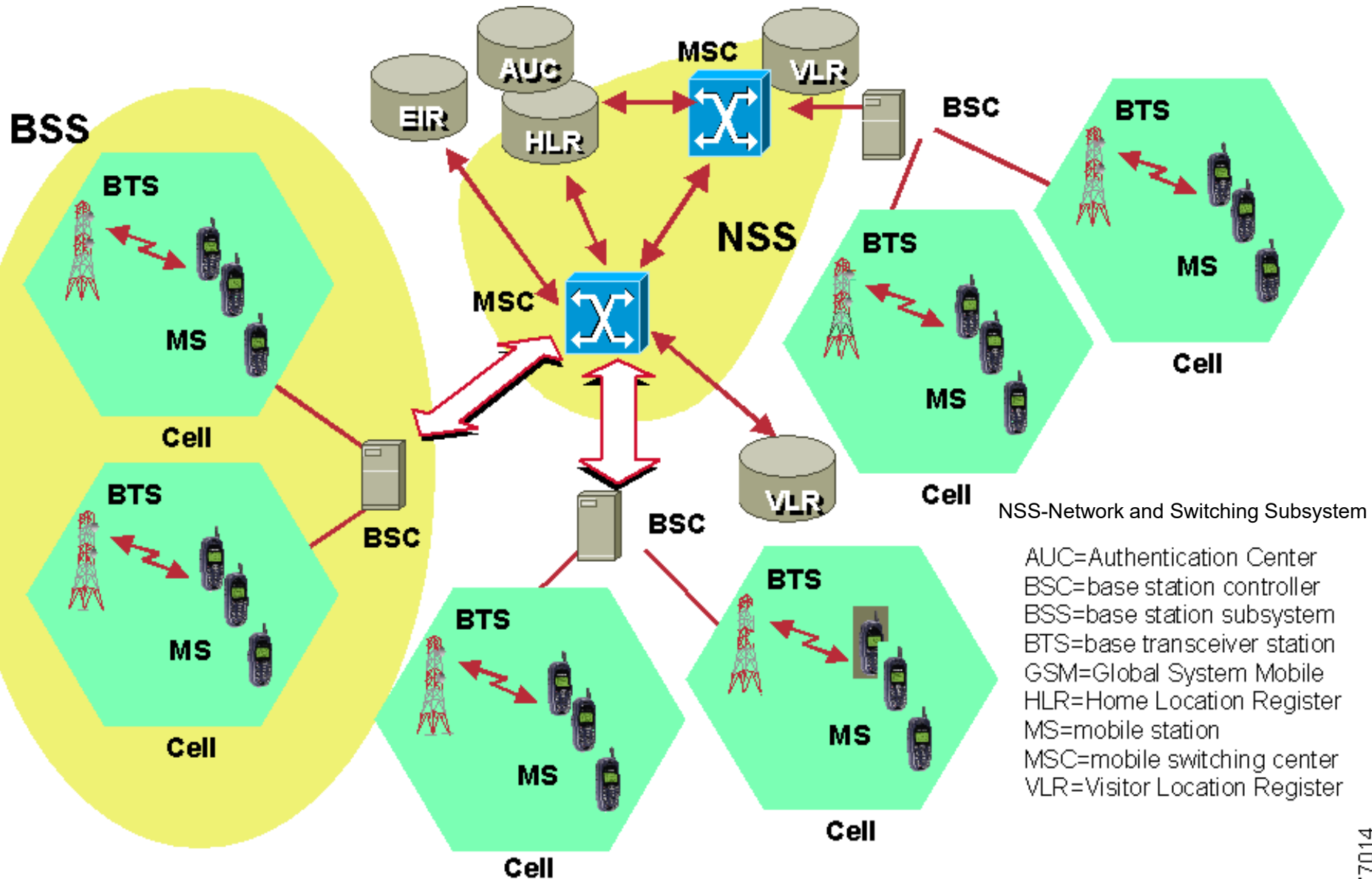
In **1982**, the European Conference of Postal and Telecommunications Administrations (CEPT) created the **Groupe Spécial Mobile (GSM)** to develop a standard for a mobile telephone system that could be used across Europe.

In **1987**, an MoU was signed by 13 countries to develop a common cellular telephone system across Europe.

In **1989**, GSM responsibility was transferred to the European Telecommunications Standards Institute (ETSI) and phase I of the GSM specifications were published in 1990.

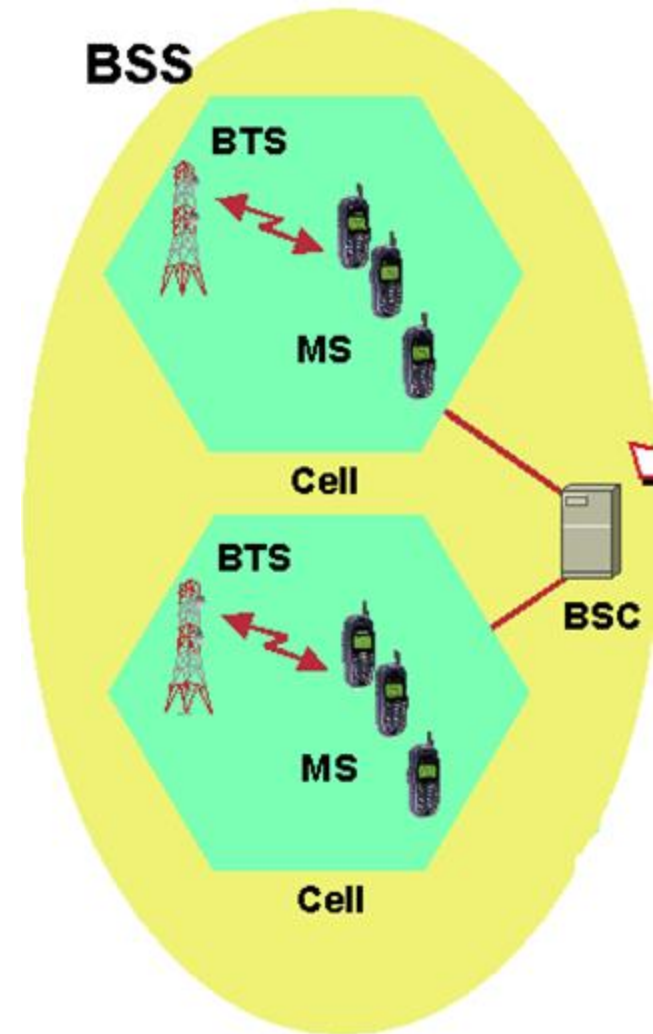
The first GSM network was launched in **1991** by Radiolinja in Finland with joint technical infrastructure maintenance from Ericsson.

GSM Network Architecture



Base Station Subsystem (BSS)

- A GSM network is comprised of many base station subsystems, each controlled by a BSC.
- Functions of BSS:
 - Monitoring radio connections to the MS
 - Coding and decoding voice
 - Rate adaptation to and from the wireless network
- A BSS can contain several BTSs.



The Mobile Station (MS)

consists of two major components:



1) Mobile Equipment (ME)

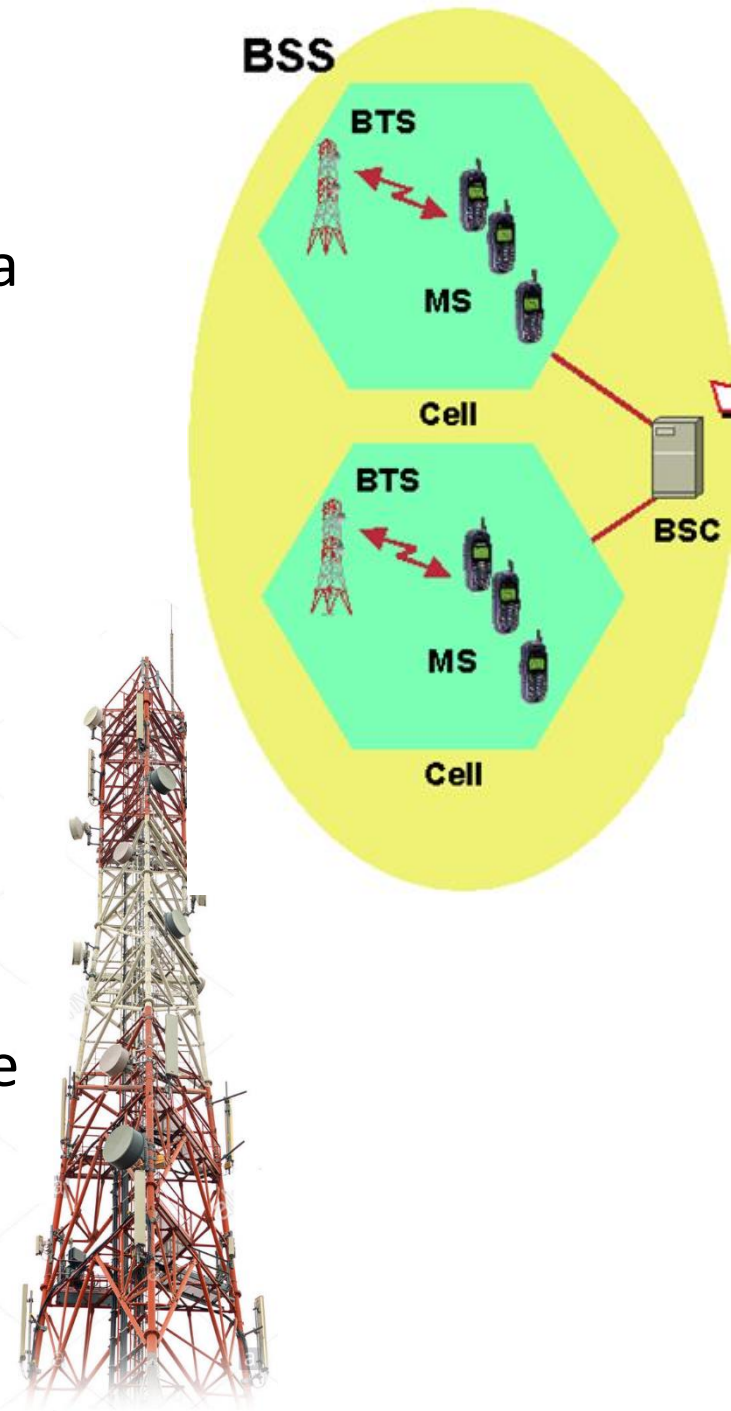
The Mobile Equipment is the actual mobile device a user uses to establish calls and other telephony services. The ME communicates with the radio channel and provides various services to the user of the mobile device.

2) Subscriber Identity Module(SIM)

The Subscriber Identity Module is located inside the ME and contains subscriber specific data. This data is used for **Identifying and Authenticating** a subscriber to the network .

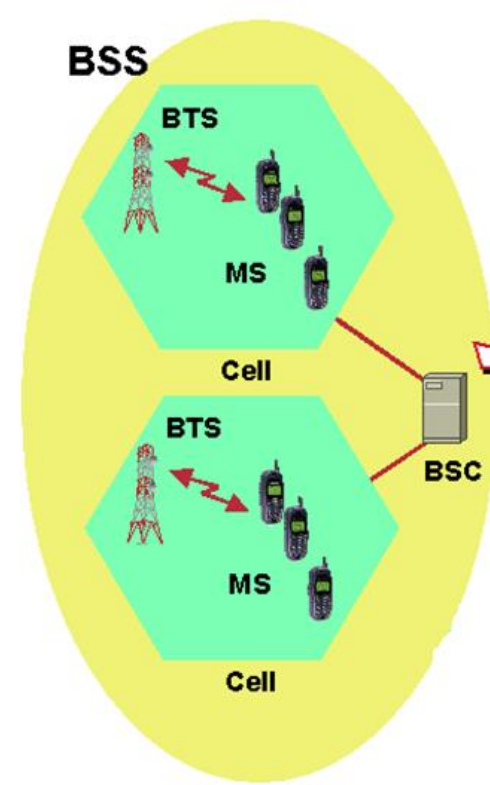
Base Transceiver Station(BTS)

- When a subscriber uses the MS to make a call in the network, the MS transmits the call request to the BTS.
- The BTS includes all the radio equipment (i.e., antennas, signal processing devices, and amplifiers) necessary for radio transmission within a geographical area called a **cell**.
- The BTS is responsible for establishing the link to the MS and for modulating and demodulating radio signals between the MS and the BTS.



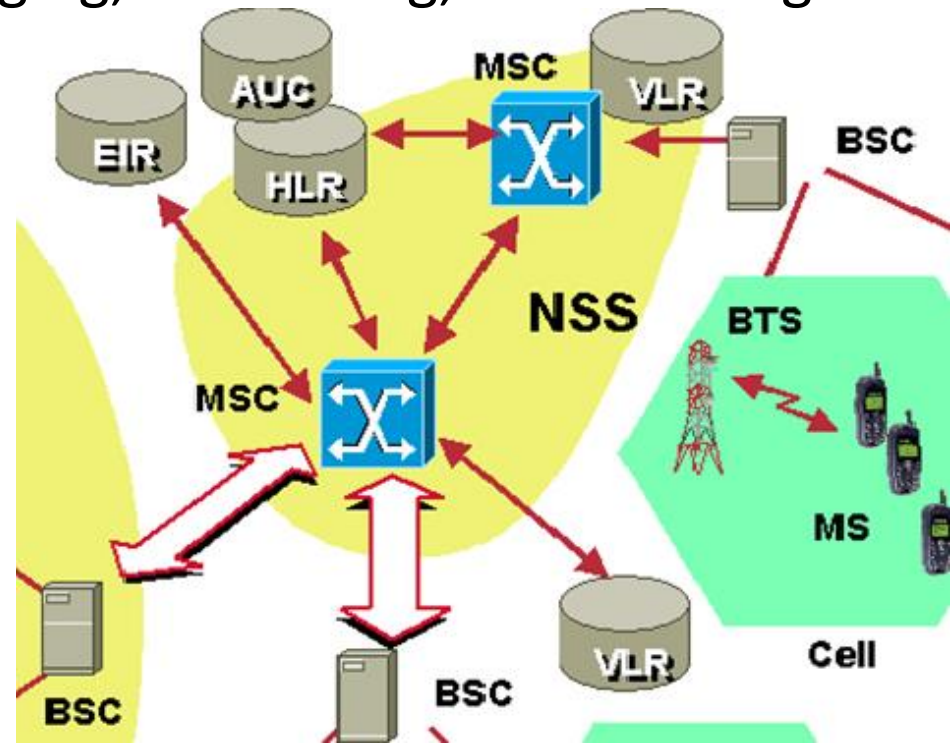
Base Station Controller (BSC)

- BSC is the controlling component of the radio network, and it manages the BTSs.
- The BSC reserves radio frequencies for communications and handles the handoff between BTSs when an MS roams from one cell to another.
- The BSC is responsible for paging the MS for incoming calls.



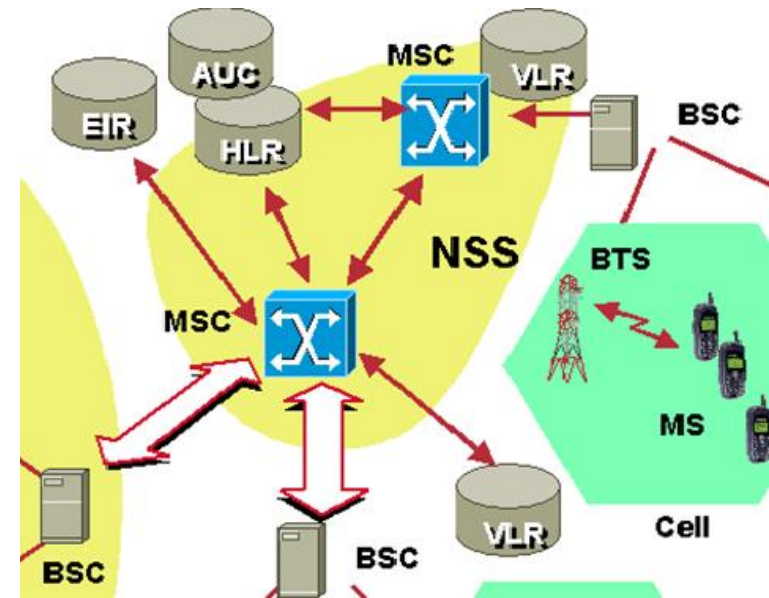
Network and Switching Subsystem(NSS)

- NSS is the heart of the GSM system
- It connects the wireless network to the standard wired network.
- Responsible for the handoff of calls from one BSS to another BSS
- Performs services such as charging, accounting, and roaming.



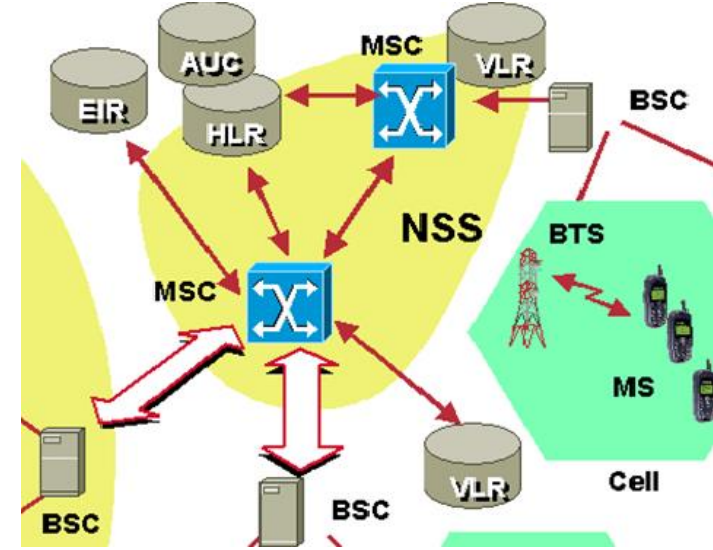
Mobile Switching Center(MSC)

- MSC is a digital ISDN(integrated services digital network) switch that sets up connections to other MSCs and to the BSCs.
- The MSCs form the wired (fixed) backbone of a GSM network and can switch calls to the public switched telecommunications network (PSTN).
- An MSC can connect to a large number of BSCs.



Equipment Identity Register(EIR)

- EIR is a database that stores the international mobile equipment identities (IMEIs) of all the mobile stations in the network.
- The IMEI is an equipment identifier assigned by the manufacturer of the mobile station.
- The EIR provides security features such as blocking calls from handsets that have been stolen.

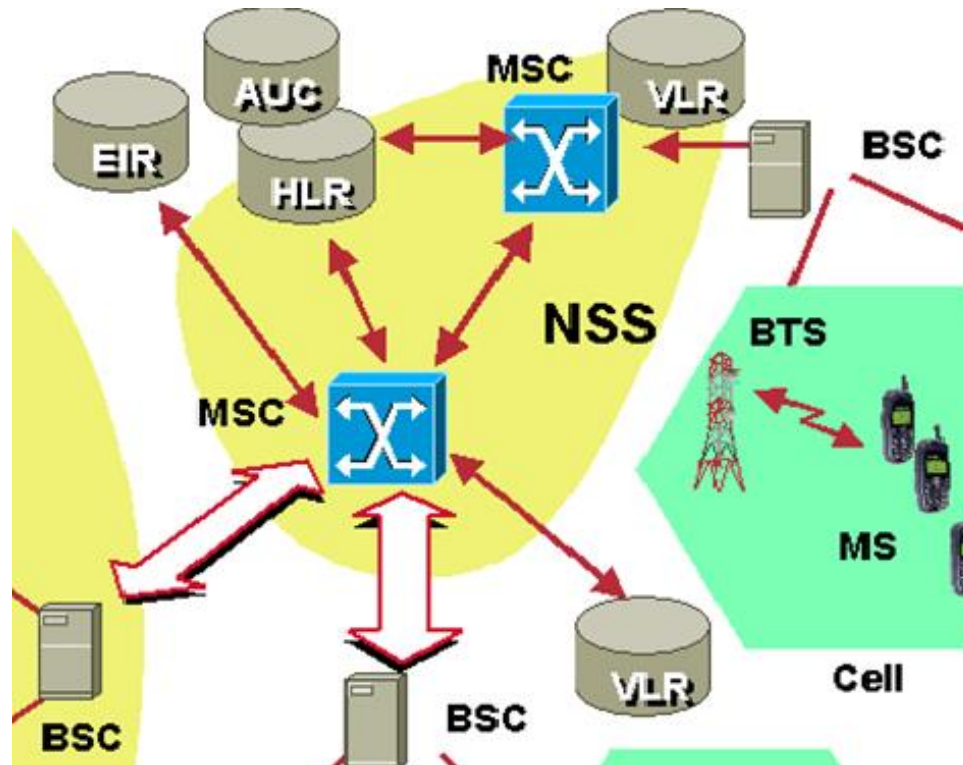


Home Location Register(HLR)

- HLR is the central database for all users to register to the GSM network.
- It stores static information about the subscribers such as the international mobile subscriber identity (IMSI), subscribed services, and a key for authenticating the subscriber.
- The HLR also stores dynamic subscriber information (i.e., the current location of the mobile subscriber).

Authentication Center(AUC)

- AUC is associated with the HLR.
- Database contains the algorithms for authenticating subscribers and the necessary keys for encryption to safeguard the user input for authentication.



Visitor Location Register(VLR)

- VLR is a database that temporarily stores information about MS's that are active in the geographic area for which VLR is responsible.
- A VLR is associated with each MSC in the network. When a new subscriber roams into a location area, the VLR is responsible for copying subscriber information from the HLR to its local database.
- This relationship between the VLR and HLR avoids frequent HLR database updates and long distance signaling of the user information, allowing faster access to subscriber information.
- The HLR, VLR, and AuC comprise the management databases that support roaming (including international roaming) in the GSM network.
- These databases authenticate calls while GSM subscribers roam between the private network and the public land mobile network (PLMN). The types of information they store include **subscriber identities, current location area, and subscription levels.**

Technical Details

- GSM uses a combination of both the time division multiple access (**TDMA**) and frequency division multiple access (**FDMA**) technologies. With this combination, more channels of communications are available, and all channels are digital.
- The GSM service is available in four frequency bands:
 - 450-MHz—Upgrade of older analog cellular systems in Scandinavia
 - 900-MHz—Original band used everywhere except North America and most of South America
 - 1800-MHz—New band to increase capacity and competition used everywhere except North America and most of South America
 - 1900-MHz—Personal communications service band used in North America and much of South America
- **The higher frequency bands provide additional capacity and higher subscriber densities.**
- GSM networks handle both voice and data traffic requirements of the mobile communication by providing two modes of operation:
 - Circuit switched (high-speed circuit switched data)
 - Packet switched (GPRS)
- The standard data rate of a GSM channel is **22.8 kbps**.

Technical Details Contd.

- The modulation used in GSM is Gaussian minimum-shift keying (GMSK), a type of continuous-phase frequency shift keying (FSK).
- In GMSK, the signal to be modulated onto the carrier is first smoothed with a Gaussian low-pass filter prior to being fed to a frequency modulator, which greatly reduces the interference to neighboring channels (adjacent channel interference).
- <https://www.youtube.com/watch?v=lyzpt3bKTTI>

From 2G to 3G

- **GPRS**- General Packet Radio System-56kbps (2.5G)
- **EDGE**- Enhanced Data rates for GSM Evolution-180kbps (2.75G)
- **UMTS**- Universal Mobile Telecommunications System -1920Kbps (3G)

4G

- **4G HSPA+** - Evolved High Speed Packet Access (1 to 10Mbps)
- **4G LTE** - Long Term Evolution (10 to 100Mbps)

5G ?

Subscriber Identity Module(SIM)



- One of the key features of GSM is the Subscriber Identity Module (SIM), commonly known as a SIM card.
- The SIM is a detachable smart card containing the user's subscription information and phone book.
- It also contains the necessary keys and algorithms needed for the authentication procedure, which enables a subscriber to connect to the home network.
- This allows the user to retain his or her information after switching handsets,
- Alternatively, the user can also change operators while retaining the handset simply by changing the SIM.

Subscriber Identity Module(SIM)



- Two security services are implemented for the SIM card
 - The first security mechanism for the SIM is access control, which controls a user from accessing the card and the information and services provided upon card access.
 - This is provided via a secret **Personal Identification Number (PIN)**, which the user must enter before gaining access to the SIM.
- The second security mechanism provided is the network challenge and response mechanism.

GSM security

- GSM Authentication is achieved by a **challenge-response mechanism** and by the encryption of the radio channel, which also guarantees confidentiality.
- **Anonymity** in GSM is achieved by using a **Temporary Mobile Subscriber Identity (TMSI)** which is agreed upon after authentication and key generation through an A5-encrypted channel.
- As a result, an eavesdropper cannot relate a TMSI being sent to a specific user of the network, and hence not gather information about a user's network activities.
- **The International Mobile Subscriber Identity (IMSI)** is a unique 15 digit identifier for a mobile subscriber. It is stored in the SIM card of the mobile station, and is assigned to a mobile subscriber at the time of subscription. It is used to identify a subscriber to a given network.
- Mobile subscribers do not have access to this number or have any knowledge of it.
- Although this number is stored in the SIM card, it cannot be reached via a telephone call. Thus, the number is not made public.

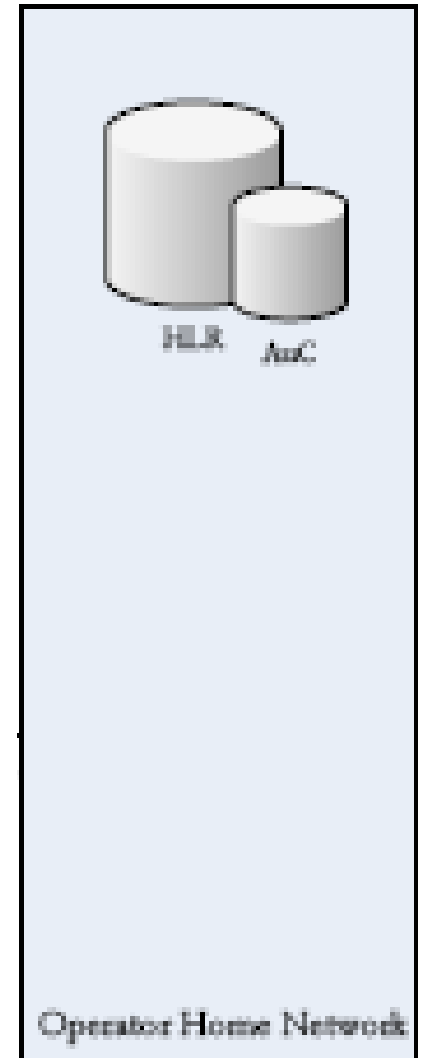
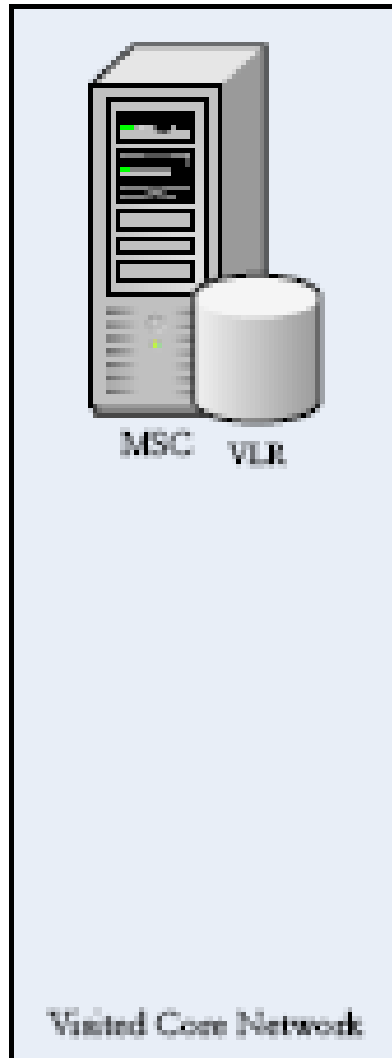
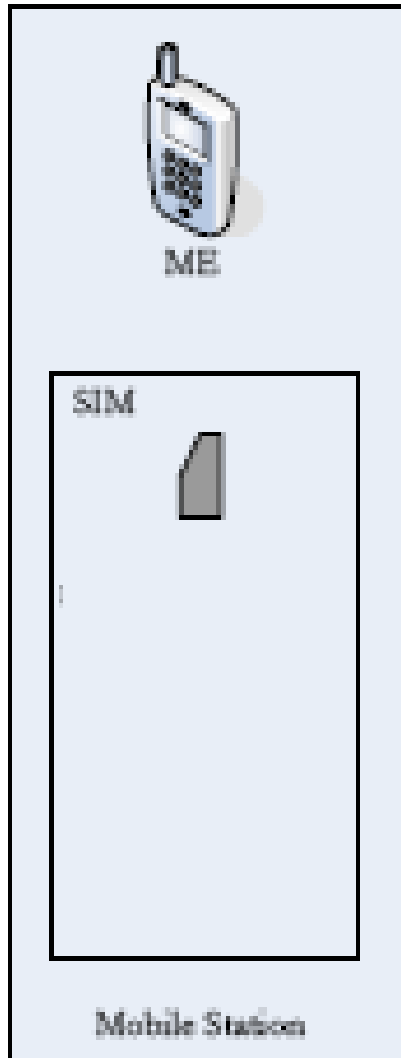
GSM Authentication

1. A secret key referred to as **K_i** is stored in the SIM card and in the AuC of the home network of the mobile operator.
2. This is a secret and is never transmitted from the AuC or SIM card. The **K_i** is a unique 128-bit key.
3. The concept behind the challenge-response type of authentication is to prove that the secret key, stored in the SIM card of the mobile station is the same as the key stored in the AuC.
4. The authentication procedure begins when a mobile station, requests access to the network. This is achieved via an authentication request, in which the **mobile device sends out the IMSI as a request for authentication.**
5. The IMSI is broadcasted to a corresponding MSC, which in turn **forwards this information to the HLR** in the home network, and also the VLR in the visited network.
6. The AuC is associated with the HLR, and is responsible for storing authentication specific parameters.
7. After the reception of the IMSI by the AuC, a **random number (RAND) is generated using the received IMSI and the stored secret key K_i.** The RAND number is a 128-bit key, and represents the challenge to be sent to the SIM by the home network.

GSM Authentication Contd.

8. The AuC and SIM card contain authentication algorithms, With the help of these algorithms an Expected Response key (XRES), which is 32-bits long, and a **Cipher key (K^c)**, 64-bits long are generated.
9. The XRES is used to verify if the SIM can generate the same response, and is based on a symmetric mechanism. The **K^c** is used for encrypting calls between the mobile and base stations, and is a temporary session key.
10. Upon generating these keys, the **HLR sends** out an authentication response known as **triplets, which consists of the (RAND, XRES and K^c)**. The triplets are generated and stored in the VLR for each subscriber.
11. The **MSC then forwards** the RAND number of the generated **triplets to the mobile station**. This RAND number is sent as a challenge to the mobile station, and challenges the mobile station to calculate the same response generated by the AuC.
12. With the use of the algorithms, the **RAND number and K^i key are used to calculate the RES and a K^c** .
13. The **RES is then forwarded to the MSC/VLR**, and a **comparison of RES and XRES is made**. If both responses match, the authentication procedure is successful and the mobile station gains access to the network and its services. If, however the XRES and RES don't match, then access is denied to the mobile station and the authentication procedure fails.

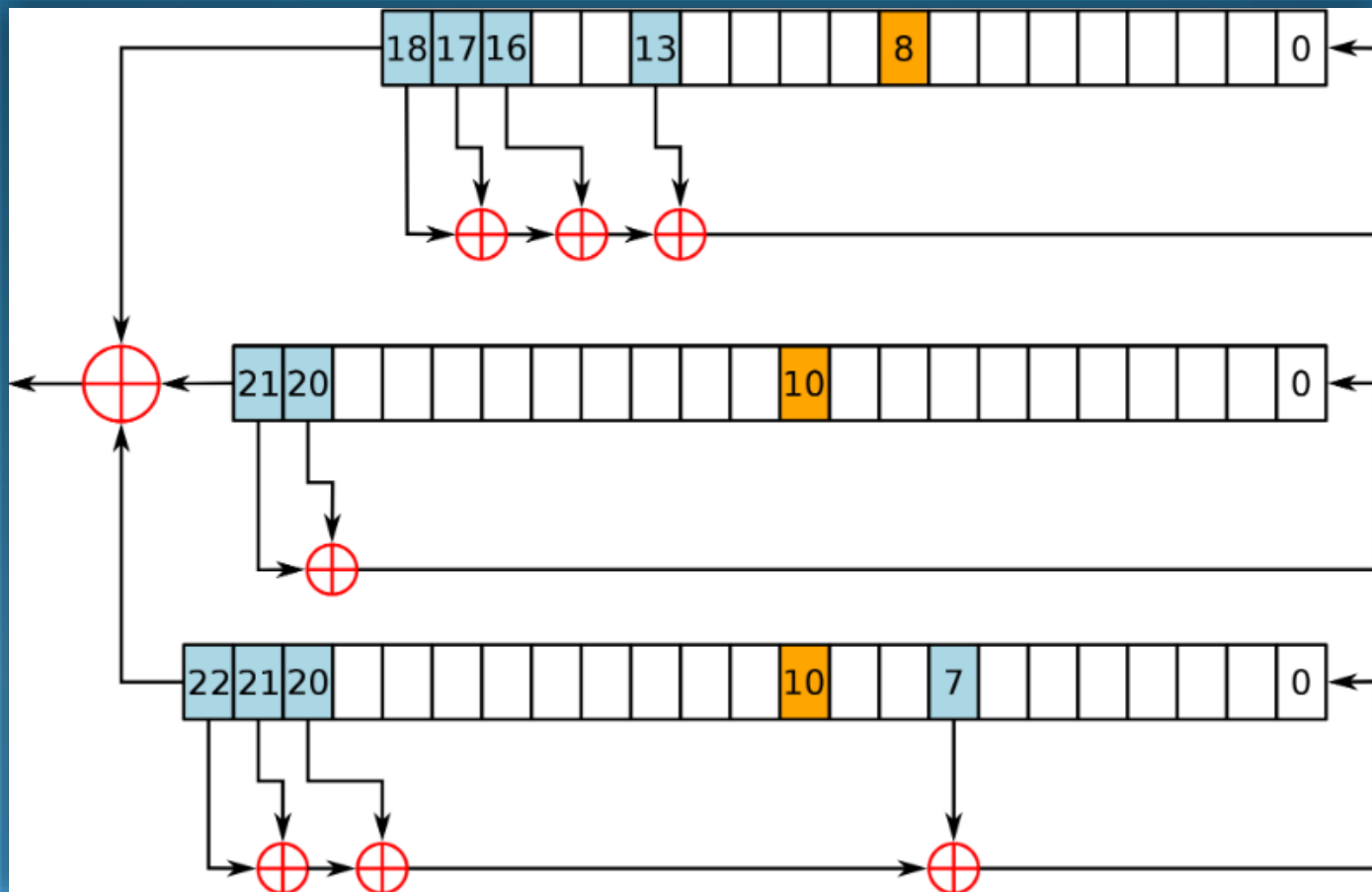
GSM Authentication



Encryption in GSM

- The A5 algorithm is the ciphering/deciphering algorithm, and resides on the mobile station of a subscriber and on the BSS.
- The A5 algorithm is used for protecting data sent from the mobile station, and the BSS and vice-versa, this provides the privacy of data and calls.
- The **Kc** ensures that all calls are encrypted between the MS and the BSS.

A5 Encryption



A5 Versions

- A5/0 – not really a version of A5; allows GSM to operate without encrypting call traffic
- A5/1 – Original A5 algorithm. Employed in Western Europe and the United States
- A5/2 – Second version of A5 algorithm. Employed outside of Europe and US
 - Weakened due to export restrictions on encryption technology during Cold War
- A5/3 – Stronger version of A5, for use in 3G networks

A5 Details

- A5 is a stream cipher
- Stream Ciphers
 - Used to encrypt small amounts of bits/bytes at a time
 - Uses keystreams combined with plaintext to produce cipher text
 - Generally, ciphertext is produced by XOR'ing keystream with plaintext
 - Uses a concept called Linear Feedback Shift Registers (LFSRs)
 - More on this next semester!

A5 Keystreams

- Consists of two parts:
 - Session key
 - Frame key
 - GSM Frames – data exchanged in blocks of 114-bit 'frames'
– similar to packets in TCP/IP

Real Time Cryptanalysis of A5/1 on a PC

Alex Biryukov, Adi Shamir, David Wagner

Used a PC containing 128 MB RAM and two or four 73 GB disks to examine at the algorithm's output.

Two attacks:

1. Records ciphertext for 2 minutes, then computes key in one second.
2. Records for 2 seconds, then computes key in several minutes.

The Biased Birthday Attack

- One could find the A5/1 key within a second, but needed the first 2 minutes of a conversation.
- 242 preprocessing steps with four 73GB disks
- 248 preprocessing steps with two 73GB disks
- Based upon direct collisions between a state in the disk and a state in the data, using approximately 71 red states.

The Random Subgraph Attack

- Only 2 seconds of data are needed, but several minutes are required for processing.
- Used 248 preprocessing steps with four 73GB disks.
- Used indirect collisions, allowing the key to be found from the first red state in the data

Cryptanalysis with COPACOBANA

Tim Güneysu, Timo Kasper, Martin Novotný, Christof Paar, and Andy Rupp

- Uses custom hardware called Cost-Optimized Parallel Code Breaker
- A cluster of 120 FPGAs
- Reconfigurable for different cryptanalysis tasks.
- One of these is an attack on A5/1.
 - COPACOBANA gives a TMDTO attack on A5/1
 - The attack "assume that a relatively small amount of only 114 consecutive bits of keystream is known."
- Assumes 114 consecutive bits of keystream is known.*
- 63% success rate; more data = better results.

Breaking GSM

Two kinds of devices:

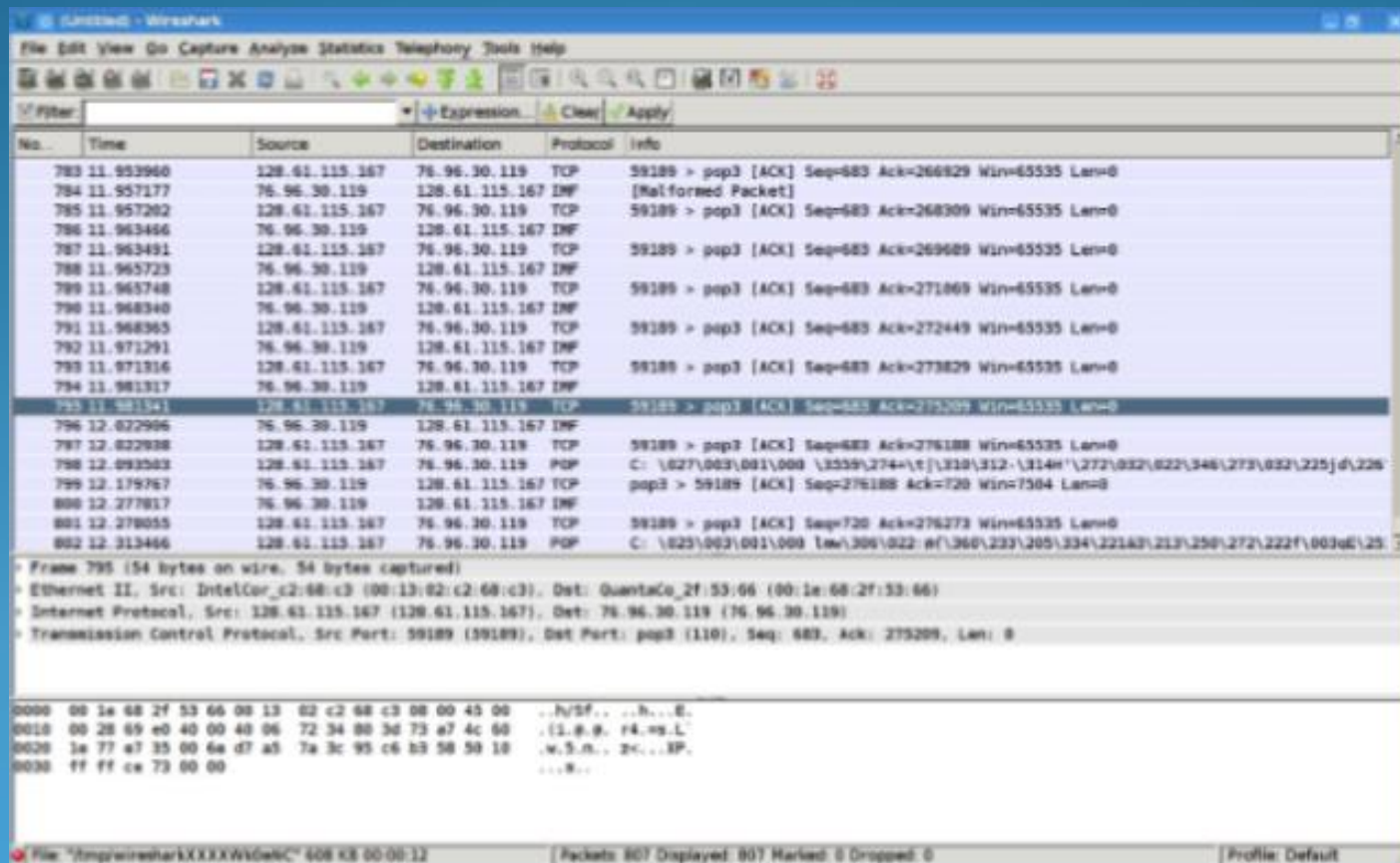
- Active intercept
 - Fake base station
 - Can be detectable
 - In practice no one is checking
- Passive cracking
 - More challenging
 - Requires special RF setup, pre-computation
 - Can be hidden.

Active

- Advertise your fake base station with a fake Mobile Country Code (MCC) and Mobile Network Code (MNC).
- Phones will connect to it if it has the strongest signal.
- Could be detected by phone, but no apps.
- Base station can choose not to use crypto.

Active

- Decode resulting data using either Wireshark (packet analyzer) or Airprobe (dedicated GSM sniffer)
- Discovered bugs in both phones and OpenBTS



Passive

- A5/1 vulnerable to pre-computation.
- Code book maps from known output to secret state.
- Stored naively, A5/1 book would be 128 PB (~ 128 million GB)
- Would take 100,000 years to be calculated.

Known plaintext

GSM phones disclose keystream through known or guessable plaintext:

- Empty ACKS
- Connect ACK
- IDLE frames
- System Information
- Call proceeding
- Alerting

Potential A5 Consequences

- Intercepting and decoding calls
- Monitoring data transfer
- Cloning of cell phones

Intercepting and Decoding Calls

- Recording of calls and decoding them later
- Listening in for personal information
 - Credit card information
 - Social security number
 - Banking information

Monitoring Data Transfer

- Reading SMS
- Banking Information
- Payments
- Web authentication

Cloning of Cell Phones

- Stealing phone services
 - Billing strangers
 - Performing illegal criminal activities over cloned phones

A5 v3

- Updated, stronger version of A5 encryption presented by the 3rd Generation Partnership Project (**3GPP**)
- Used for 3G communications
 - 3G supports voice communications and data
 - Enough bandwidth to support both operations simultaneously

Thank You!

3G UMTS - Security

IE3112 – Mobile Security

Story starts at 2G...

- **2G - Second-generation wireless telephone technology. Commercially launched on the GSM (Global System for Mobile Communications) standard in Finland in 1991**
- **Positives:**
 - Secure due to digital encryption
 - Spectrally efficient over analogue counterpart allowing greater take-up of the technology
 - 2G technologies enabled the various mobile phone networks to provide services such as text messages, picture messages and MMS (multi media messages)
- **2G has been superseded by newer technologies such as 2.5G, 2.75G, 3G, and 4G; however, 2G networks are still used in many parts of the world.**

EDGE

- **EDGE (Enhanced Data rates for GSM Evolution) is standardized by 3GPP as part of the GSM family**
 - Upgrade that provides a potential three-fold increase in capacity of GSM/GPRS networks.
 - EDGE uses higher-order PSK/8 phase shift keying (8PSK) for the upper five of its nine modulation and coding schemes.
 - EDGE produces a 3-bit word for every change in carrier phase. This effectively triples the gross data rate offered by GSM
 - EDGE can carry a bandwidth up to 236 kbit/s (with end-to-end latency of less than 150 ms)

UMTS/3G packet access

➤ 3GPP- Release 99

- first release of 3G (Evolution of Circuit switched technology) with simple data
- Up to 2Mbps ...perhaps only 384 Kbps speed in practise

➤ HSPA (High Speed Packet Access)

- Purely Packet switched – better resource utilization
- Throughput speeds:
 - 14.4 Mbps Downlink
 - 5.76 Mbps Uplink

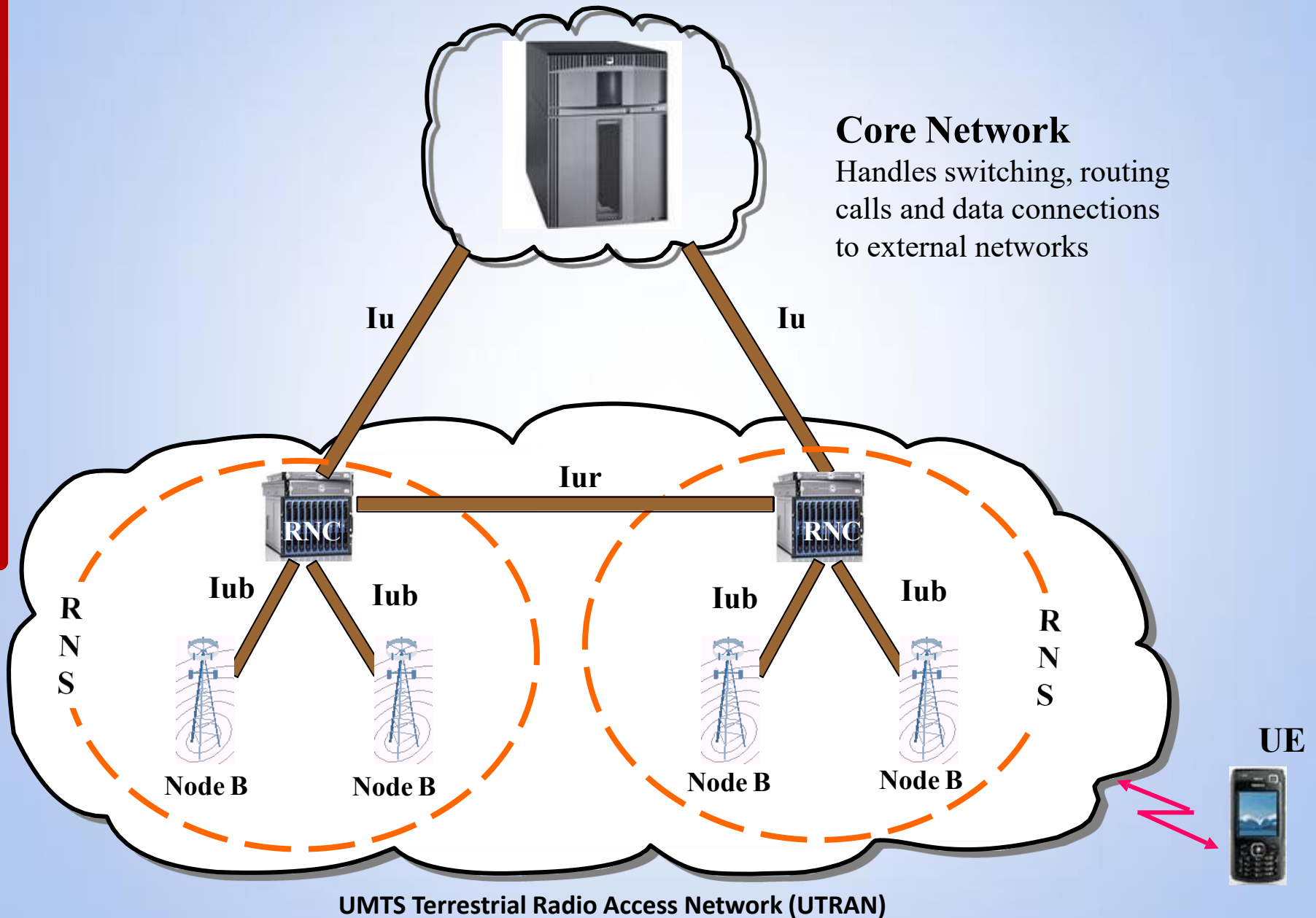
➤ HSPA evolution commonly referred to as HSPA+ (2010)

- Throughput speeds:
 - 56 Mbps Downlink
 - 22 Mbps Uplink

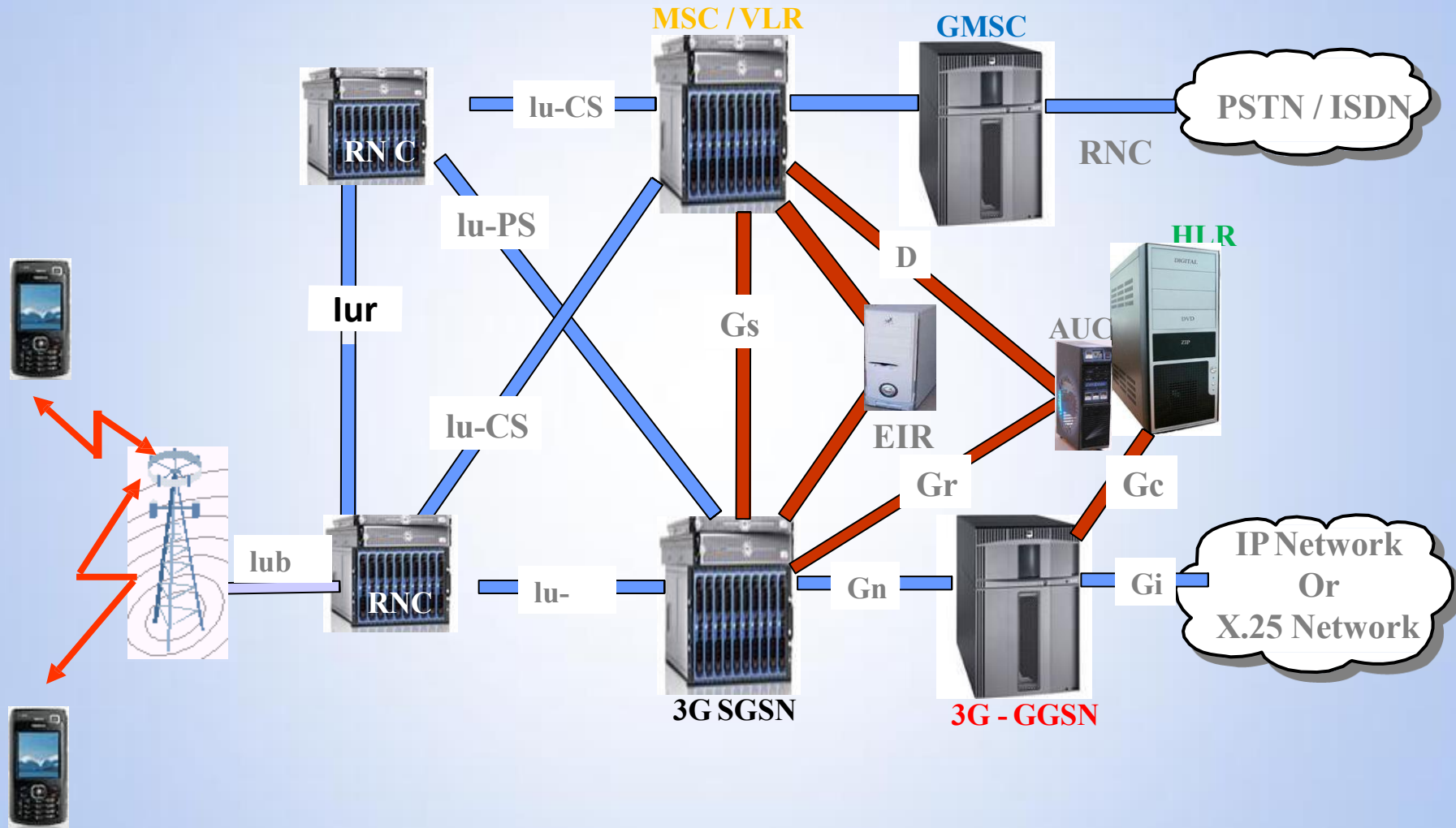


Communication Systems for Ubiquity

UMTS Network Architecture



General UMTS Architecture

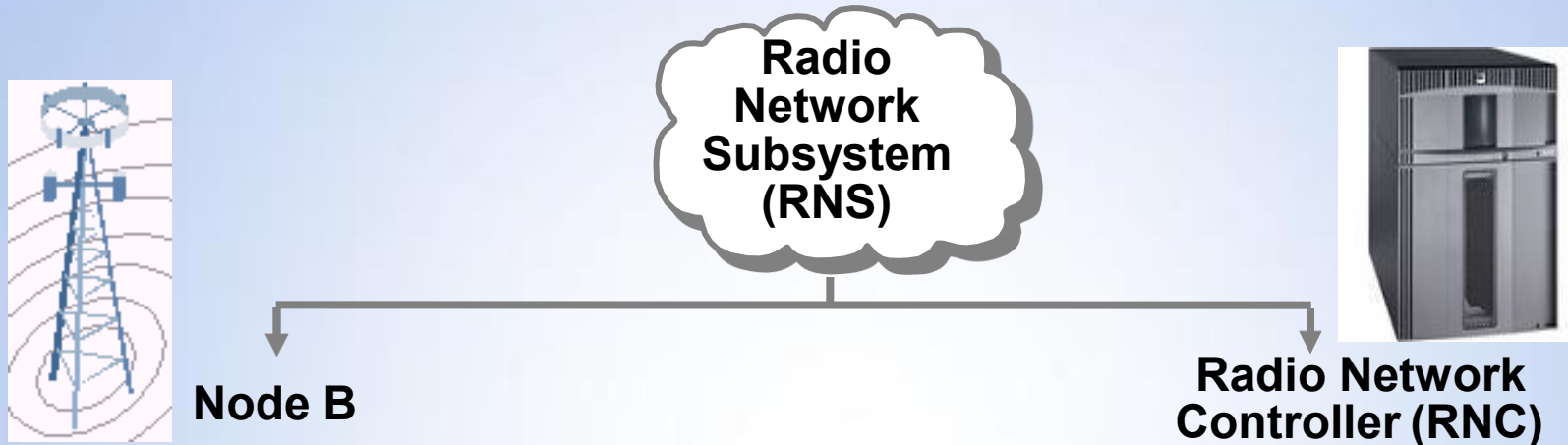


Core Network

Core Network Elements

- **HLR**- Located at users home system, stores users service profile
- **MSC/VLR**- Switch and Database that serves UE in its current location for circuit switched (CS) services.
- **GMSC**- is the Switch at the point where UMTS is connected to external CS networks.
- **SGSN**- Its functionality is similar to that of MSC/ VLR but for packet switched (PS) services. Mobility Management. SGSN Keeps track of the individual MS's location
- **GGSN**- GGSN acts as a logical interface to the GPRS network and to external public data networks such as IP and X.25

UTRAN Architecture

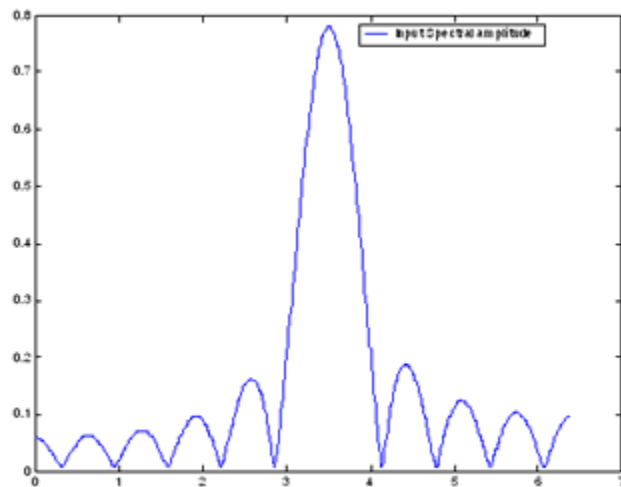


- Power Control
- Call connection with UE
- Hard & Softer hand-over control
- Traffic data collection/statistics
- Modulation/Demodulation
- Transmission/Reception
- CDMA Physical Channel Coding
- Error Protection
- FDD/TDD or Dual Mode

- Connection with Node B, MSC
- Termination of control signal from Node-B, MSC
- Call connection control with UE
- Diversity hand-over control
- Traffic data collection/statistics
- Resource management of radio network

Moving away from AWGN

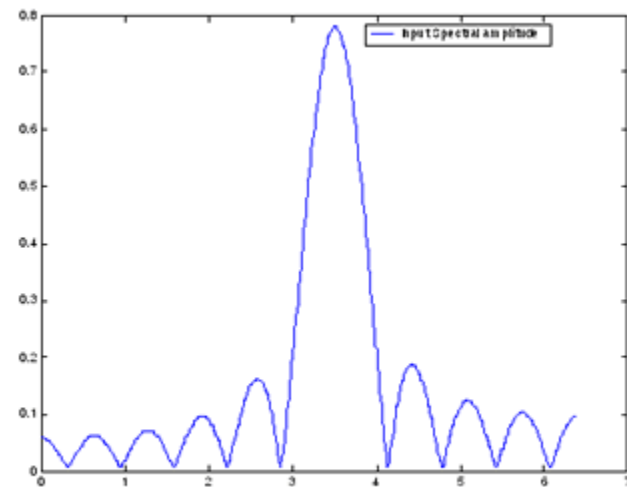
- **AWGN is the best possible communication channel you can get; but in reality seldom do we attain a AWGN scenario.**



Tx spectrum



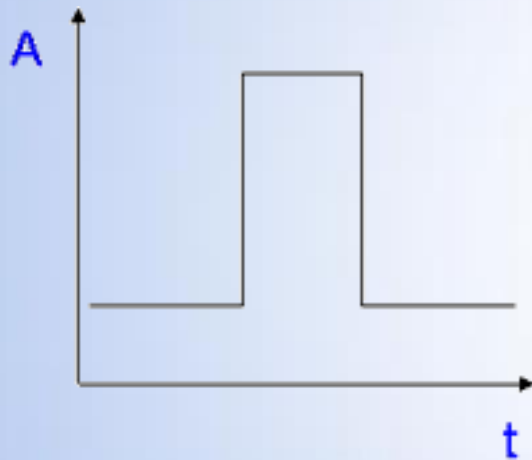
Never in practice



RX spectrum

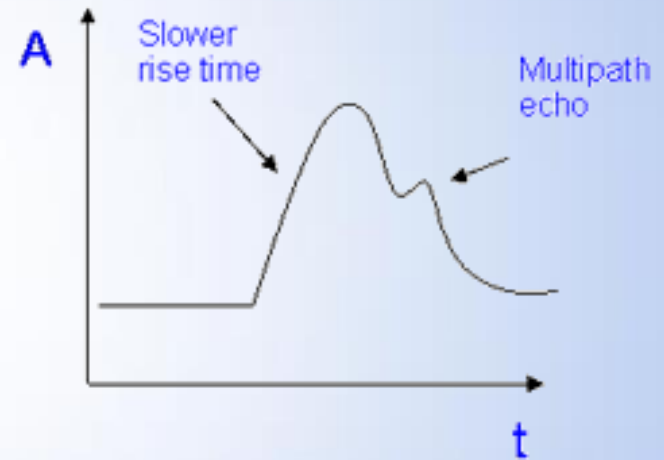
Moving away from AWGN

- Real channels show distortion and the time domain representation is frequently used. The received pulse is distorted (always) because of the frequency and delay response of the channel.



Nice clean transmitted pulse

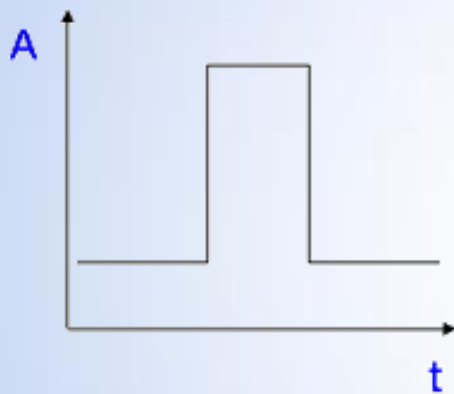
➔
E.g.



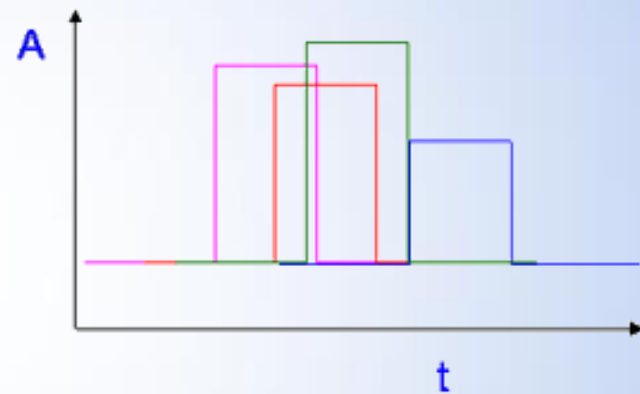
Distorted received pulse

Multipath Channel

- Apart from this filtering effect – pulses are also spread out over time; this can be due to scatterers; building, hills, clutter, etc... We represent this with the channel impulse response:



Send this – a nice clean pulse



Receive this – several overlapping copies with various amplitudes and time delays



RAKE

- **Original concept by PRICE and GREEN - 1952**
- **Combines the received signal constructively to maximise the SNR at the output. Its name come from the fact that its able to collect the energy from the prominent taps analogous to a RAKE instrument to provide a diversity gain.**
- **Delay Estimation Unit:** To estimate the tap delays of the channel impulse response to ensure tap alignment.
- **Channel Estimation unit:** To estimate the channel phase and amplitude of each tap, so as to recover the original phase of the modulated symbol.

Soft / Softer Handover

- When UE moves from a coverage area to another
- Better coverage and load balance
- This is based on defined received signal thresholds

Soft(er) Handover Terminology:

- 'Neighbours', 'Candidates', 'Actives'
- Maximum Number of Active Base Stations
- Hysteresis Margin (e.g. +/- 3dB)
- Window Size (e.g. 250ms)

The last three play an important role in Resource Management as they define the conditions for a Base Station to be included in the Active Set for a period of time.

VIDEO

3G Security Principles

- Build on GSM Security
- Correct Problems with GSM Security
- Add New Security Features

Problems with GSM Security - 1

Active Attacks

- Impersonating network elements such as false BTS is possible

Key Transmission

- Cipher keys and authentication values are transmitted in clear within and between networks (IMSI, RAND, XRES, Kc)

Limited Encryption Scope

- Encryption terminated too soon at edge of network to BTS
- Communications and signaling in the fixed network portion aren't protected
- Designed to be only as secure as the fixed networks

Channel Hijack

- Protection against radio channel hijack relies on encryption.
- However, encryption is not used in some networks.

Problems with GSM Security - 2

Implicit Data Integrity

- No integrity algorithm provided

Unilateral Authentication

- Only user authentication to the network is provided.
- No means to identify the network to the user.

Weak Encryption Algorithms

- Key lengths are too short, while computation speed is increasing
- Encryption algorithms have been broken (theoretically and practically)
- Replacement of encryption algorithms is quite difficult

Unsecured Terminal

- IMEI is an unsecured identity
- Integrity mechanisms for IMEI were introduced late

New Security Features - 1

Network Authentication

- The user can identify the network

Explicit Integrity

- Data integrity is assured explicitly by use of integrity algorithms
- Also stronger confidentiality algorithms with longer keys

Network Security

- Mechanisms to support security within and between networks

Switch Based Security

- Security is based within the switch rather than the base station

IMEI Integrity

- Integrity mechanisms for IMEI provided from the start

New Security Features - 2

Secure Applications

- Provide security for applications resident on USIM

Fraud Detection

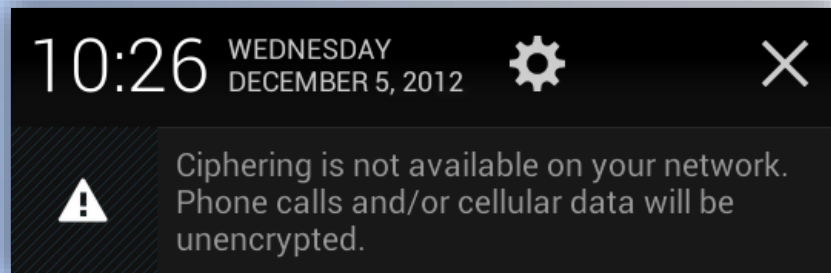
- Mechanisms to combating fraud in roaming situations

Flexibility

- Security features can be extended and enhanced as required by new threats and services

Visibility and Configurability

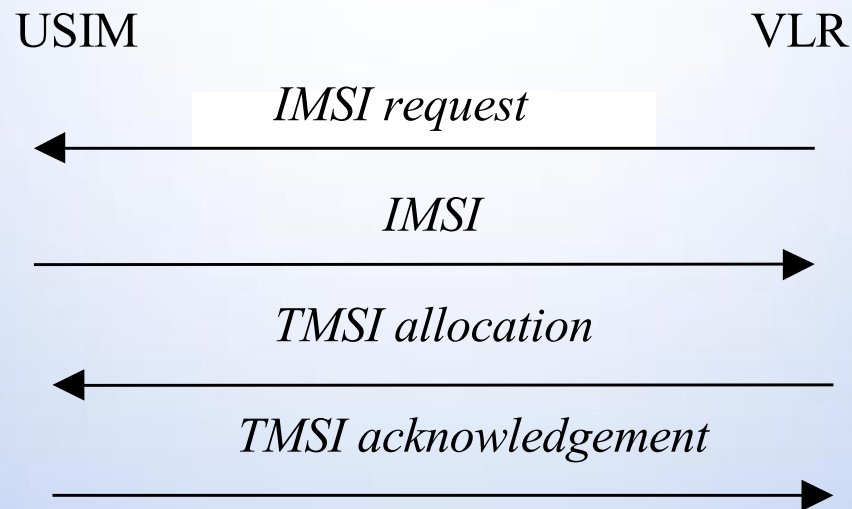
- Users are notified whether security is on and what level of security is available
- Users can configure security features for individual services



Summary of 3G Security Features - 1

User Confidentiality

- Permanent user identity IMSI, user location, and user services cannot be determined by eavesdropping
- Achieved by use of temporary identity (TMSI) which is assigned by VLR
- IMSI is sent in clear text when establishing TMSI



Summary of 3G Security Features - 2

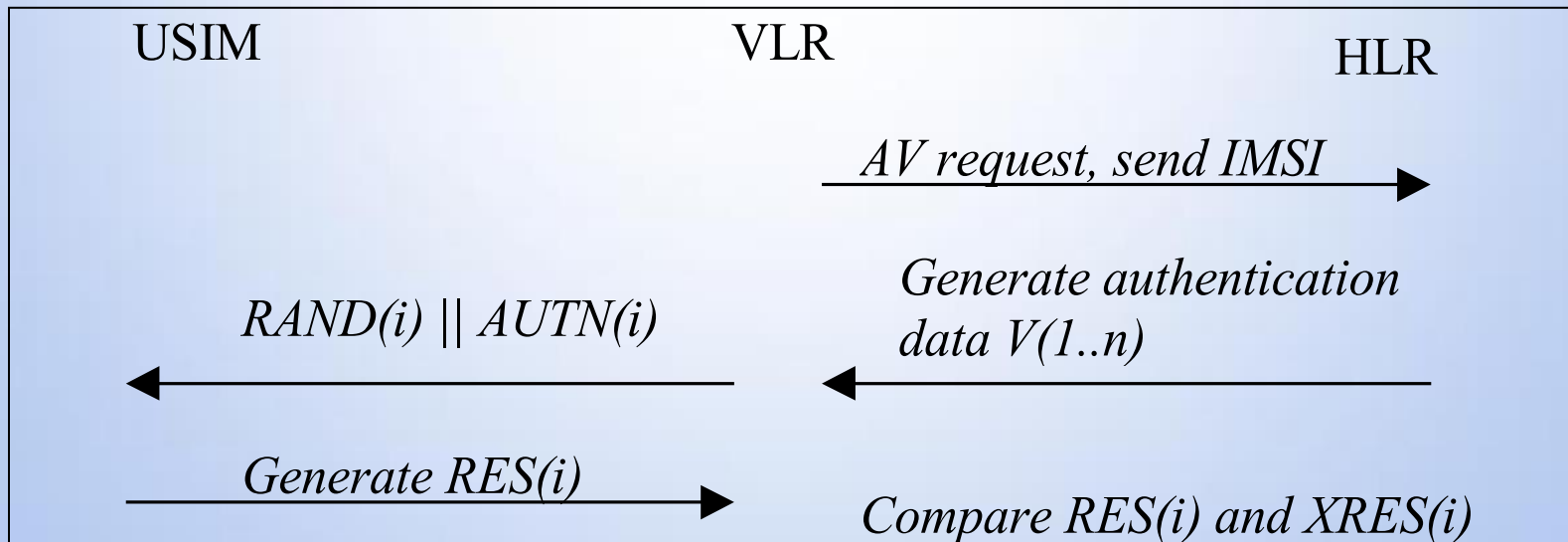
Mutual Authentication

During Authentication and Key Agreement (AKA) the user and network authenticate each other, and also they agree on cipher and integrity key (CK, IK). CK and IK are used until their time expires.

Assumption: Trusted Nodes, and trusted links between them.

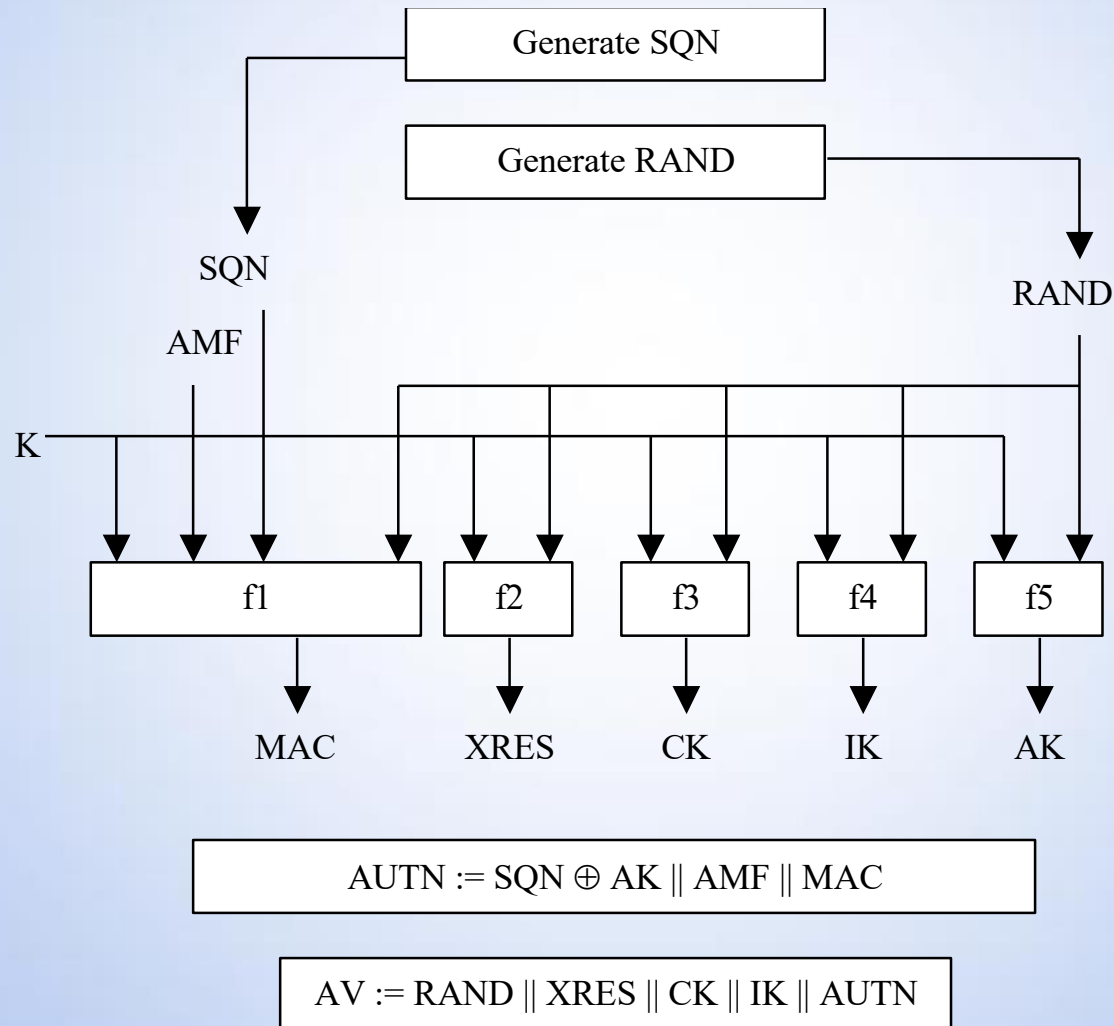
After AKA, security mode must be negotiated to agree on encryption and integrity algorithm.

AKA process:



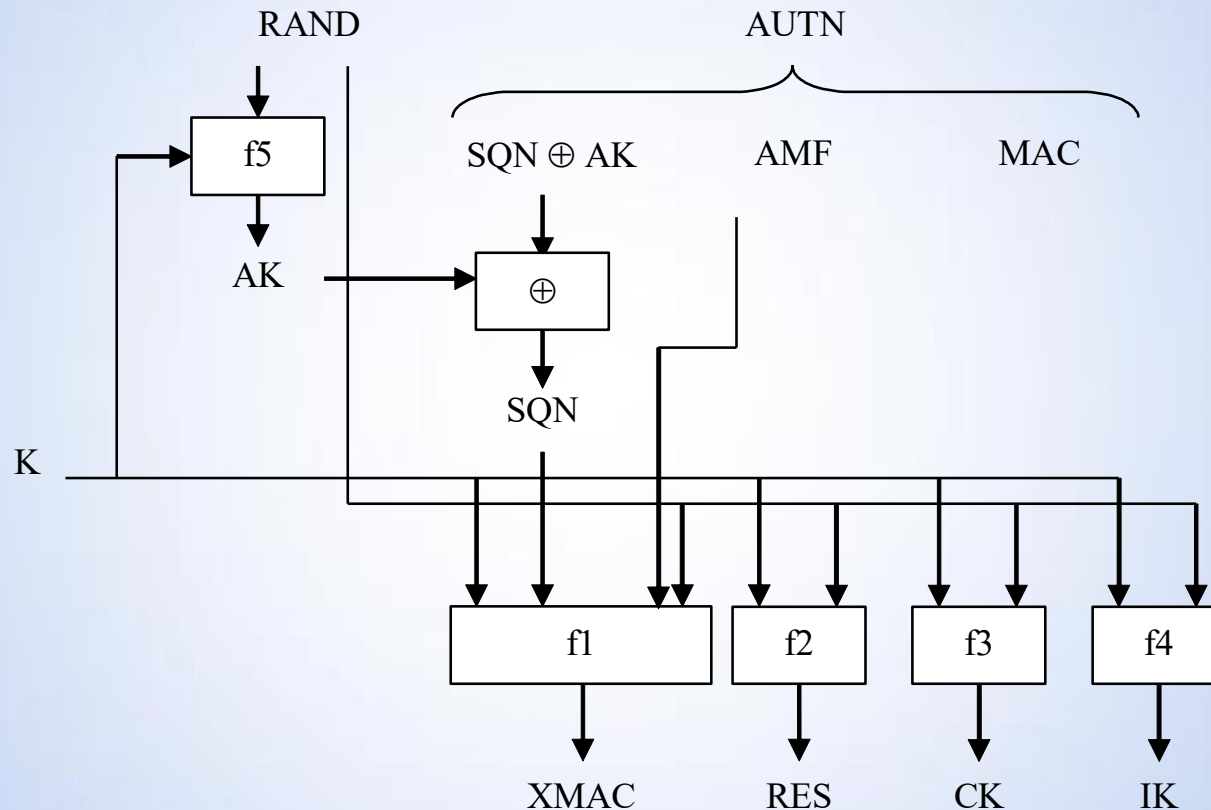
Summary of 3G Security Features - 3

Generation of authentication data at HLR:



Summary of 3G Security Features - 4

Generation of authentication data in USIM:



UMTS AKA Process

Step 1

In the beginning both the USIM and the Network are not authenticated. That means USIM does not know whether the network is a real network and network does not know whether the USIM is a valid Subscriber.

Step 2

Network starts the authentication procedure by sending the **User Authentication Request** with the parameter **RAND** and **AUTN**.

Step 3

After UE receives **RAND** and **AUTN**, the USIM first computes the anonymity key **AK = f5K (RAND)** and retrieves the SQN = $(SQN \oplus AK) \oplus AK$

After that UE computes **XMAC = f1K (SQN || RAND || AMF)** and compares with MAC.

- If both are different UE send user authentication **reject** back to the VLR/SGSN.
- If the USIM finds the SQN is not in the correct range, it sends **synchronization failure**.

Step 4

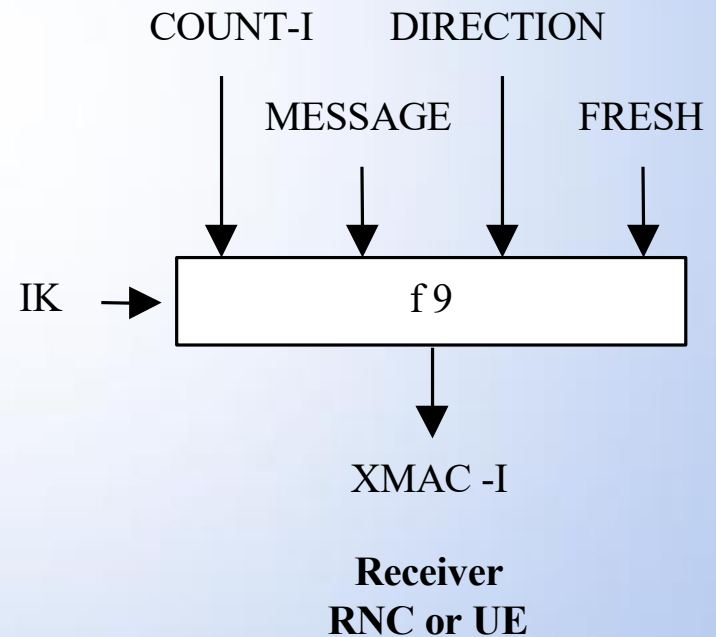
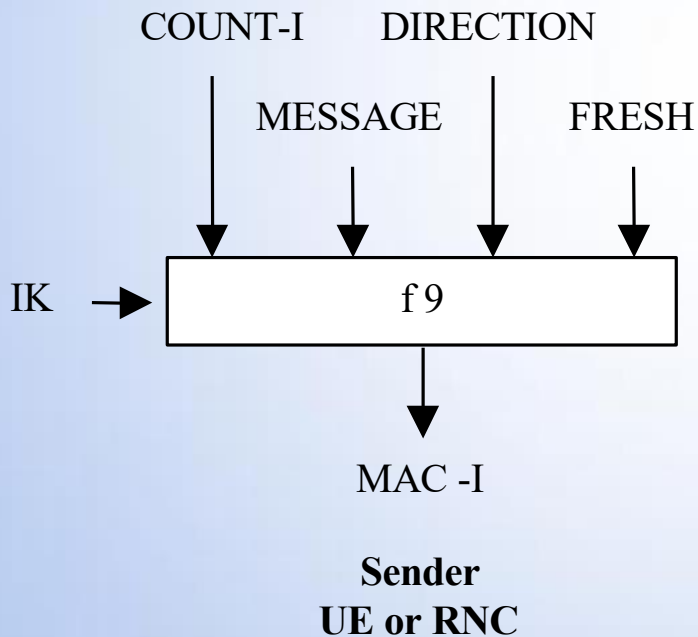
UE sends expected response **RES** to the **VLR/SGSN**.

If **RES = XRES**, then the authentication procedure completes

Summary of 3G Security Features - 5

Data Integrity

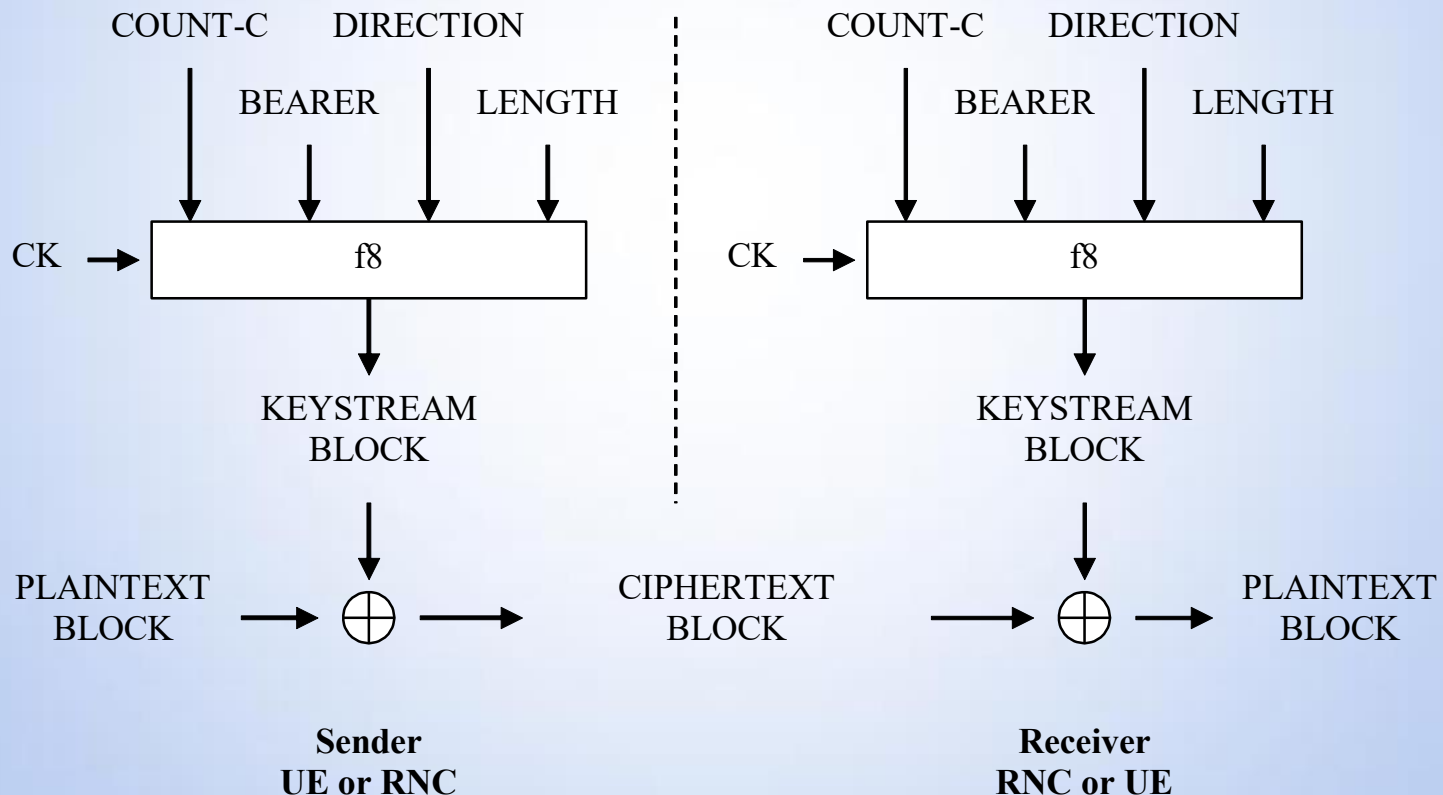
Integrity of data and authentication of origin of signaling data must be provided
The user and network agree on integrity key and algorithm during AKA and security mode set-up



Summary of 3G Security Features - 6

Data Confidentiality

- Signaling and user data should be protected from eavesdropping
- The user and network agree on cipher key and algorithm during AKA and security mode set-up



Summary of 3G Security Features - 7

IMEI

IMEI is sent to the network only after the authentication of Service Network
The transmission of IMEI is **not** protected

User-USIM Authentication

Access to USIM is restricted to authorized users
User and USIM share a secret key, PIN

Secure Applications

Applications resident on USIM should receive secure messages over the network

Visibility

Indication that encryption is on
Indication what level of security (2G, 3G) is available

Problems with 3G Security

- **IMSI is sent in cleartext** when allocating TMSI to the user
- The transmission of **IMEI is not protected**; IMEI is not a security feature
- A user can be enticed to camp on a **false BS**. Once the user camps on the radio channels of a false BS, the user is out of reach of the paging signals of Service Node.
- Hijacking outgoing/incoming calls in networks with **disabled encryption is possible**. The intruder poses as a man-in-the-middle and drops the user once the call is set-up

Questions?

4G: Long Term Evolution (LTE) *Security*

Growing Consumer Trends Worldwide

 Media Downloads	 VoIP
 Video Streaming	 Instant Messenger
 Online Gaming	 Email
 Social Networking	 Search

Apple iTunes

7
billion
songs downloaded

You Tube

5
billion
videos viewed per day

Facebook

2.9
billion
users

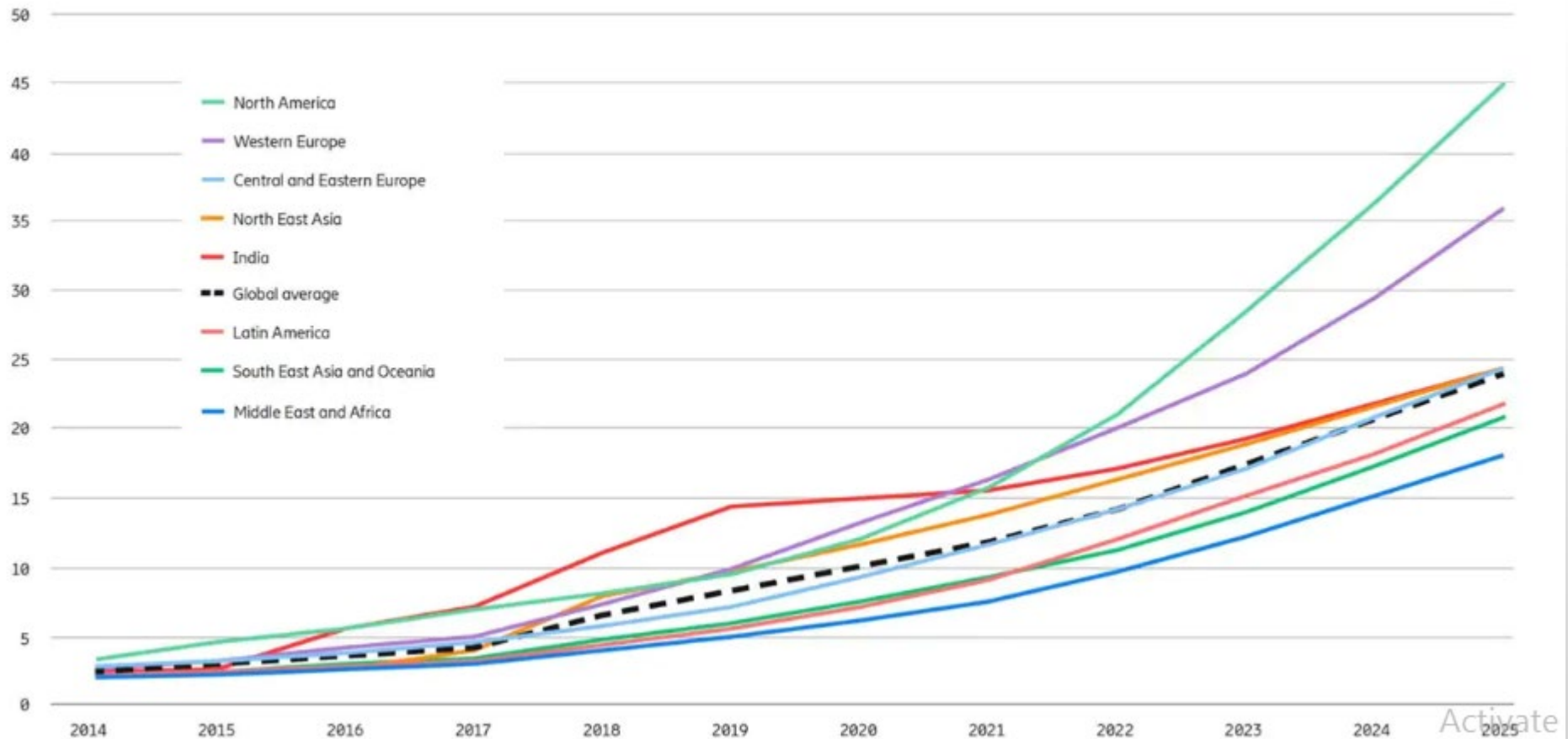
Google Search

8.5
billion
daily

Anytime, anywhere and on any device

Implication: Mobile Data Traffic Is Exploding.....

Mobile data traffic per smartphone (GB per month)



Access Network Evolution

The driver for LTE is data...

3GPP

EDGE

WDCMA

HSPA

LTE

3GPP2

CDMA
1X

EV-DO

DOrA

LTE

But voice and SMS are
still a necessity for mobile users



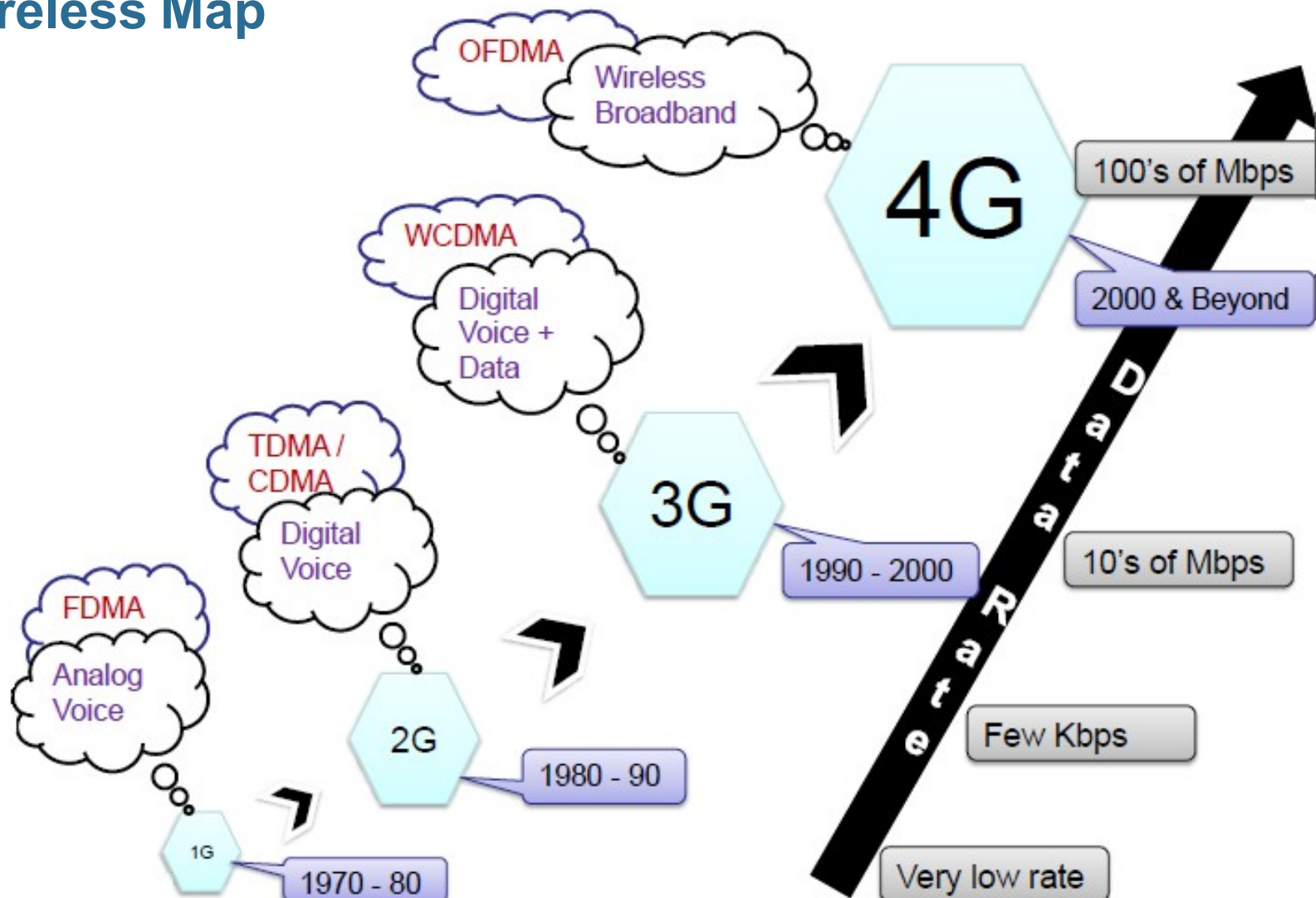
3GPP Evolution

- Release 99 (Mar. 2000): UMTS/WCDMA
- Rel-5 (Mar. 2002): HSDPA
- Rel-6 (Mar. 2005): HSUPA
- Rel-7 (2007): DL MIMO, optimized real-time services (VoIP, gaming, ...)
- Long Term Evolution (LTE) / 3.95G
 - 3GPP work on the Evolution of the 3G Mobile System started in November 2004.
 - Standardized finalized and approved in January 2008.
- LTE-Advanced standardized finalized and approved in January 2011

3GPP Evolution

	WCDMA (UMTS)	HSPA (HSDPA/HSUPA)	HSPA+	LTE	LTE Advanced
Transmissão do Downlink	384 Kbps	14 Mbps	28 Mbps	100 Mbps	1 Gbps
Transmissão do Uplink	128 Kbps	5.7 Mbps	11 Mbps	50 Mbps	500 Mbps
Latência (Round Trip Time-RTT)	150 ms	100 ms	50 ms(Max)	10 ms	10 ms
Realese do 3GPP	Rel 99/4	Rel 5/6	Rel 7	Rel 8/9	Rel 10/11
Acesso	CDMA	CDMA	CDMA	OFDMA/SC-FDMA	OFDMA/SC-FDMA

Wireless Map



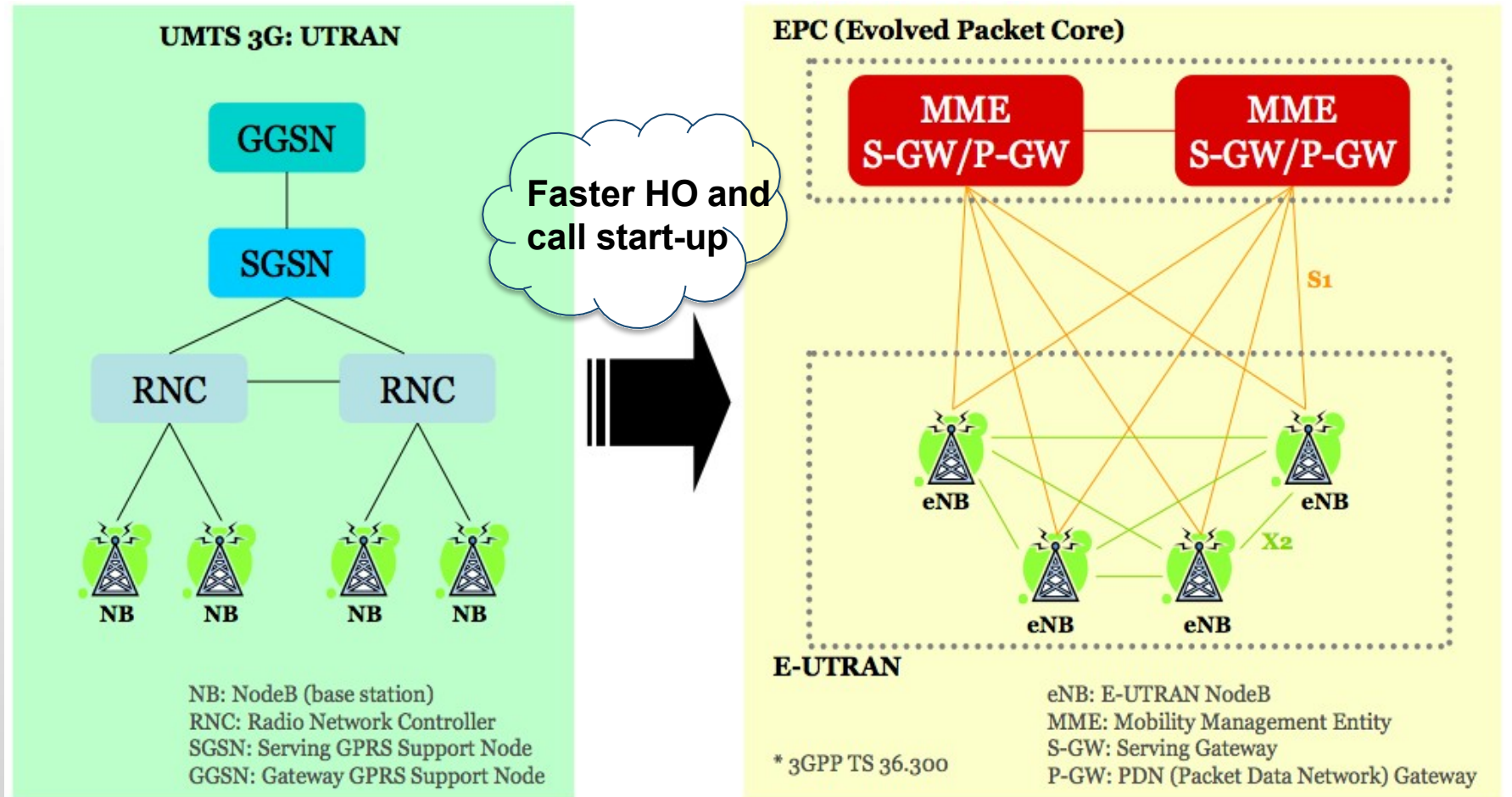
Why 4G ?

Pre – 4G	4G
Voice communication	VoIP, high quality video conferencing
SMS, MMS	Video messaging
Internet browsing	Super-fast internet
Downloadable games	Online gaming with mobility
Downloadable video	High quality audio & video streaming
No TV service	Broadcast TV on-demand
Peer-to-peer messaging	Wide-scale distribution of video clips
	Mobile payment
	File transfer
	Many other innovative ideas

LTE Targets

- Packet-Domain-Services only (e.g. VoIP) → upon LTE, TCP/IP- based layers
- Higher peak data rate/ user throughput → 100 Mbps DL/50 Mbps UL @20MHz bandwidth
- Reduced delay/latency → latency<10ms
- Improved spectrum efficiency → up to 200 active users in a cell @5MHz bandwidth
- Mobility → optimized for low-mobility (up to 15Km/h), supported with high performance for medium mobility (up to 120 Km/h), supported for high mobility (up to 500 Km/h)
- Multimedia broadcast & multicast services
- Spectrum flexibility
- Multi-antenna configuration
- Coverage → up to 30 Km
- Frequency range: 800 MHz to 3.5GHz

Network Architecture



Network Architecture – E-UTRAN

- User Equipment

- Evolved Node B (eNB)

 - Functionalities:*

- 1) Resource management (allocation and HO)
- 2) Admission control
- 3) Application of negotiated UL QoS
- 4) Cell information broadcast
- 5) Ciphering/deciphering of user and control plane data

Network Architecture - Evolved Packet Core

- Mobility Management Entity → key control-node for the LTE access-network.

Functionalities:

- 1) Idle mode UE tracking and paging procedure including retransmissions
- 2) Bearer activation/deactivation process and choice of the SGW for a UE
- 3) Authentication of users : it checks the authorization of the UE to camp on the service provider's Public Land Mobile Network (PLMN)
- 4) Control plane function for mobility between LTE and 2G/3G access

Network Architecture - Evolved Packet Core

- Serving Gateway (SGW) → *Functionalities*:
 - 1) Routing and forwarding user data packets
 - 2) Acts as mobility anchor for the user plane during inter-eNB handovers and for mobility between LTE and other 3GPP
 - 3) For idle state UEs, terminates the DL data path and triggers paging when DL data arrives for the UE

Network Architecture - Evolved Packet Core

- Packet Data Network Gateway → *Functionalities*:

- 1) Provides connectivity to the UE to external packet data networks (IP addresses.). A UE may have simultaneous connectivity with more than one PDN GW for accessing multiple PDNs
- 2) Performs policy enforcement, packet filtering for each user, charging support, lawful Interception and packet screening

LTE PHY Layer

+ Includes methods for contrasting distortion due to multipath:

- a) OFDM

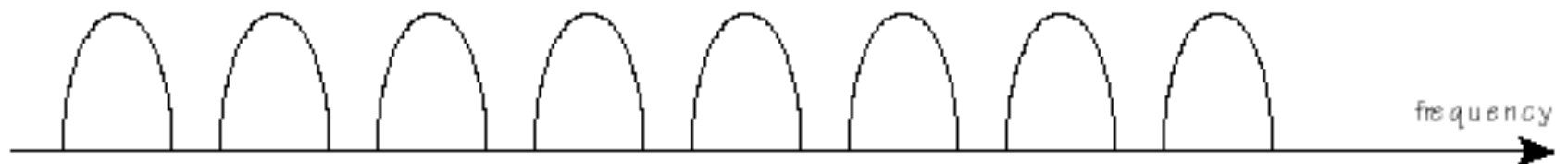
- b) MIMO

+ New access method scheme:

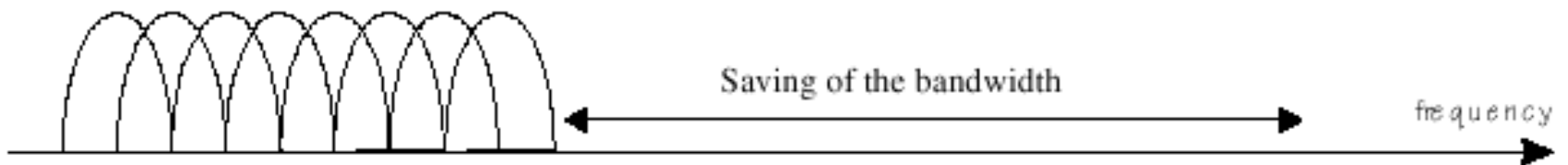
- a) OFDMA

- b) SC-FDMA

FDM vs. OFDM

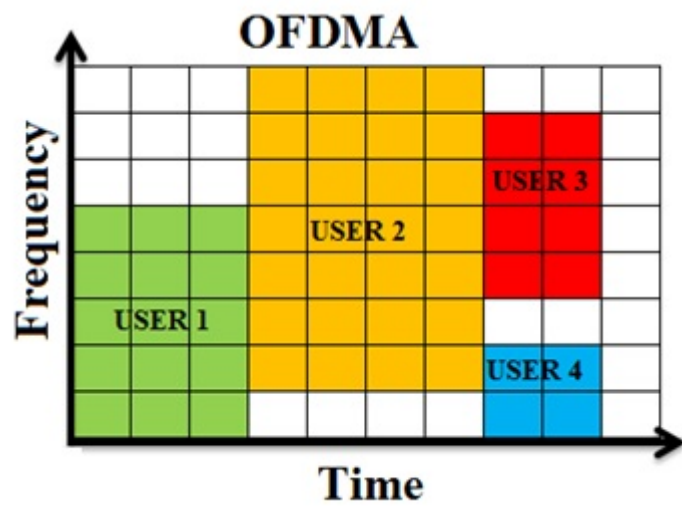
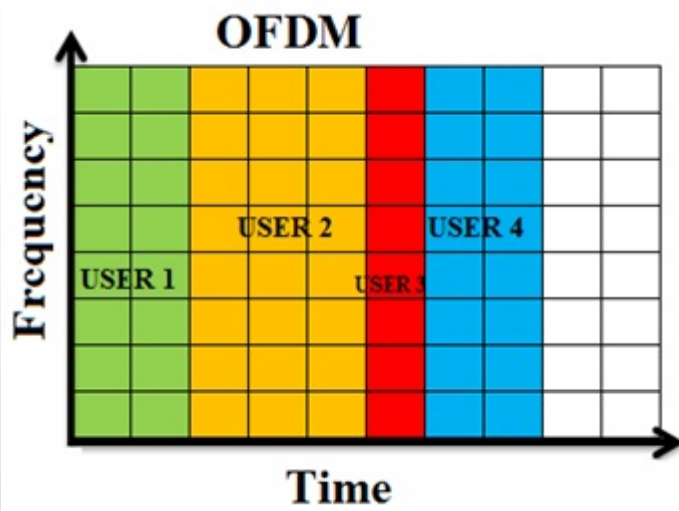


Conventional Frequency Division Multiplex (FDM) multicarrier modulation technique



Orthogonal Frequency Division Multiplex (OFDM) multicarrier modulation technique



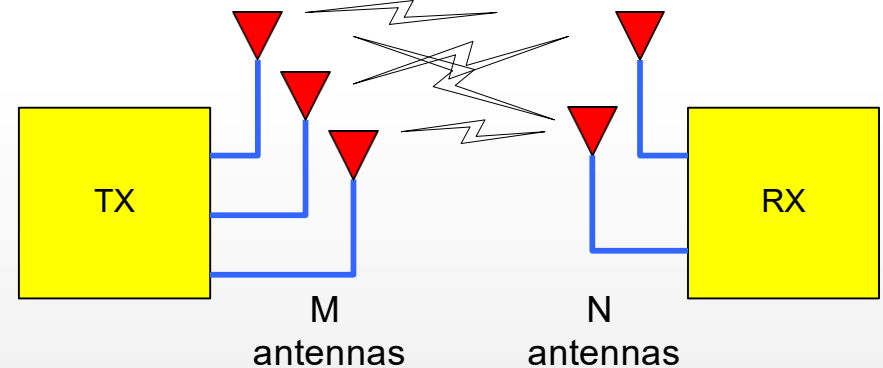


Multiple-Input Multiple-Output (MIMO)

- Future wireless services require high data rates and high signal quality
- The wireless resources such as the bandwidth are scarce
- Wireless channels have a lot of impairments such as fading, shadowing, and multiuser interference
- One solution is the use of Diversity achieving schemes
- Spatial diversity is of special interest!

MIMO

- Signal transmitted from multiple antennas (Multiple In)
- Signal received by multiple antennas (Multiple Out)



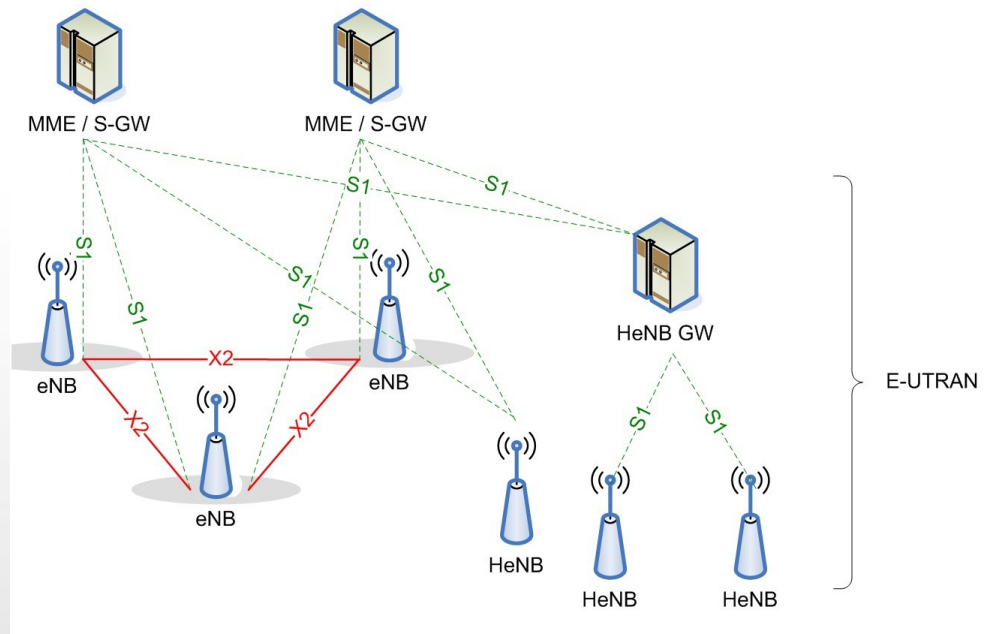
- Receiver combines the received signals and optimally combine energy from $M \times N$ channels
- Advantages:
 - 1) Higher data rate $\rightarrow$ more than one flow simultaneously
 - 2) Spatial diversity $\rightarrow$ taking advantage from multiple paths $\rightarrow$ multipath as a resource

LTE Handover

- UE is moving from one place to other connecting to different eNB. In turn eNB's are connected to MME and SGW.
- MME is connected to SGW and eNB's. So when UE move from one area to other the following combinations arise : -
 - **Inter eNB handover or Intra MME** handover: That means UE is moving from one eNB to other where both eNB's are connected to same MME.
 - **Inter eNB handover with MME change**: Here UE is moving from one eNB to other where each of the eNB's is connected to different MME's.
 - **Inter eNB handover with MME and SGW change**: Here UE is moving from one eNB to other where each eNB is connected to different MME and SGW.

LTE Security

Threats



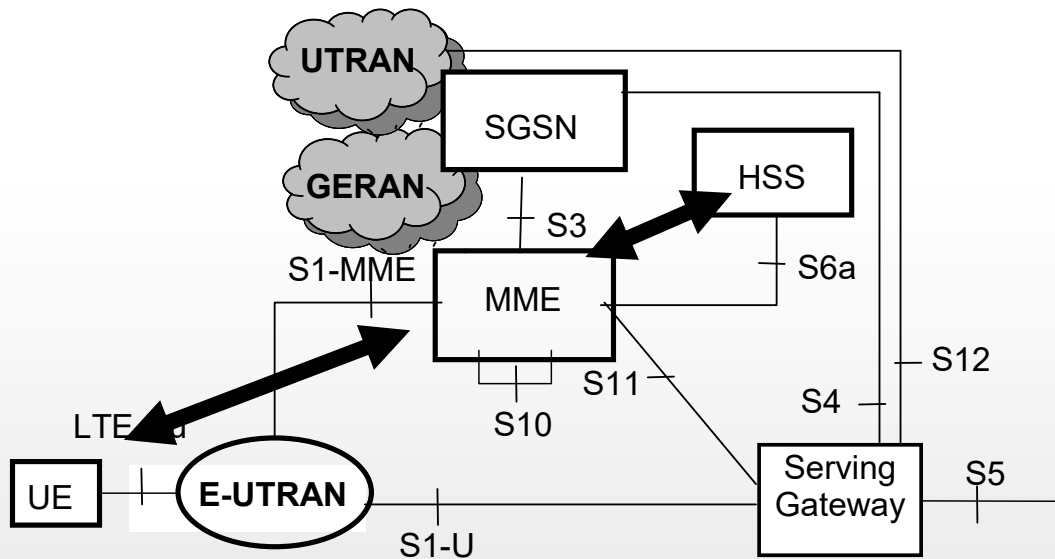
- Cloning of credentials
- Physical tampering
- Fraudulent software updates
- Man-in-the-middle attacks
- Denial of service against core network
- Eavesdropping (identity theft, privacy breaches, ...)

LTE Security

Characteristics of LTE Security

- Re-use of UMTS Authentication and Key Agreement (AKA)
- Use of USIM required
- Extended key hierarchy
- Possibility for longer keys
- Greater protection for backhaul
- Integrated interworking security for legacy networks

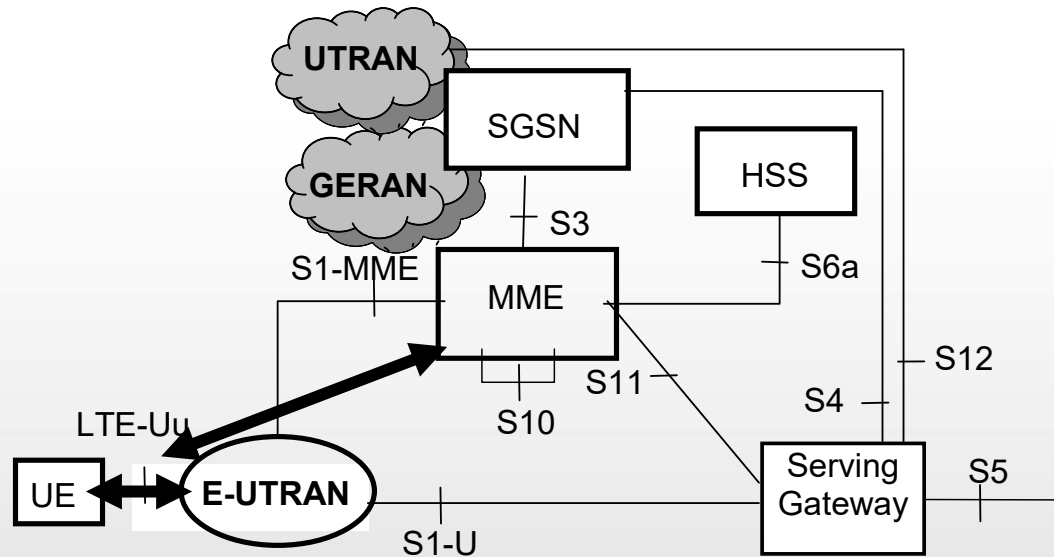
Authentication and Key Agreement (AKA)



- Home Subscriber Server (HSS)
- Mobility Management Entity (MME)

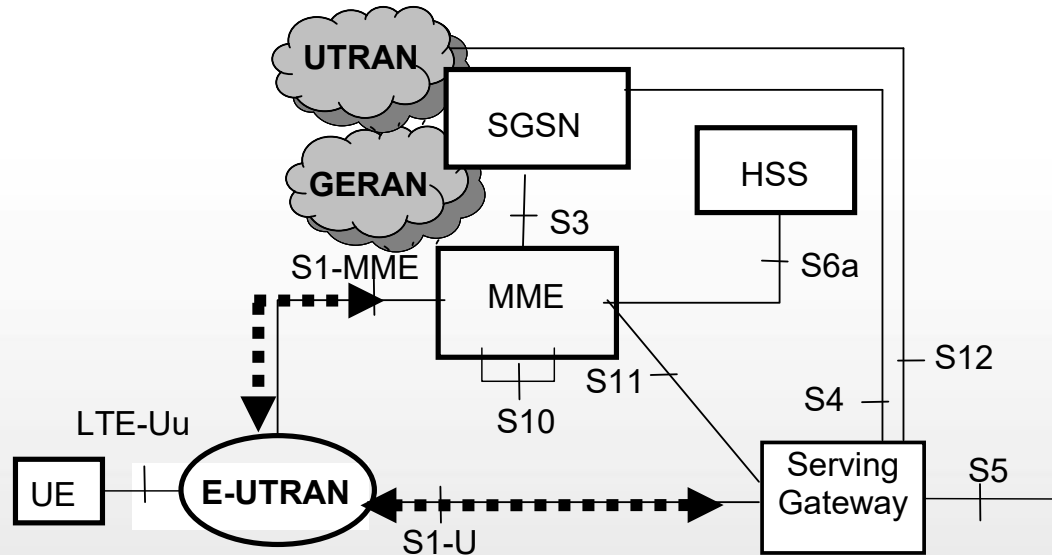
- HSS generates authentication data and provides it to MME
- Challenge-response authentication and key agreement procedure between MME and UE

Confidentiality and Integrity of Signaling



- RRC signaling between UE and E-UTRAN
 - Encryption on PDCP layer
- NAS signaling between UE and MME
 - Used to convey non-radio signaling between the UE and MME
 - Encrypted using a shared NAS security key

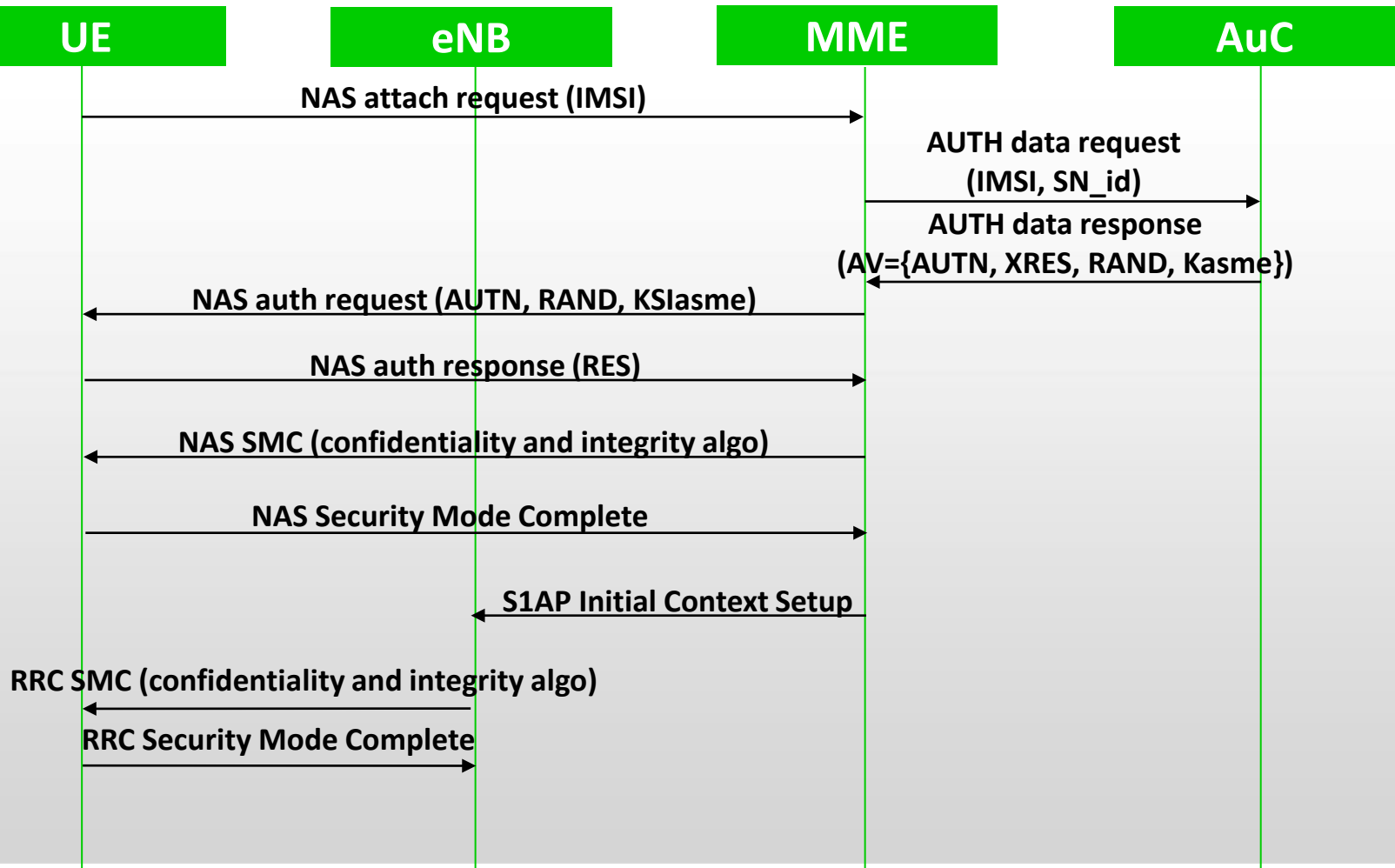
User Plane Confidentiality



S1 protection is not UE-specific

- (Enhanced) network domain security mechanisms
 - Based on IPSec
- Integrity protection not available

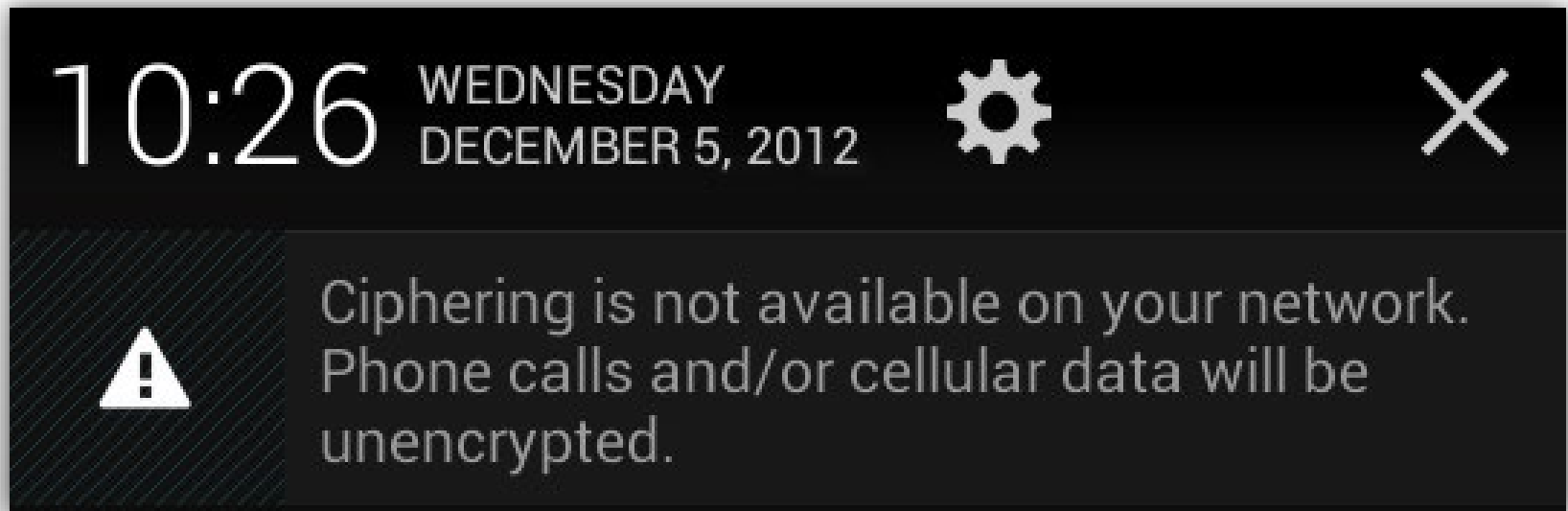
LTE Authentication and Key Agreement



Indication of access network encryption

Indication of access network encryption

user is informed whether confidentiality of user data is protected on the radio access link in particular when non-ciphered calls are set-up



Security Algorithms

LTE Security Algorithms (1/2)

- **Three separate algorithms specified**
 - In addition to one NULL algorithm
- **Current keylength 128 bits**
 - Possibility to extend to 256 in the future
- **Confidentiality protection of NAS/AS signalling recommended**
- **Integrity protection of NAS/AS signalling mandatory**
- **User data confidentiality protection recommended**
- **Ciphering/Deciphering applied on PDCP and NAS**

- **128-EEA1/EIA1**

Based on SNOW 3G: stream cipher; key stream produced by Linear Feedback Shift Register (LFSR) and a Finite State Machine (FSM)

Low power consumption

- **128-EEA2/EIA2**

AES block cipher

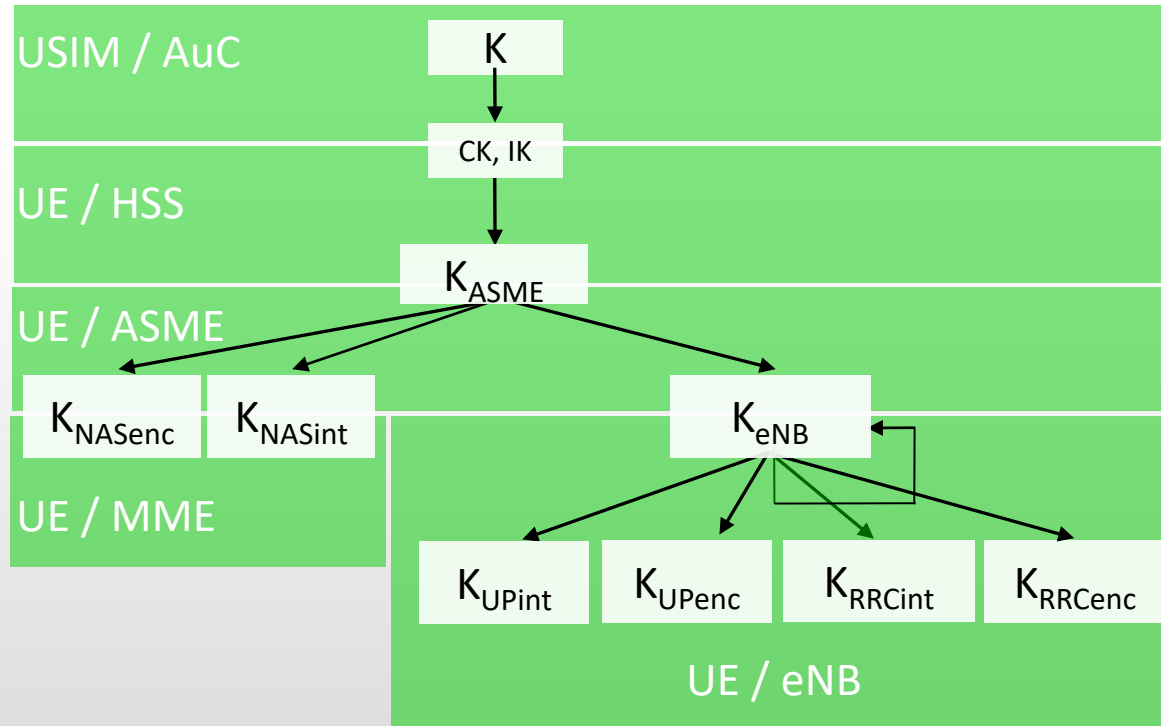
- Counter (CTM) Mode for ciphering
- GMAC Mode for MAC-I creation (integrity)

- **128-EEA3/EIA3 (Rel-11 onwards)**

Based on ZUC (Zu Chongzhi): stream cipher

Developed by Data Assurance and Communication Security Research Center of Chinese Academy of Sciences (DACAS)

Deeper Key hierarchy in LTE



- **Faster handovers and key changes, independent of AKA**
- **Added complexity in handling of security contexts**
- **Makes security breaches local**

Backhaul Security

Base stations becoming more powerful

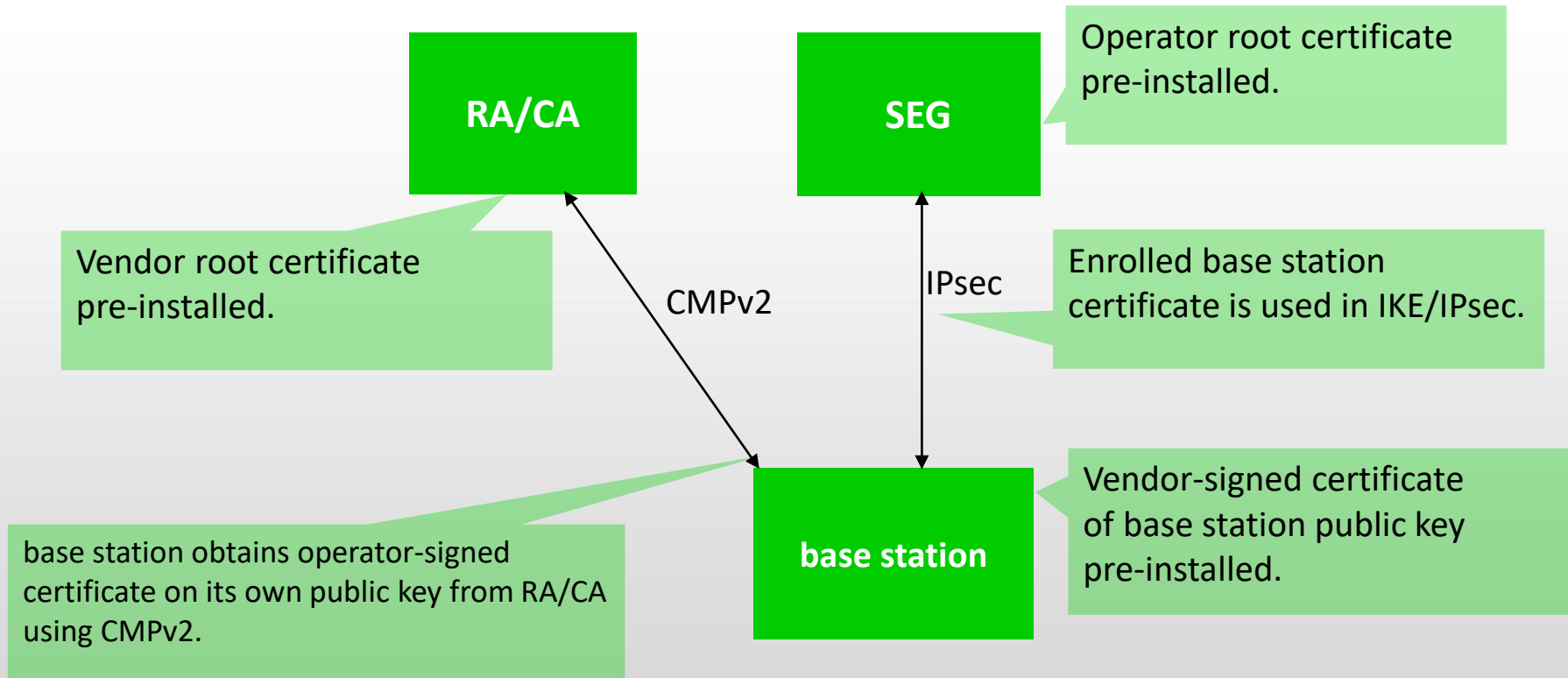
LTE eNode B includes functions of NodeB and RNC

Coverage needs grow constantly **Infrastructure sharing**



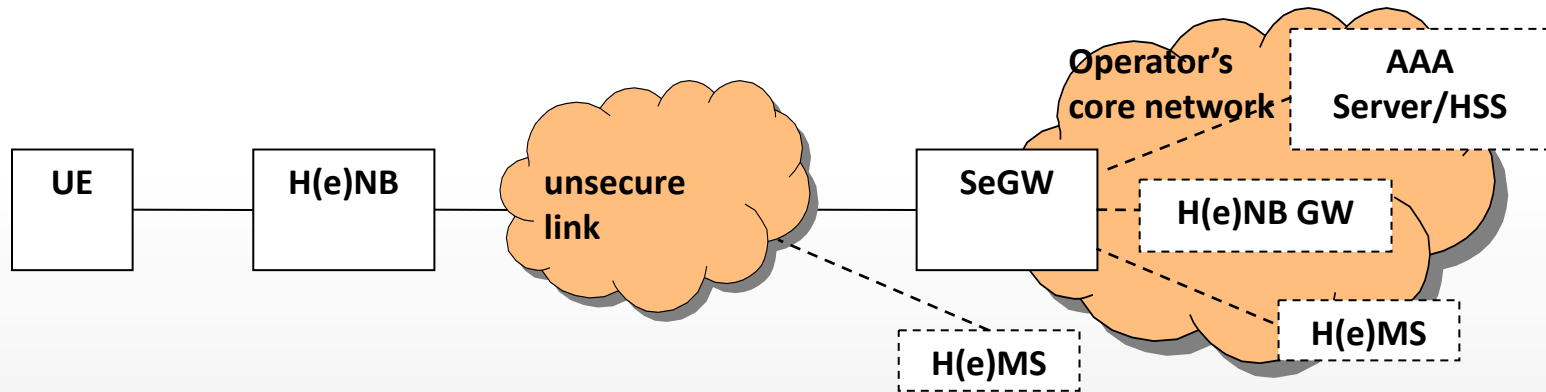
Not always possible to trust physical security of eNB
Greater backhaul link protection necessary

Certificate Enrollment for Base Stations



Picture from 3GPP TS 33.310

Home (e)NB Security Architecture



Security Gateway (SeGW)

- Element at the edge of the core network terminating security association(s) for backhaul link between H(e)NB and core network
- H(e)NB access operator's core network via a Security Gateway (SeGW)
- Security tunnel established between H(e)NB and SeGW to protect information transmitted in backhaul link

H(e)MS – Home (e) NodeB Management System

Management server that configures the H(e)NB according to the operator's policy, installs software updates on the H(e)NB etc.

Conclusions

- Security is a major point of interest from GSM (2G) up to LTE (4G)
- GSM/UMTS Security: continues to evolve, recent introduction of A5/3 (planned before attack on old A5/1 succeeded)
- LTE Security: building on GSM and UMTS Security with newer security algorithms, longer keys, Extended key hierarchy
- Security aspects taken into consideration each time the system evolves



5G Security

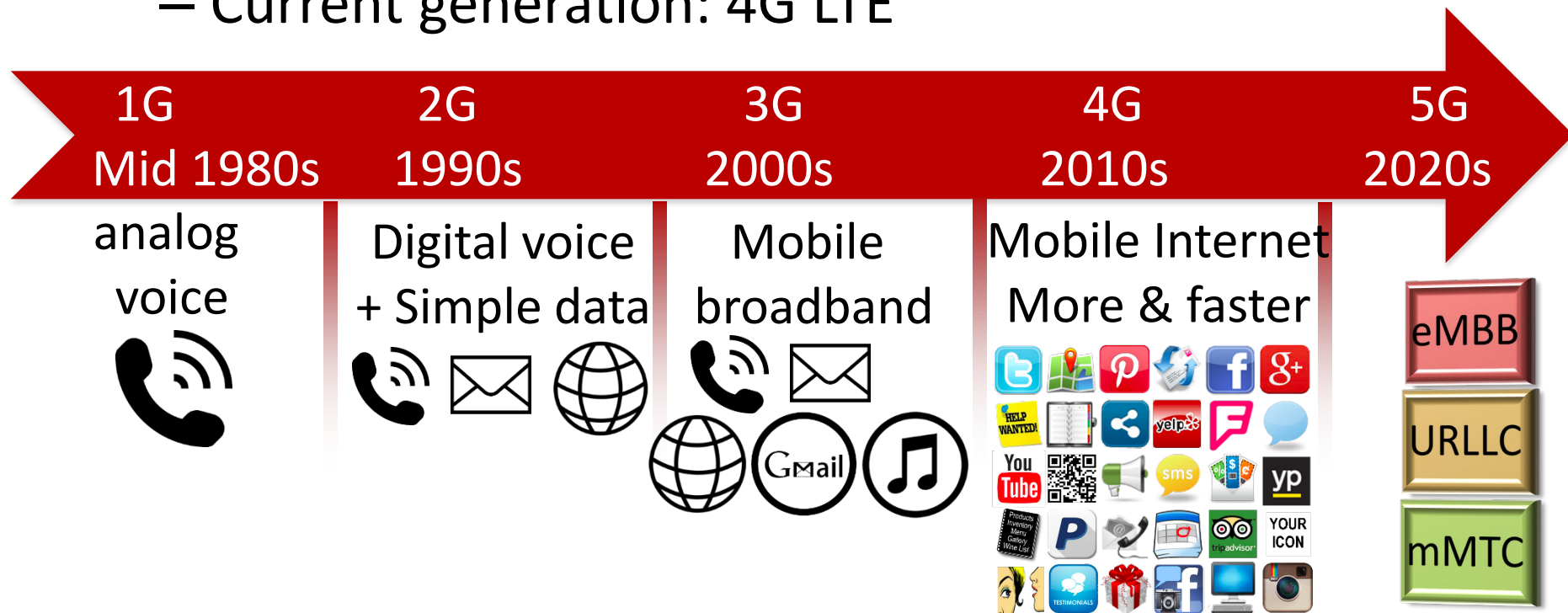
Kavinga Yapa Abeywardena

Lecturer - Computer Systems Engineering Department

Faculty of Computing - SLIIT

What is 5G?

- Next generation mobile network
 - Current generation: 4G LTE



Enhanced Mobile
Broadband



Gigabytes in seconds



3D video, UHD Screens



Work and Play in the cloud



Smart Home/
Building



Voice



Augmented reality



Industrial Automation



Smart City

Mission Critical,
health



Self-driving Cars

Massive Machine
Type Communication

Ultra-Reliable and Low
Latency Communication

Pull from User Demands: Mobile Internet Anytime, Anywhere



In-building



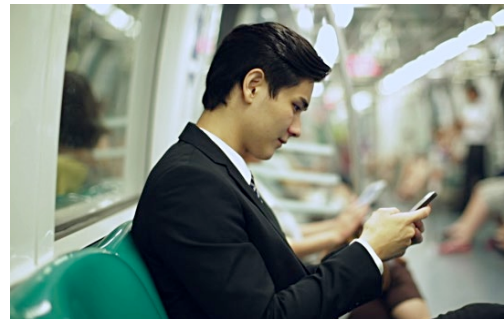
Outdoor



Walking



Driving



Subway



High-speed train

No. 1 disruptive technology in the past decade
Smartphones (not PCs): primary access points to Internet

Pull from User Demands in 5G

4G
2010s

5G
2020s

Much Faster
10Gps peak rate
< 1ms latency



Super-connected
10000x traffic
1000x



10-100x devices

Higher mobility
300+ Kmh

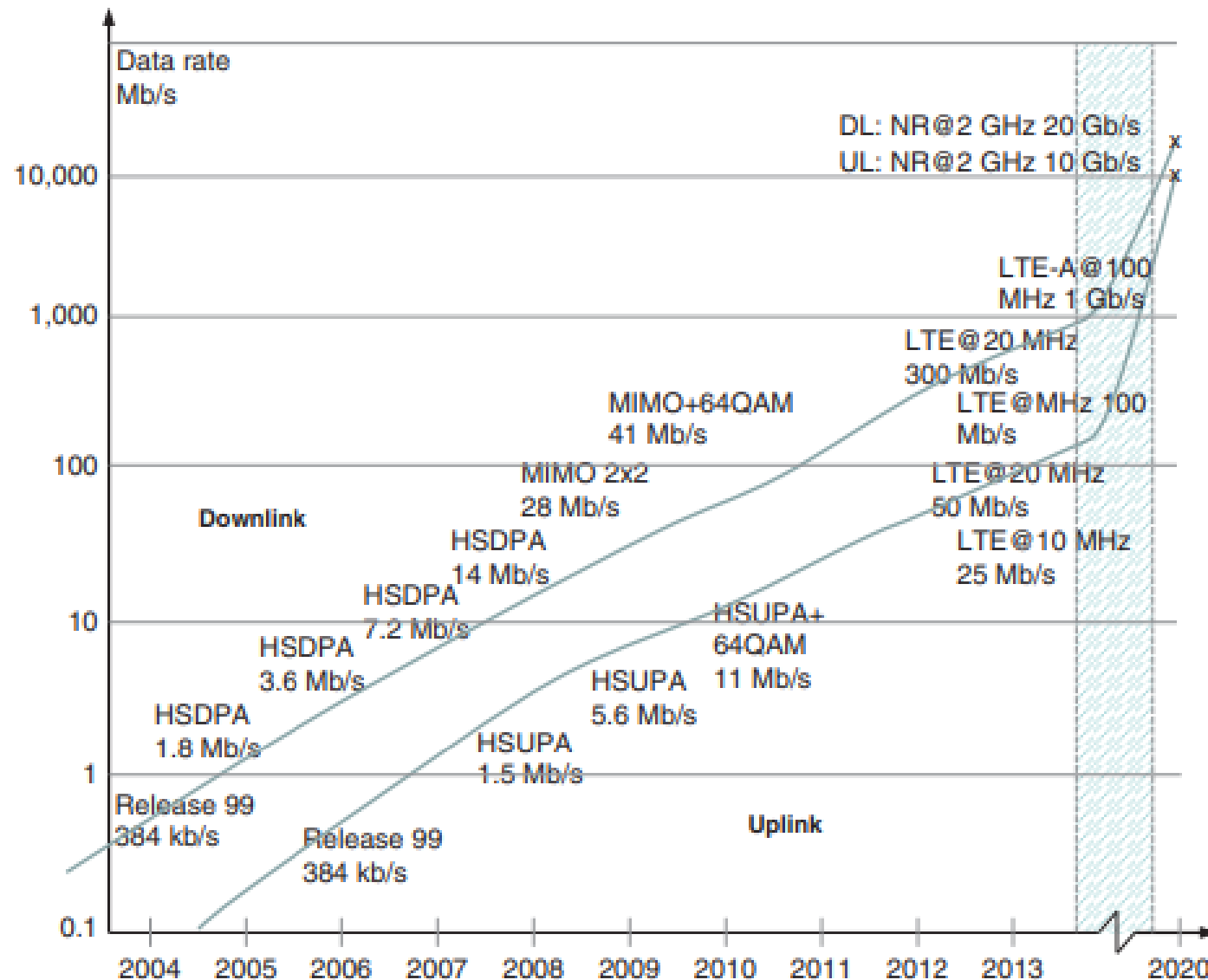


Ultra-reliable
99.999%

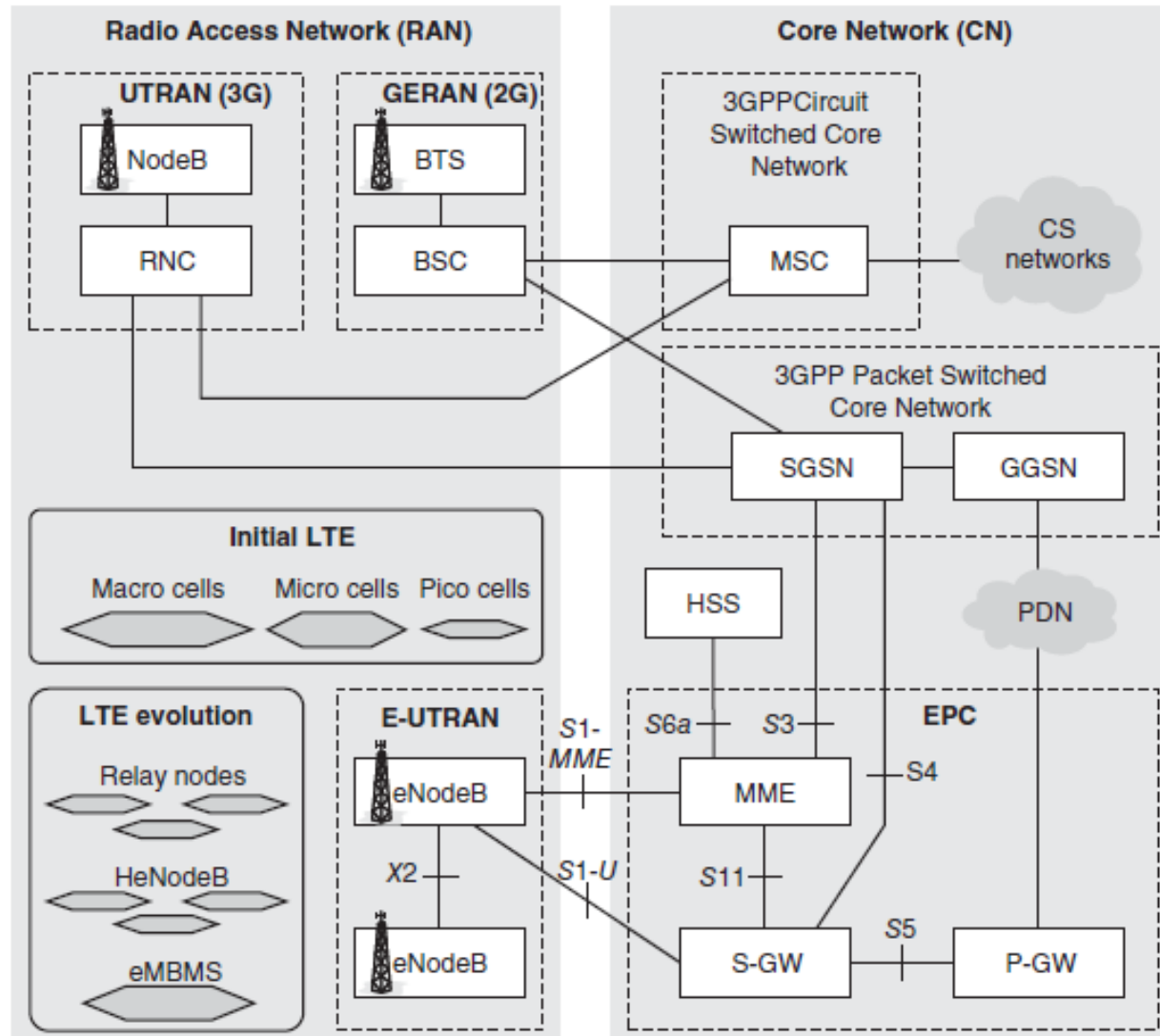


Energy-efficient
...

Development of Mobile Data Rates

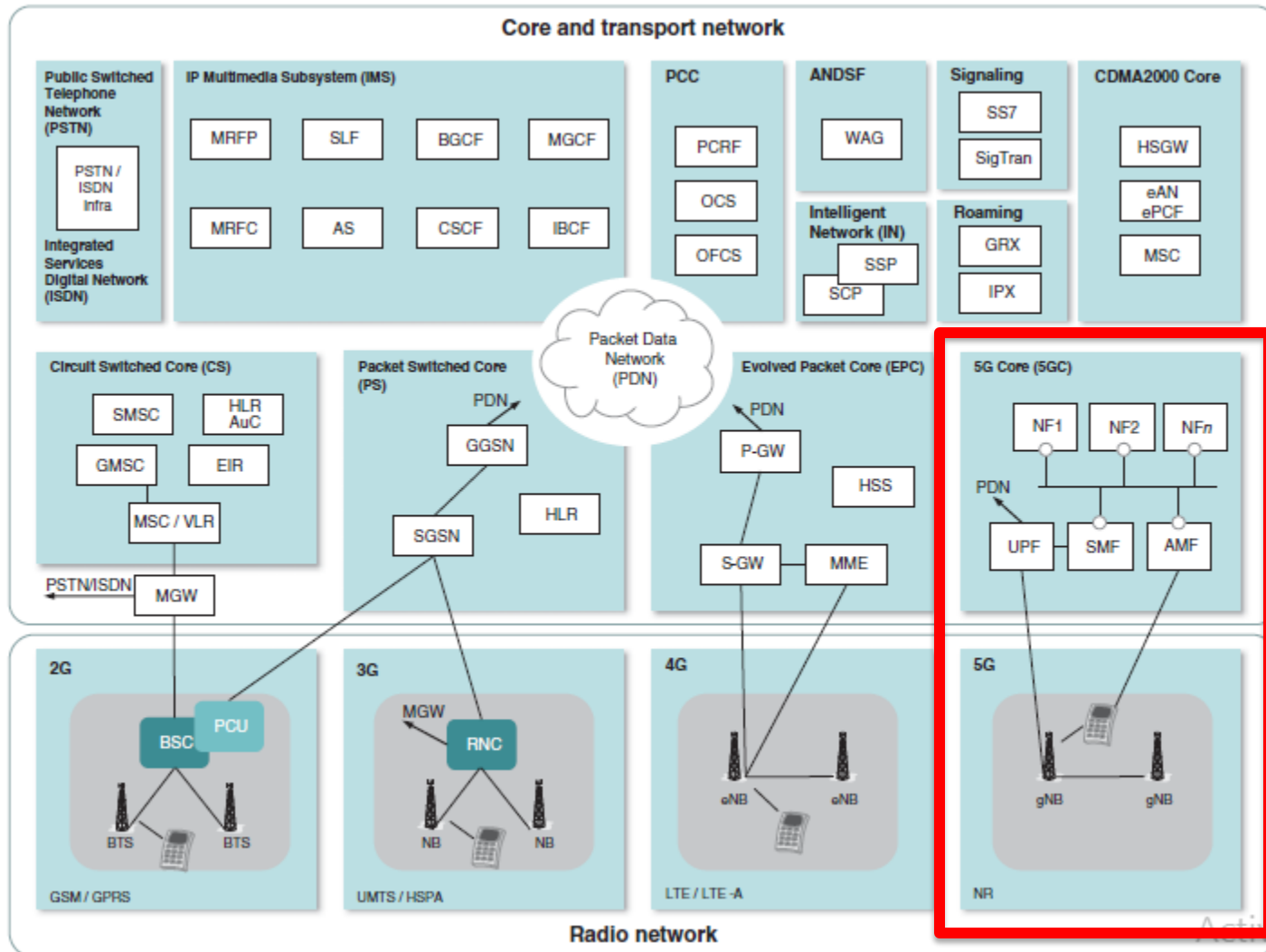


Before 5G Architecture



2G, 3G, and 4G network architecture prior to 5G deployment.

5G Architecture



High-level snapshot of the mobile network evolution and the supporting core elements





5G Architecture (Cont.)

Access & Mobility Function

- **Subscriber Mobility**
- **Registration & Security**

Unified Data Management

- **Access Authorization (Keys)**
- **AMF Tracking**
- **Data Profiles**

Policy Control Function

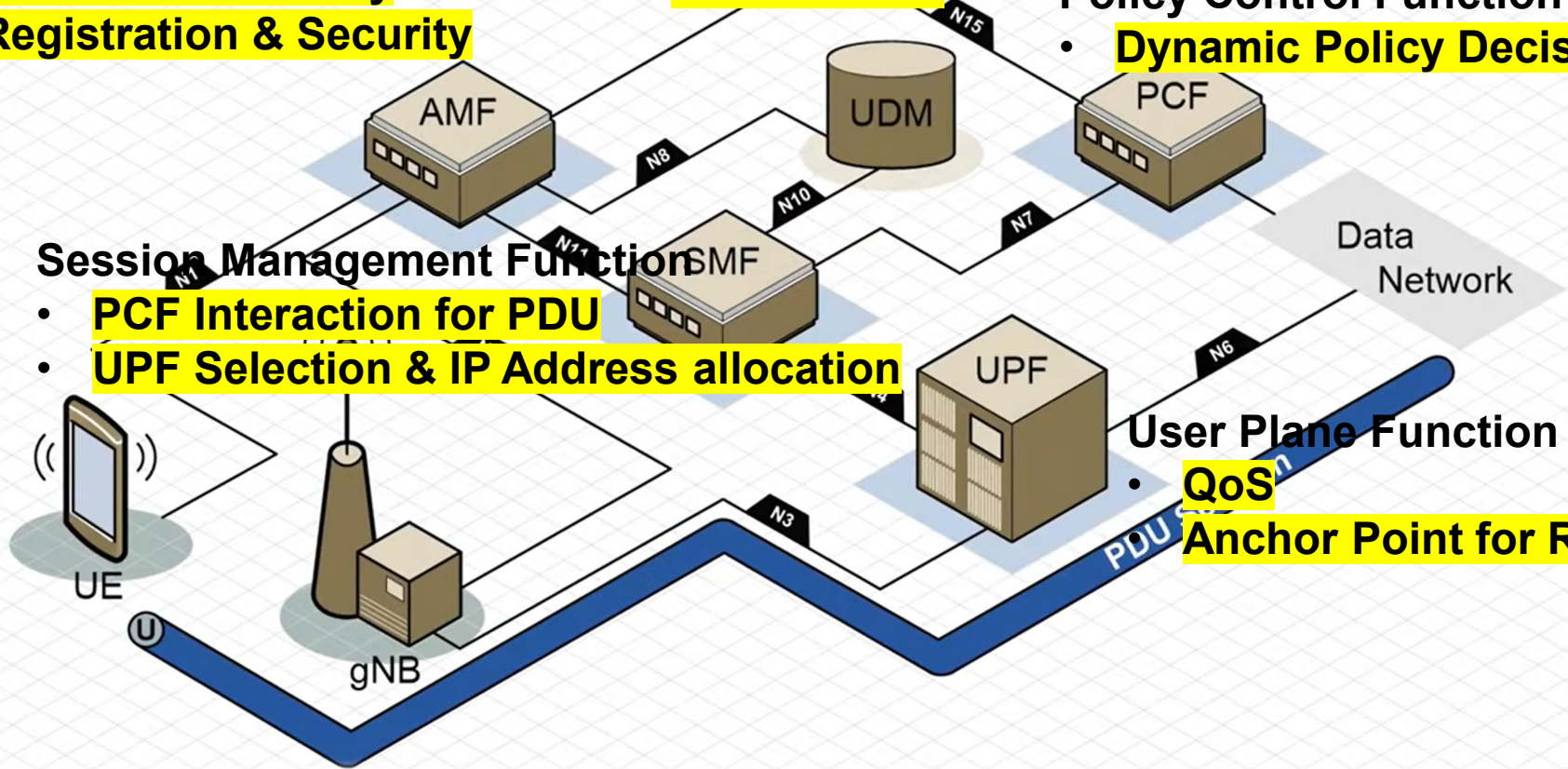
- **Dynamic Policy Decisions**

Session Management Function

- **PCF Interaction for PDU**
- **UPF Selection & IP Address allocation**

User Plane Function

- **QoS**
- **Anchor Point for RAN**





Questions to Answer

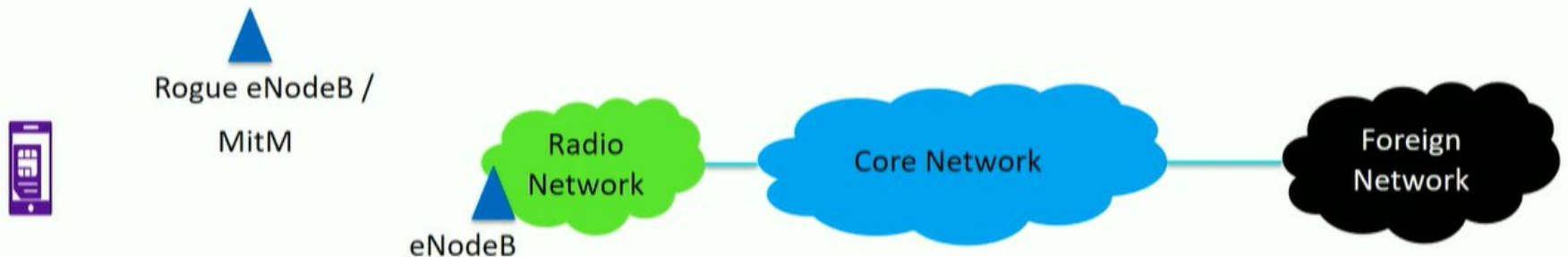
- Understand critical demands in 5G
- Learn the state-of-the-art technologies
 - Giant players: Industry, standardization, policy makers, research (academia)
 - Wireless technologies (PHY, spectrum, radio)
 - Architecture (radio/core architecture, SDN/NFV, edge computing, cloudlet, network slicing...)
 - Network protocol and functions (control and management)

Questions to Answer

- Understand critical demands in 5G
 - Driven by 5G apps
 - Drives to 5G technology evolution
 - On extremely low latency, scalability, efficiency, reliability, **security** ...

Potential Threats on 4G

- IMSI in clear
- Temporary identity not changed
- No UP integrity protection
- Bid-down to GSM



5G Security Requirements

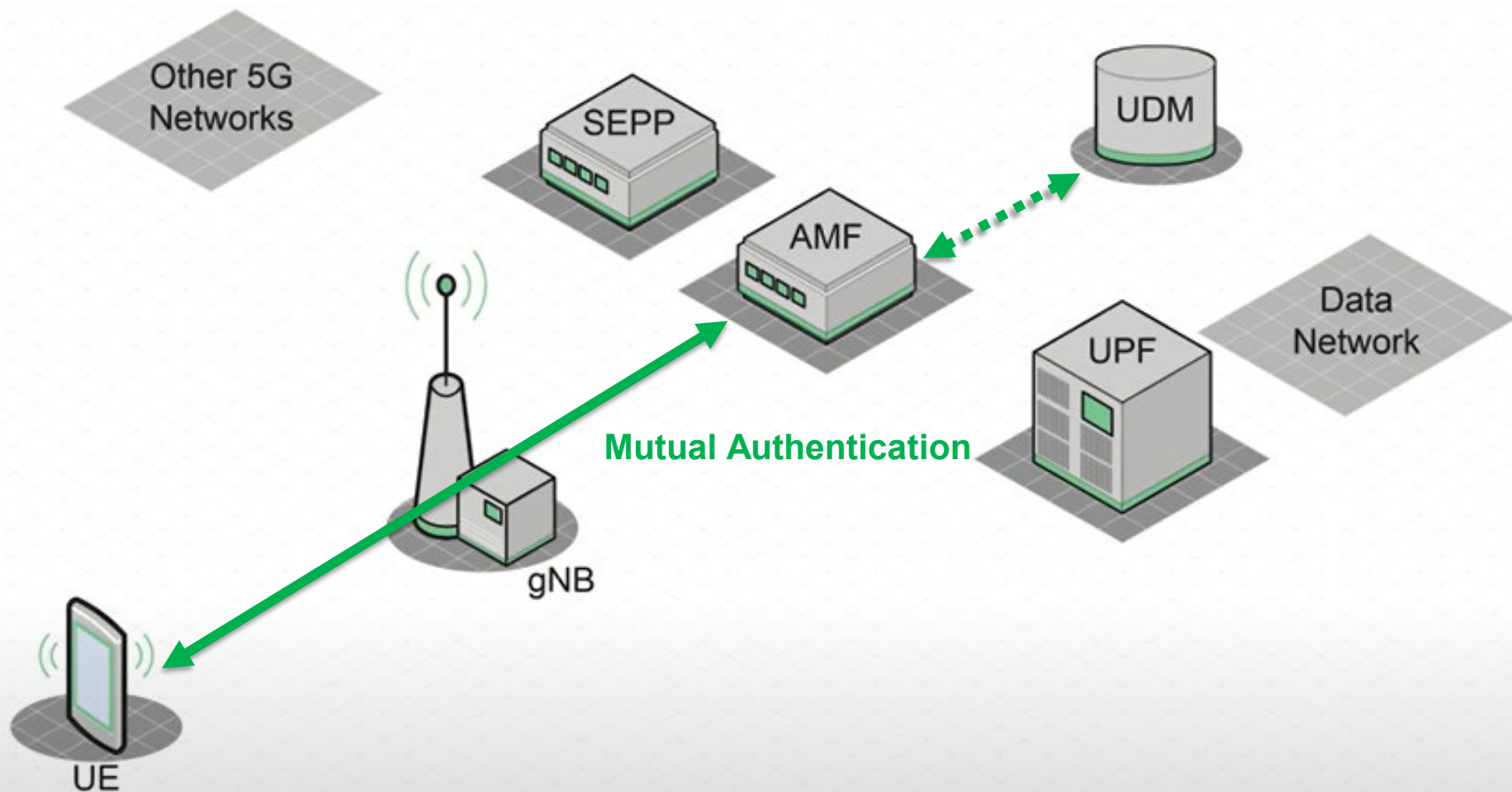
- Confidentiality and integrity protection **of voice, data, and signaling**
- Authorization, confidentiality, and integrity protection between **network elements**
- Authorization, confidentiality, and integrity protection for **next generation services**
- Authorization for **users, devices, and networks**
- Extensible systems for **new algorithms** and procedures to mitigate risks e.g. coming from quantum computing
- Prevention against **DoS and signaling attacks**
- **Privacy** protection, e.g. by means of pseudonyms or temporary identifiers and user location information
- Support **emergency cases** i.e. granting temporary access to the network based on operator policies
- **Device theft prevention**: supporting a secure mechanism to disable or re-enable a stolen device; protecting device identifiers when being store



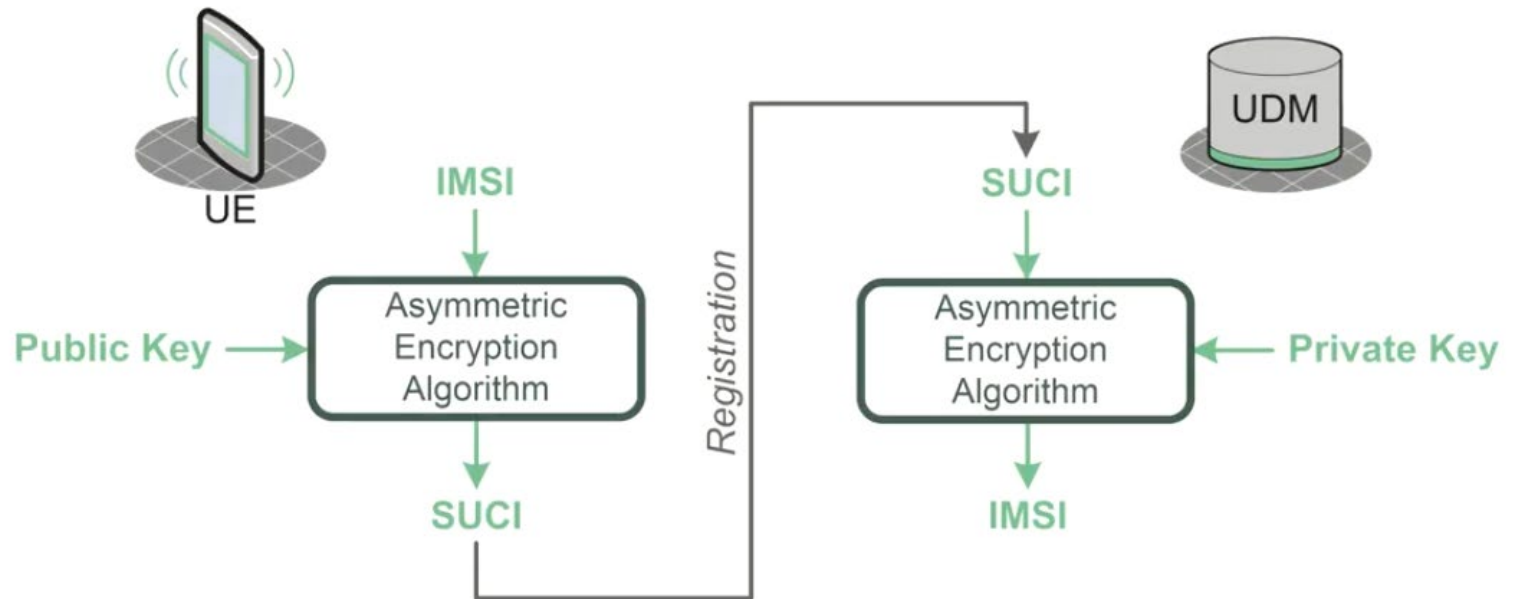
5G & Security

- End of SIM/UICC?
- The cloud-based security such as tokenization
- Host card emulation (HCE)
- Device-based technologies such as Trusted Execution Environment (TEE)
- Traditional SIM/UICC can still act as a base for the high security demands.
- Large scale IoT deployments & security needs

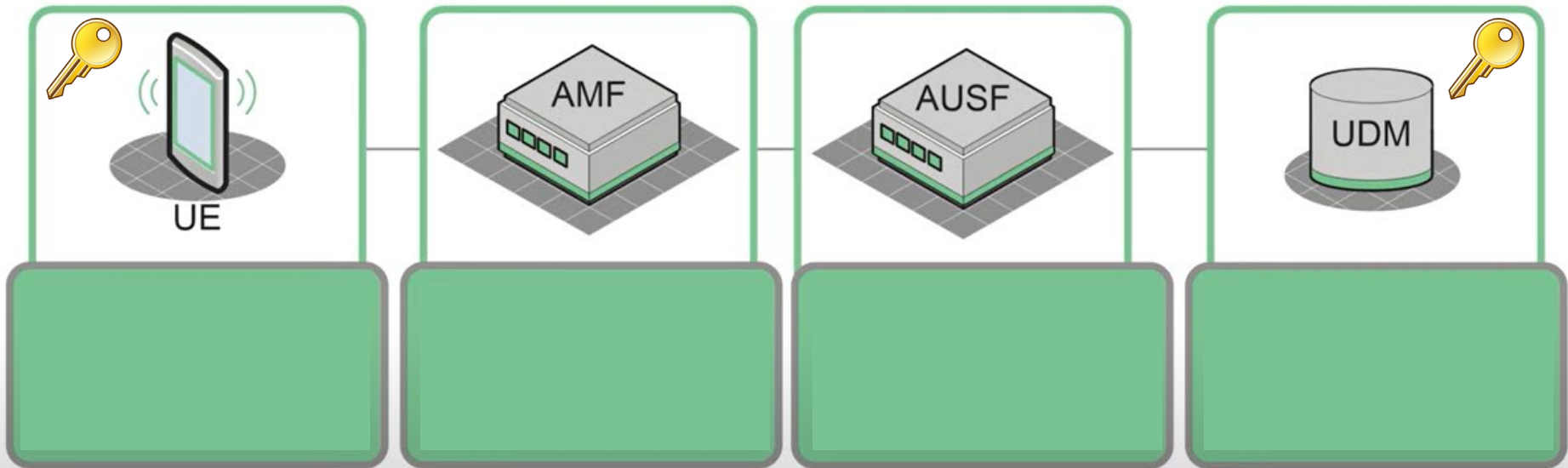
5G Security



Subscription Concealed Identifier

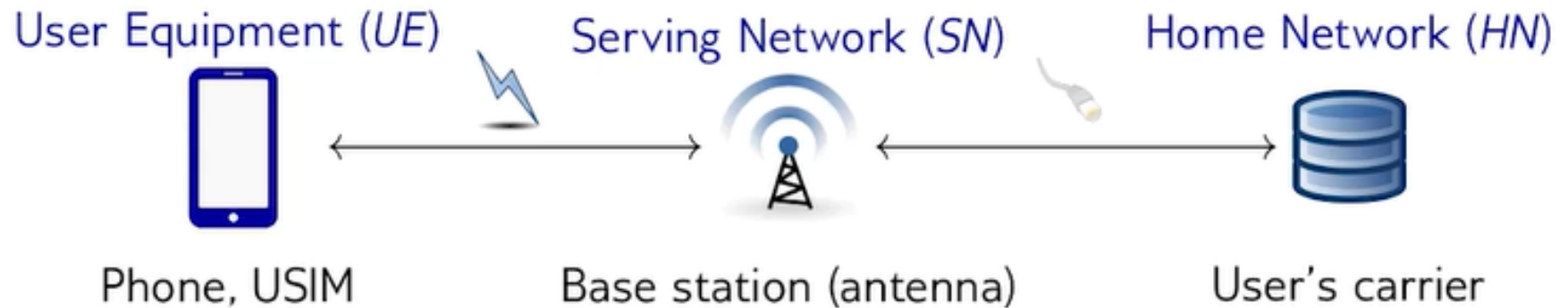


Mutual Authentication





AKA in 5G



5G AKA designed to:

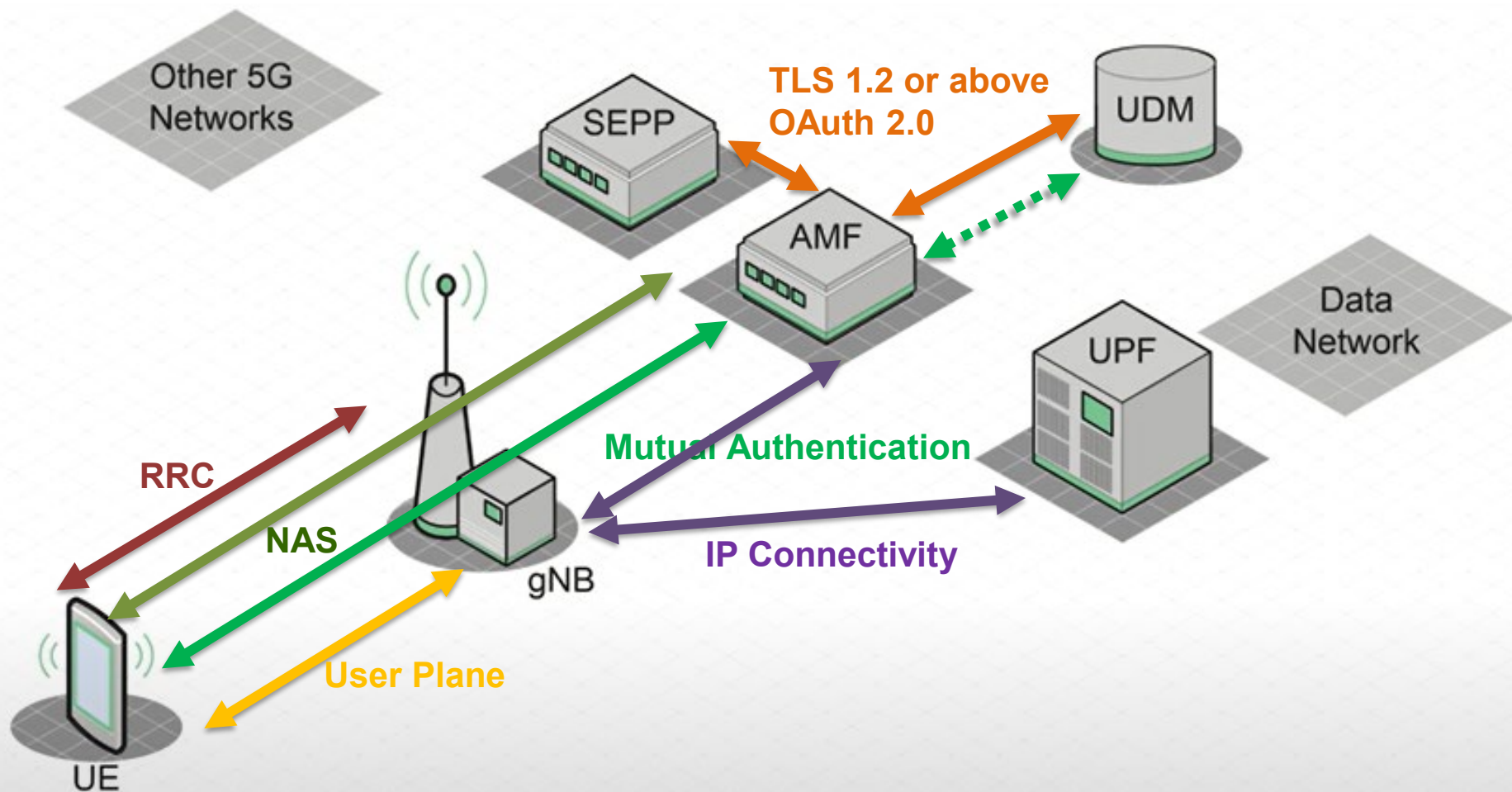
- ▶ **mutually authenticate** User Equipment with its Home Network (carrier)
- ▶ **establish session keys** btw. the User Equipment and its Serving Network

User Equipment (Phone with USIM) and Home Network **share**:

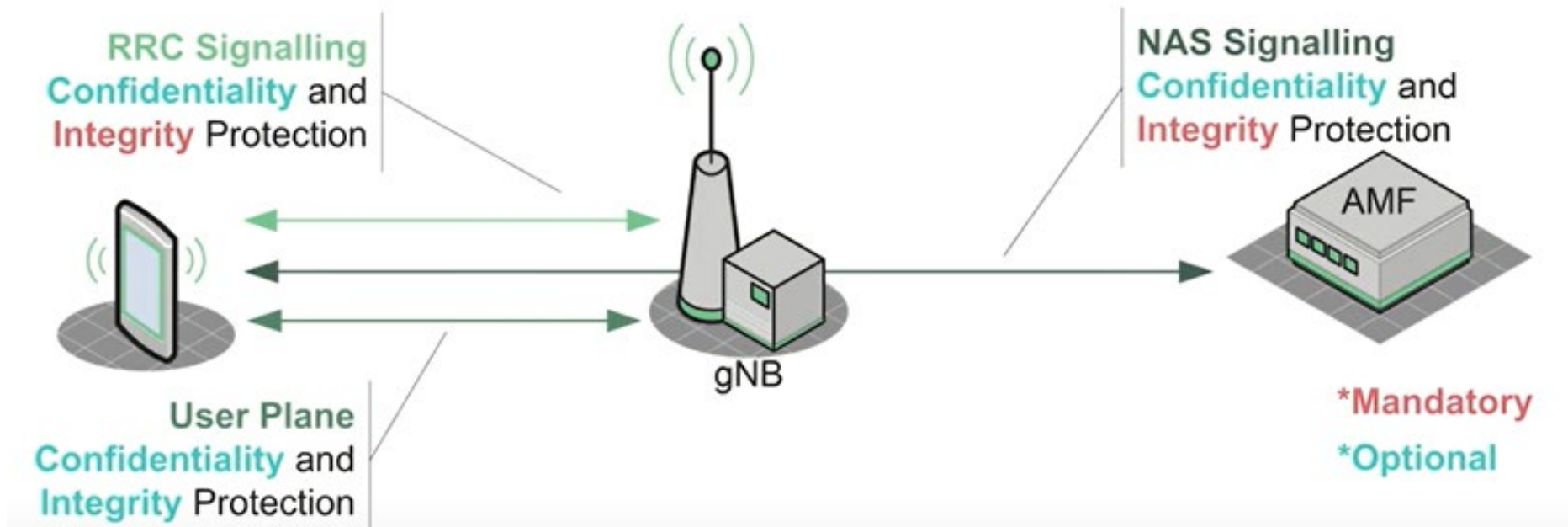
- ▶ a permanent UE's **identifier** $SUPI$ (for identification)
- ▶ a **symmetric key** K (shared secret)
- ▶ a **sequence number** SQN (for replay protection)

User Equipment knows the Home Network's **public key** pk_{HN}

5G Security



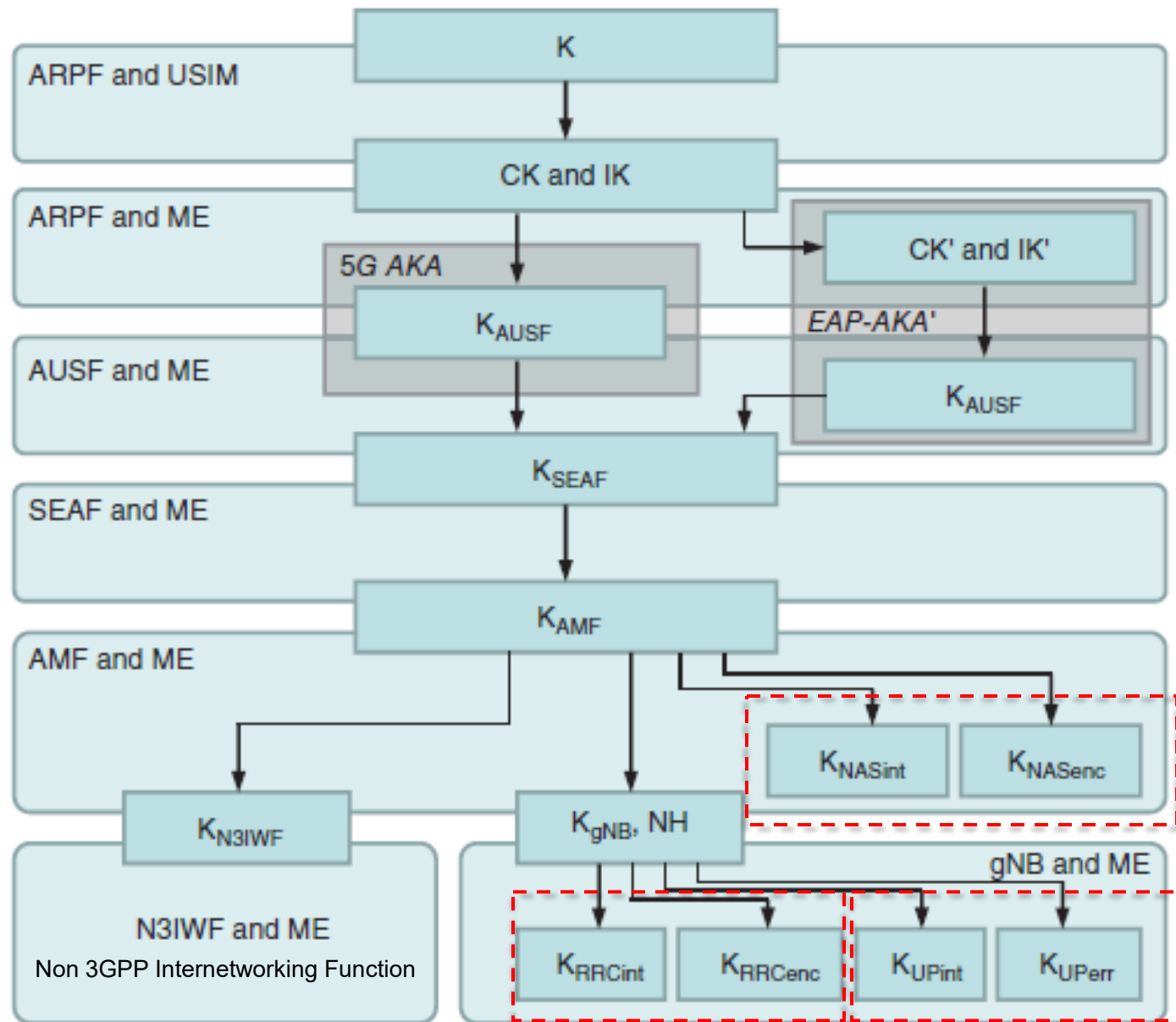
Encryption & Integrity



*Will depend on the use case scenario
E.g. Human User Vs. Non-Critical IoT Device*

All these entities will require many keys!

5G Key Hierarchy



5G Key Hierarchy Contd.

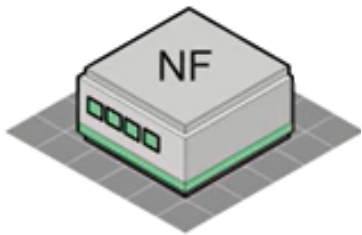
Key	Description
K, CK, IK	Authentication keys.
CK', IK'	Authentication keys derived from CK and IK if EAP-AKA' applied.
K_{AUSF}	Key for AUSF in home network. AUSF generates K_{AUSF} from the authentication material (K, CK and IK for 5G AKA, or from CK' and IK' in case EAP-AKA' is used).
K_{SEAF}	Anchor key. AUSF and ME derive it from K_{AUSF} .
K_{AMF}	Key for AMF. K_{AMF} is derived from K_{SEAF} .
K_{NASint}	Key for NAS signaling. AMF and ME derive it from K_{AMF} . Protects NAS signaling by applying integrity algorithm.
K_{NASenc}	Key for NAS signaling. AMF and ME derive it from K_{AMF} . Protects NAS signaling by applying encryption algorithm.
K_{N3IWF}	The N3IWF receives $KN3IWF$ from the AMF, and uses it as the key MSK for IKEv2 between the UE and N3IWF in the procedures for untrusted non-3GPP access.
K_{gNB}	Key for gNB. AMF and ME derive it from K_{AMF} . gNB and ME derive it further for performing horizontal or vertical key derivation.
K_{RRCint}	Key for RRC signaling. gNB and ME derive it from K_{gNB} . Protects RRC signaling with integrity algorithm.
K_{RRCenc}	Key for RRC signaling. gNB and ME derive it from K_{gNB} . Protects RRC signaling with encryption algorithm.
K_{UPint}	Key for UP traffic. gNB and ME derive it from K_{gNB} . Protects UP traffic between ME and gNB with encryption algorithm.
K_{UPerr}	Key for UP traffic. gNB and ME derive it from K_{gNB} . Protects UP traffic between ME and gNB with integrity algorithm.
NH	Intermediate key. Refers to Next Hop. AMF and ME derive it for providing forward security.



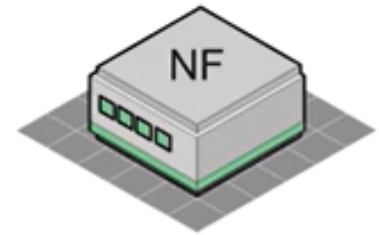
Protecting Service Based NFs with OAuth 2.0



OAuth 2.0
Token Server



Service Consumer
(OAuth 2.0 Client)



Service Producer
(OAuth 2.0 Resource Server)

5G Security Threats and Challenges

- 5G is not designed only for consumer markets but is planned to support remarkable volumes of IoT traffic
- This opens doors for malicious intentions.
 - DoS
 - Weaknesses of payment via near-field communications (NFCs).
 - Ransomware
 - Virtualization related threats
 - All successful attacks on IPs now also automatically affect the 5G infrastructure

Enhanced Mobile Broadband Security (eMBB)

- Content protection
 - The content may also need to be protected by encrypting it both on the network layer (via encryption and integrity protection on the radio interface) and the service layer (DRM service).
- Seamless Authentication
 - When moving from Wi-Fi (indoor) to the 5G network (outdoor), should not require a re-establishment of the session because of re-authentication.
- Privacy
 - Users do not want others to determine which data they consumed when and where.
- Strong authentication and user identification
- Fixed and mobile convergence

Massive IoT & Protection (mMTC)

mIoT is designed to be flexible and efficient. This addresses use cases that involve large numbers of devices with non-time-critical data transfer. Devices are either simple or complex and need to consider efficiency of security and device configuration.

- Data represents the asset in mIoT. Therefore, it is necessary to prevent **unauthorized manipulation** of that data and to thus ensure data integrity.
- Data need to be **confidentiality**-protected
- Battery-powered devices, security protocols and mechanism need to **be energy-efficient**.
- mIoT device should not misuse **lower security mechanisms** during interoperation with networks such as Bluetooth & Wi-Fi.
- **Device-to-device** communication & authentication

Critical Communications Security (URLLC)

CriC covers all use cases that support low end-to-end latency ranging from 1ms up to 10ms even in the highest-mobility scenarios. Its security must be by default both fast and reliable. The 3GPP system shall support ultra-high reliability (99.999% or higher) and high uplink data rate

- Data confidentiality
- Data integrity
- Strong mutual authentication.
 - Authentication protocols and frequency of re-authentication need to be realized in away that latency requirements can be fulfilled.
- Verified protocols & algorithms
- Private networks operated by factories or enterprises, third-party authentication schemes must be supported
- Prioritized access to the network over other devices.

Other 5G Safety concerns

- Quantum-safe computing
 - As 5G is supposed to be used during the next 20 and more years, quantum computing may develop into the next levels, jeopardizing all the previous security measures.
- Physical layer security.
 - Second phase of 5G (e.g. when using the new spectrum) physical layer security may have to be considered.
- mmWave.
 - A technology that is required to provide a new spectrum in the higher frequency bands, above 6 GHz.
 - *Disruption to Aviation Industry is a serious concern.*
<https://www.faa.gov/5g>

Thank You!



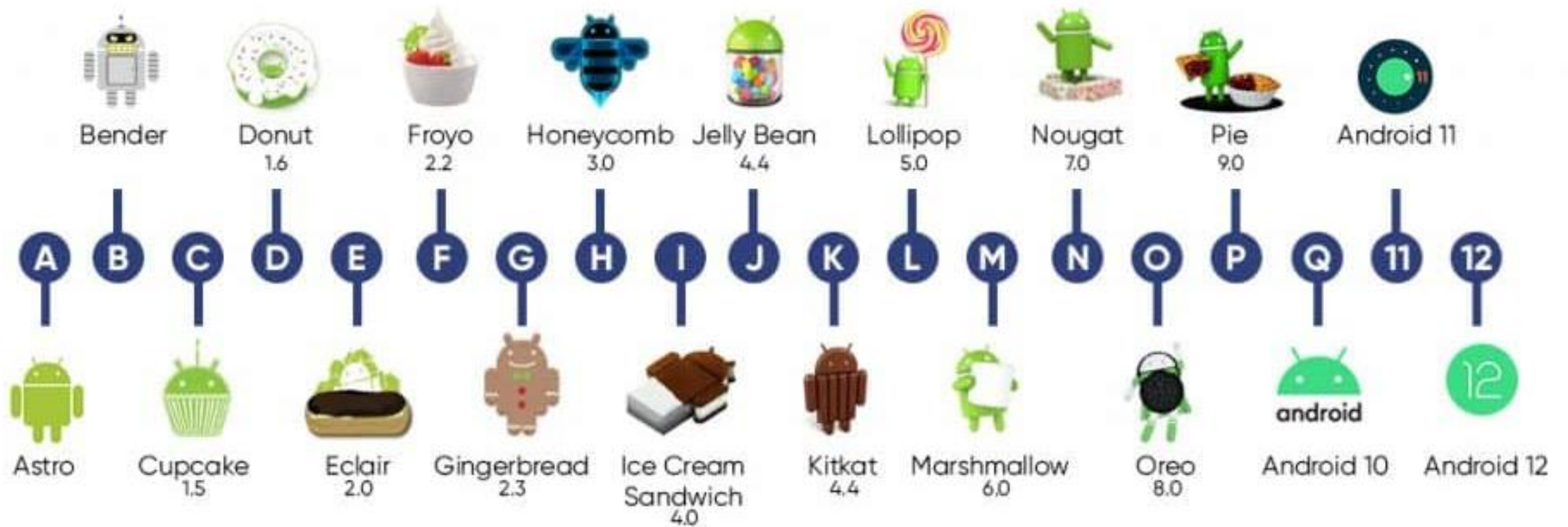
Fundamentals of Android



What is Android???



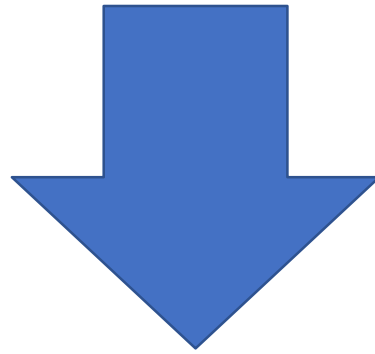
Android Versions

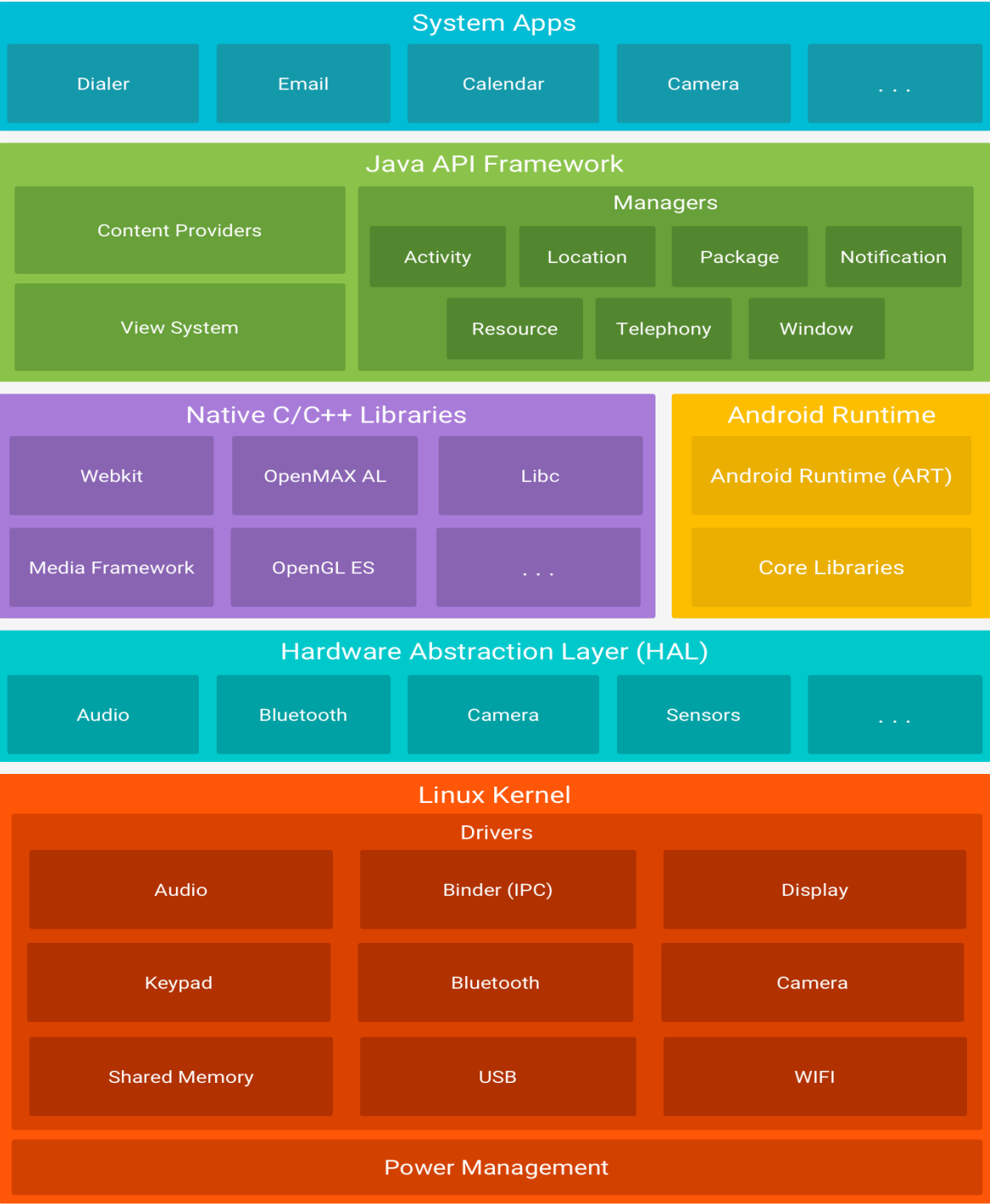




Android - Architecture

Android operating system is a stack of software components which is roughly divided into five sections and four main layers as shown below in the architecture diagram.







ART :

Runs Android framework and applications

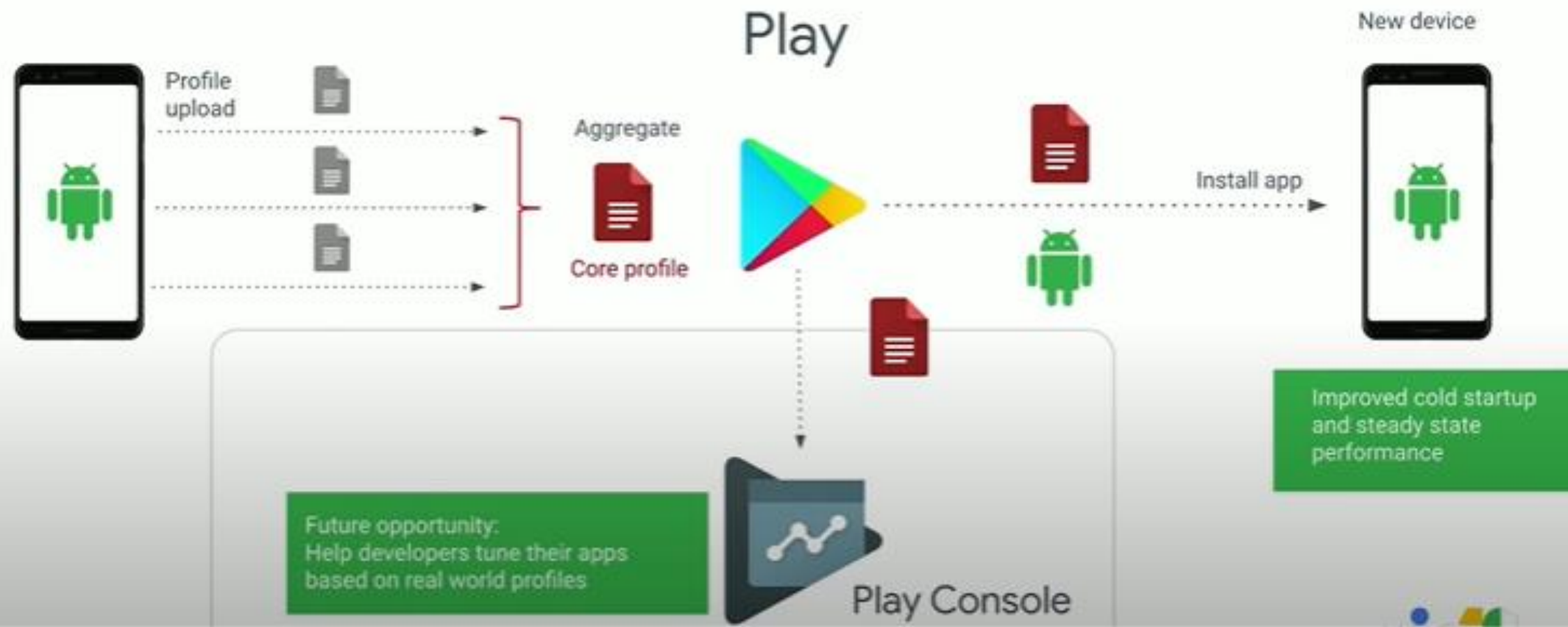
Interpreter, AOT , Profile based compiler,

manages application memory

Memory allocator, garbage Collector

Process

Existing devices



Application fundamentals

Android apps can be written using Kotlin, the Java programming language, and C++ languages. The Android SDK tools compile your code along with any data and resource files into an APK or an Android App Bundle. An Android package, which is an archive file with an .apk suffix, contains the contents of an Android app required at runtime, and it is the file that Android-powered devices use to install the app.

The core framework components

App components are the essential building blocks of an Android app. Each component is an entry point through which the system or a user can enter your app. Some components depend on others.

Building Blocks of Android: **Four Components**

Activity

Service

Broadcast Receiver

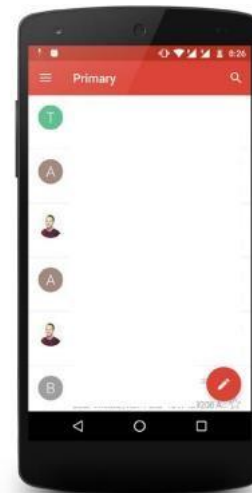
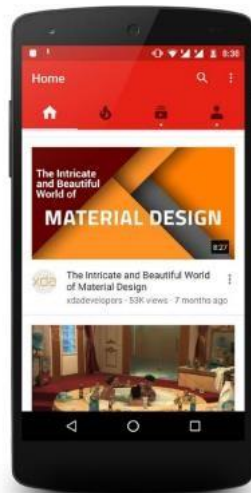
Content Provider

App components

- **Activities** - Entry point for interacting with the user. It represents a single screen with a user interface.
- **Services** – General purpose entry point for keeping an app running in the background for all kinds of reasons. It is a component that runs in the background to perform long-running operations or to perform work for remote processes. A service does not provide a user interface.
- **Broadcast receivers** - Component that lets the system deliver events to the app outside of a regular user flow so the app can respond to system-wide broadcast announcements.
- **Content providers** - Manages a shared set of app data that you can store in the file system, in a SQLite database, on the web, or on any other persistent storage location that your app can access.

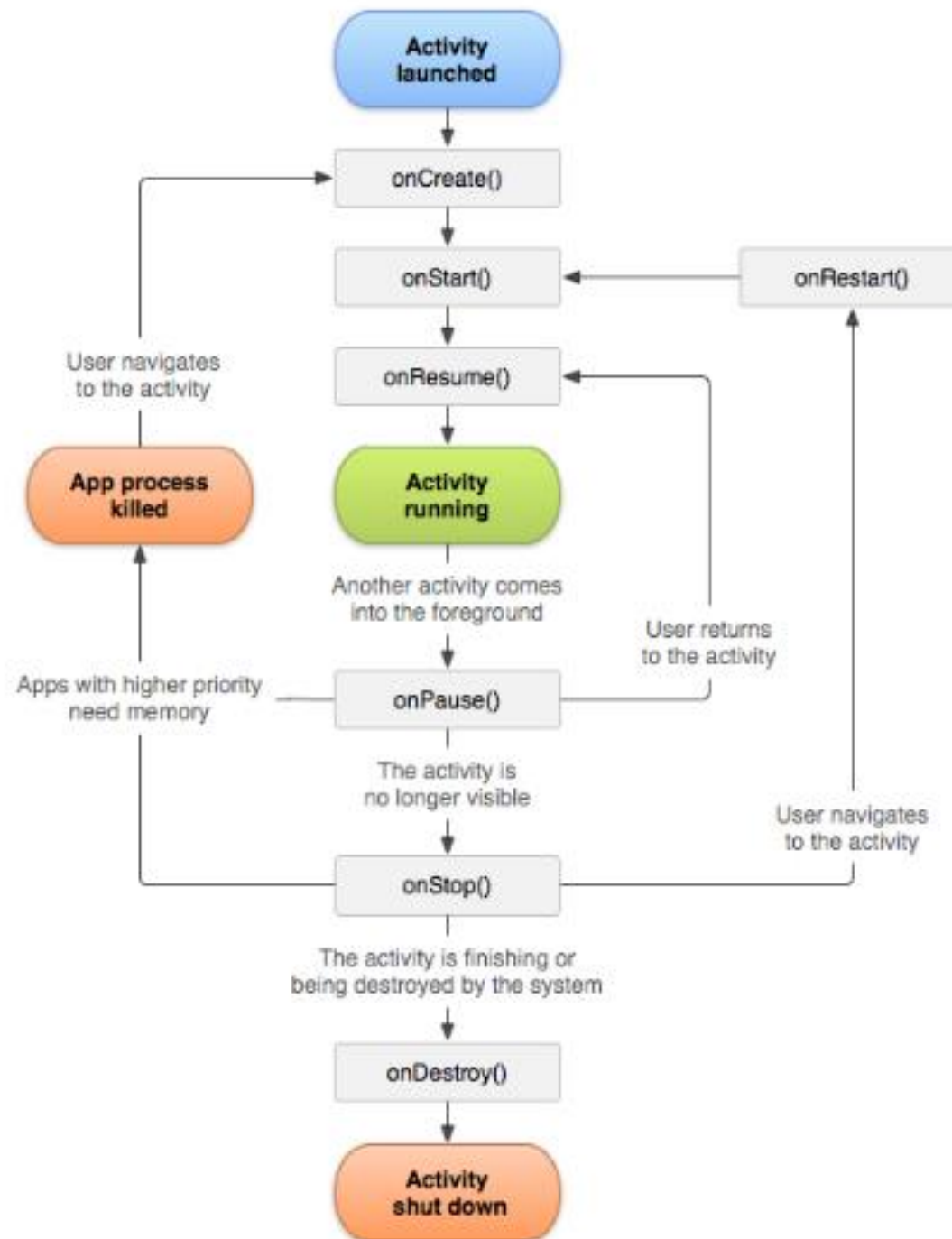
Activity

An activity represents a single screen with a user interface, in-short Activity performs actions on the screen. For example, an email application might have one activity that shows a list of new emails, another activity to compose an email, and another activity for reading emails. If an application has more than one activity, then one of them should be marked as the activity that is presented when the application is launched.



Activities

- An activity facilitates the following key interactions between system and app:
- Keeping track of what the user currently cares about—what is on-screen—so that the system keeps running the process that is hosting the activity.
- Knowing which previously used processes contain stopped activities the user might return to and prioritizing those processes more highly to keep them available.
- Helping the app handle having its process killed so the user can return to activities with their previous state restored.
- Providing a way for apps to implement user flows between each other, and for the system to coordinate these flows. The primary example of this is sharing.
- You implement an activity as a subclass of the Activity class.

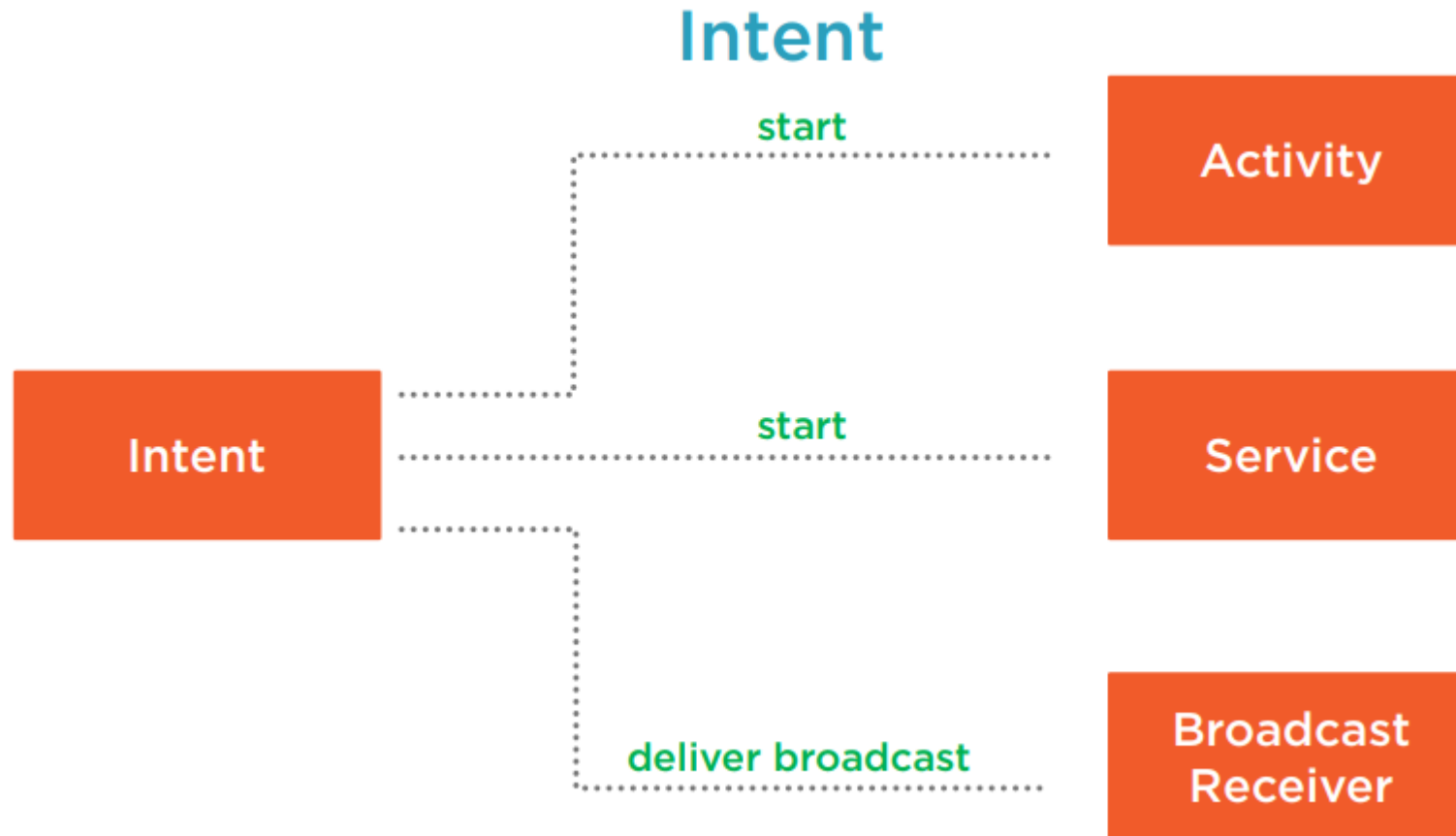


Activity Life Cycle

Method	Description
onCreate	Called when activity is first created
onStart	Called when activity is becoming visible to the user
OnResume	Called when activity will start interacting with the user
onPause	Called when activity is not visible to the user
onStop	Called when activity is no longer visible to the user
onRestart	Called after your activity is stopped, prior to start
onDestroy	called before the activity is destroyed.

INTENTS

- ❑ Start an Activity
- ❑ Start a service
- ❑ Deliver a broadcast for Broadcast Receiver



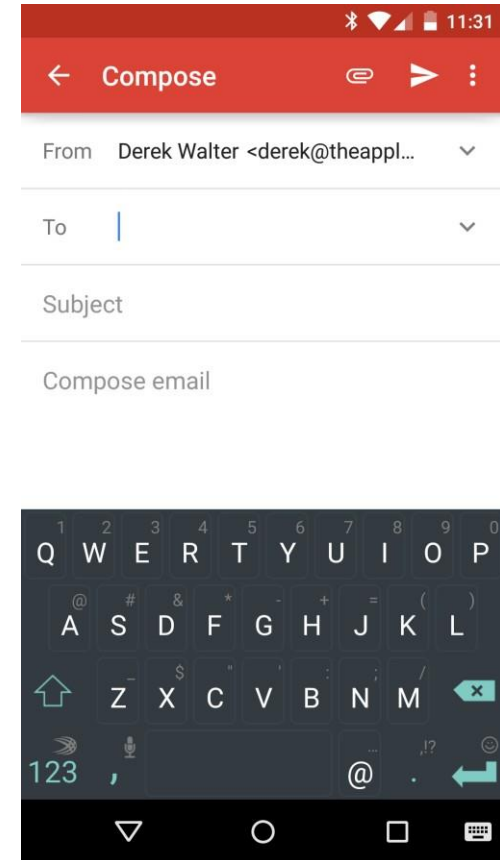
Activate components - Intents

- An asynchronous message called an intent activates three of the four component types: activities, services, and broadcast receivers. Intents bind individual components to each other at runtime. You can think of them as the messengers that request an action from other components, whether the component belongs to your app or another.
- An intent is created with an Intent object, which defines a message to activate either a specific component (an explicit intent) or a specific type of component (an implicit intent).
- For activities and services, an intent defines the action to perform, such as to view or send something, and might specify the URI of the data to act on, among other things that the component being started might need to know.

Intent & Activity



First Activity



Second Activity

Explicit Intent : An explicit intent is used to start a specific component within the same application or a different application.

// Creating an explicit intent to navigate from Activity A to Activity B

```
Intent intent = new Intent(ActivityA.this, ActivityB.class);  
startActivity(intent);
```

Implicit Intent

An implicit intent is used when you want to start a component based on an action or data type.

// Creating an implicit intent to share an image

```
Intent intent = new Intent(Intent.ACTION_SEND);  
intent.setType("image/*");  
startActivity(Intent.createChooser(intent, "Share Image"));
```

Service

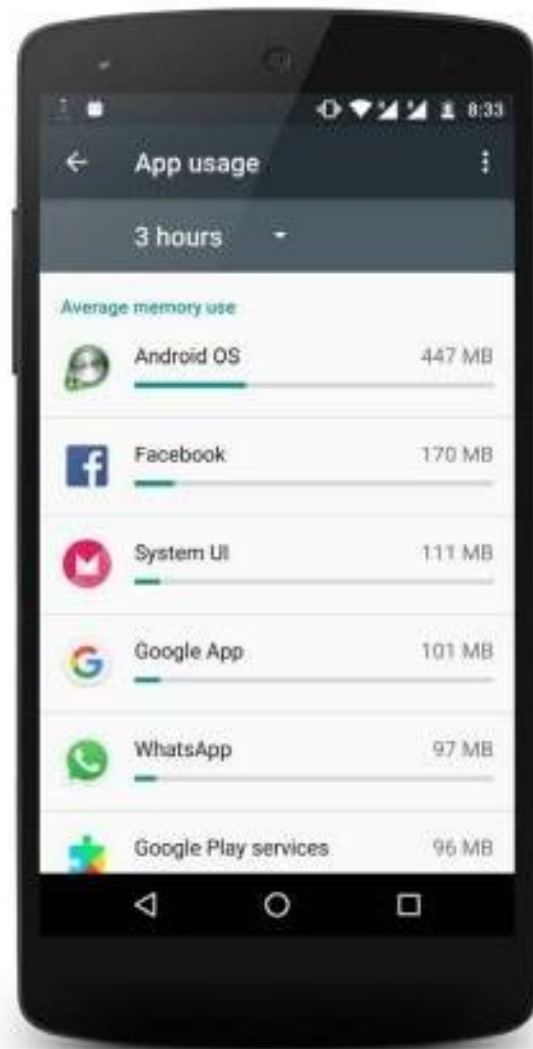
- A service does not provide a user interface.
- A service is a component that runs in the background to perform long-running operations.
- Can be started by activity.

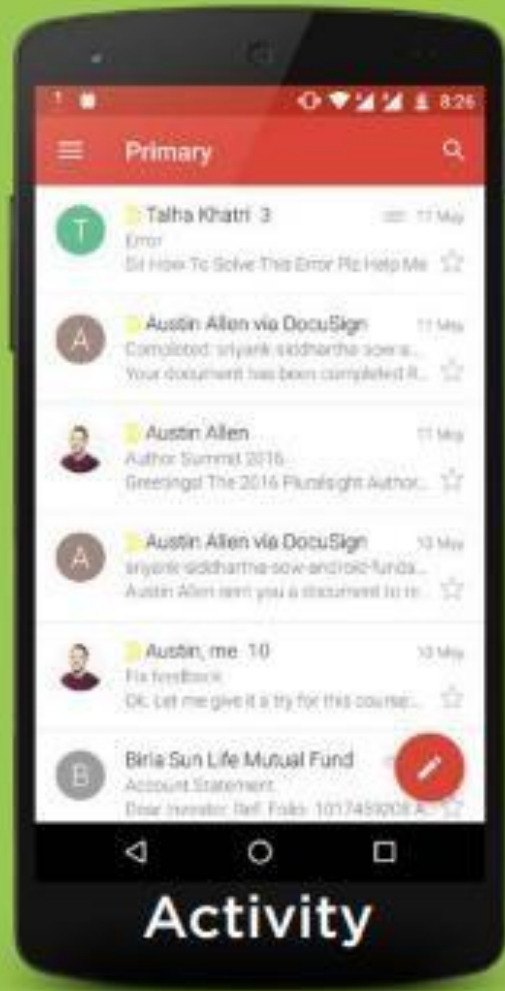
Examples -

- A service might play music in the background while the user is in a different application, or it might fetch data over the network without blocking user interaction with an activity.
- Downloading a file
- Updating an app

Services

- There are two types of services that tell the system how to manage an app:
- Started services and bound services. Started services tell the system to keep them running until their work is completed. This might be to sync some data in the background or play music even after the user leaves the app. Syncing data in the background or playing music represent different types of started services, which the system handles differently
- Bound services run because some other app (or the system) has said that it wants to make use of the service. A bound service provides an API to another process, and the system knows there is a dependency between these processes. So, if process A is bound to a service in process B, the system knows that it needs to keep process B and its service running for A.





Background Tasks



Playing Music



Downloading a File

Service



Background Tasks



Updating apps



Uploading File

Service

Broadcast Receiver

Broadcast Receivers simply respond to broadcast messages from other applications or from the system. For example, applications can also initiate broadcasts to let other applications know that some data has been downloaded to the device and is available for them to use, so this is broadcast receiver who will intercept this communication and will initiate appropriate action.

Broadcast receivers

- Many broadcasts originate from the system, like a broadcast announcing that the screen is turned off, the battery is low, or a picture is captured. Apps can also initiate broadcasts, such as to let other apps know that some data is downloaded to the device and is available for them to use.
- Although broadcast receivers don't display a user interface, they can create a status bar notification to alert the user when a broadcast event occurs. More commonly, though, a broadcast receiver is just a gateway to other components and is intended to do a very minimal amount of work.
- For instance, a broadcast receiver might schedule a `JobService` to perform some work based on an event using `JobScheduler`. Broadcast receivers often involve apps interacting with each other, so it's important to be aware of the security implications when setting them up.



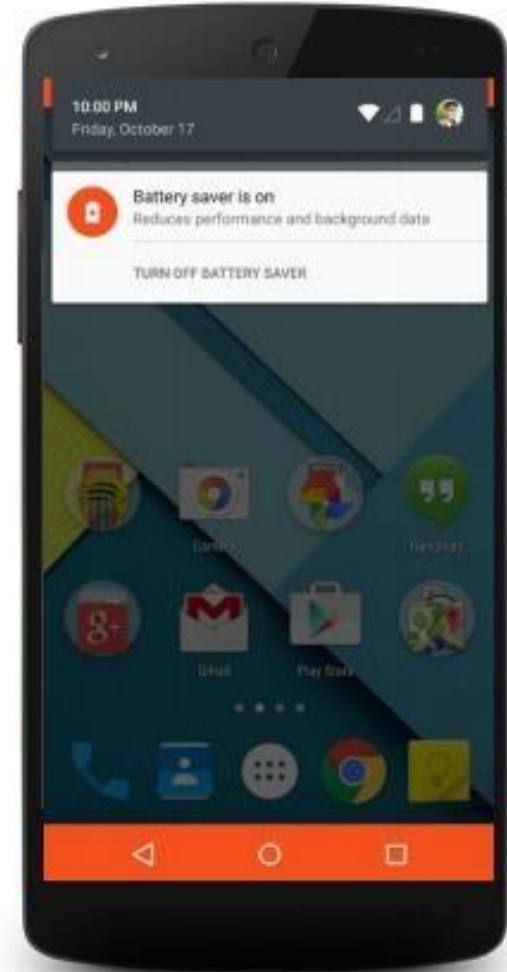
Android Operating System

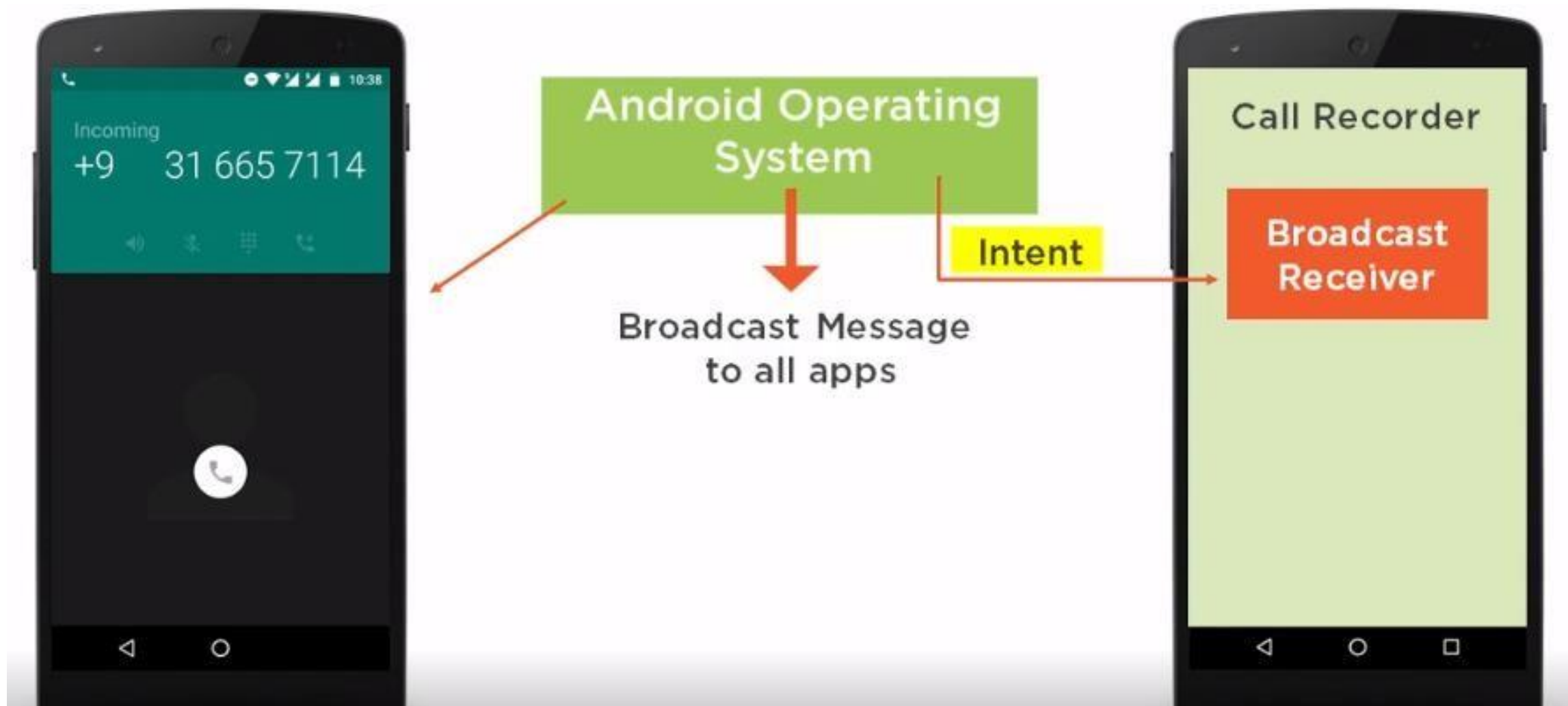


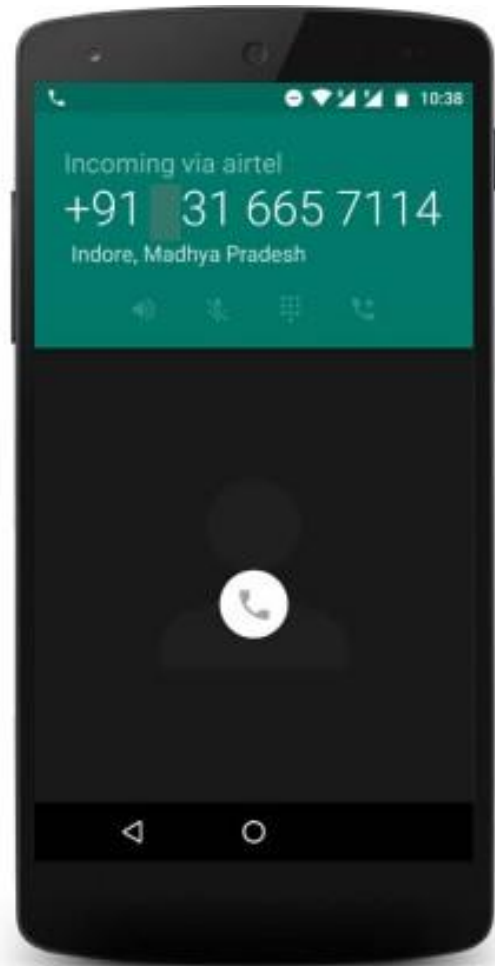
Broadcast Message
to all apps

Few Examples : -

- Incoming Call
- Flight Mode on/off
- Wi-fi on/off







Android Operating System

Intent

Broadcast Message
to all apps

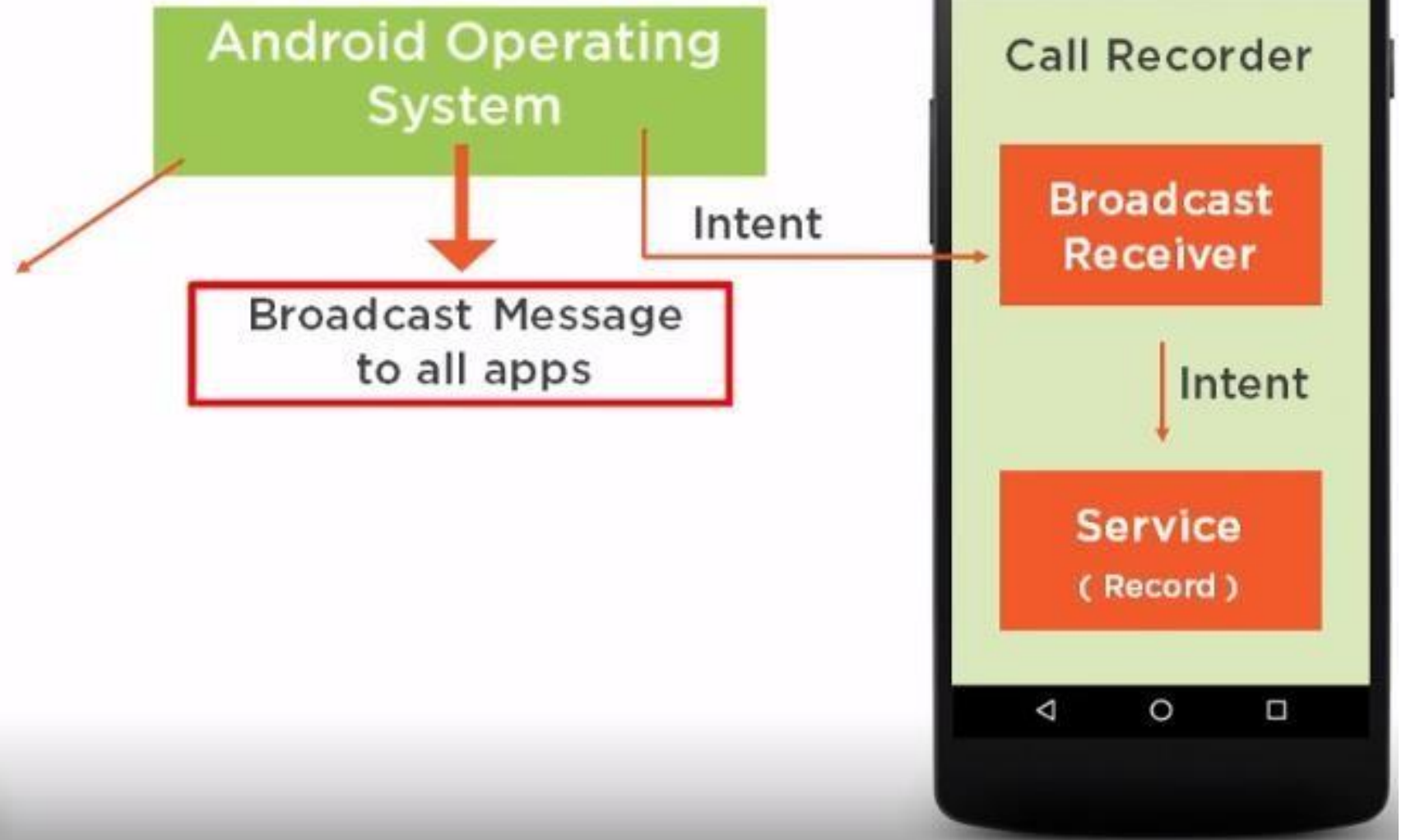
Call Recorder

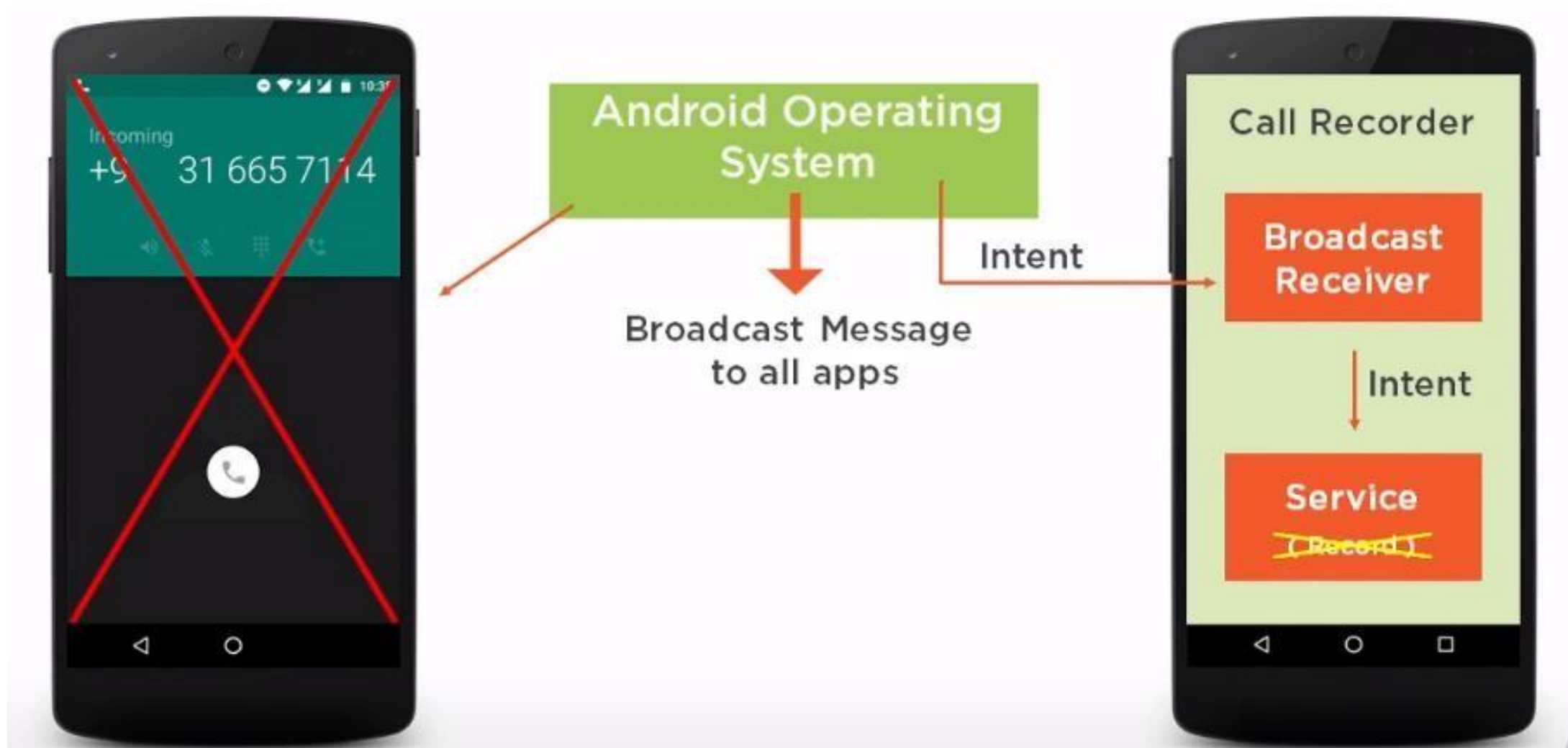
Broadcast
Receiver

Intent

Service
(Record)







Imagine you are developing an app that needs to perform certain actions when the device's battery level reaches a critical point, such as 10% or less. You can use a broadcast receiver to listen for the system's {battery low broadcast message} and take appropriate actions when the message is received.

Here's an example implementation:

javaCopy code

```
public class BatteryReceiver extends BroadcastReceiver {

    @Override
    public void onReceive(Context context, Intent intent) {
        if (intent.getAction().equals(Intent.ACTION_BATTERY_LOW)) {
            // Do something when the battery level is low
            Toast.makeText(context, "Battery level is low!", Toast.LENGTH_SHORT).show();
        }
    }
}
```

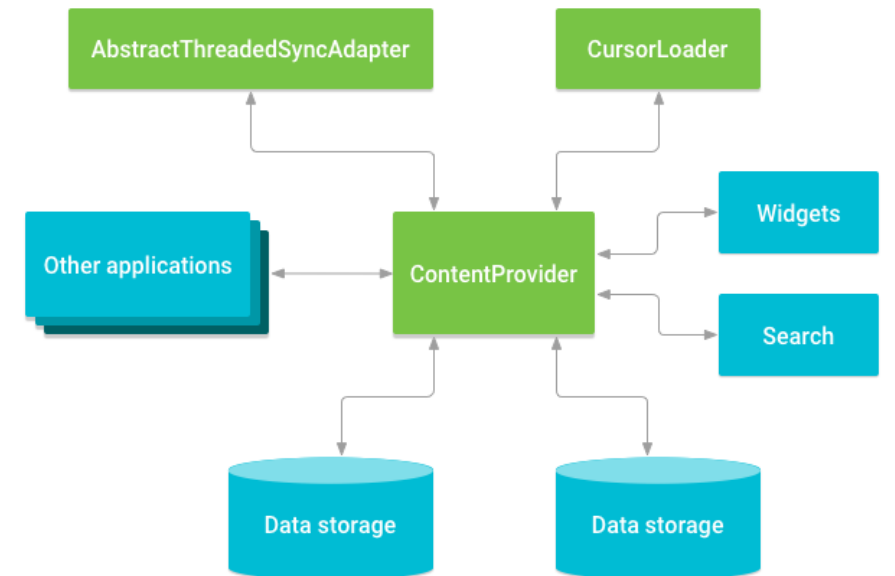
To register the broadcast receiver, you need to add it to the AndroidManifest.xml file with an intent filter that specifies the message types the receiver can handle. In this case, the intent filter should specify the action "android.intent.action.BATTERY_LOW".

```
<receiver android:name=".BatteryReceiver">
    <intent-filter>
        <action android:name="android.intent.action.BATTERY_LOW" />
    </intent-filter>
</receiver>
```

Content Provider

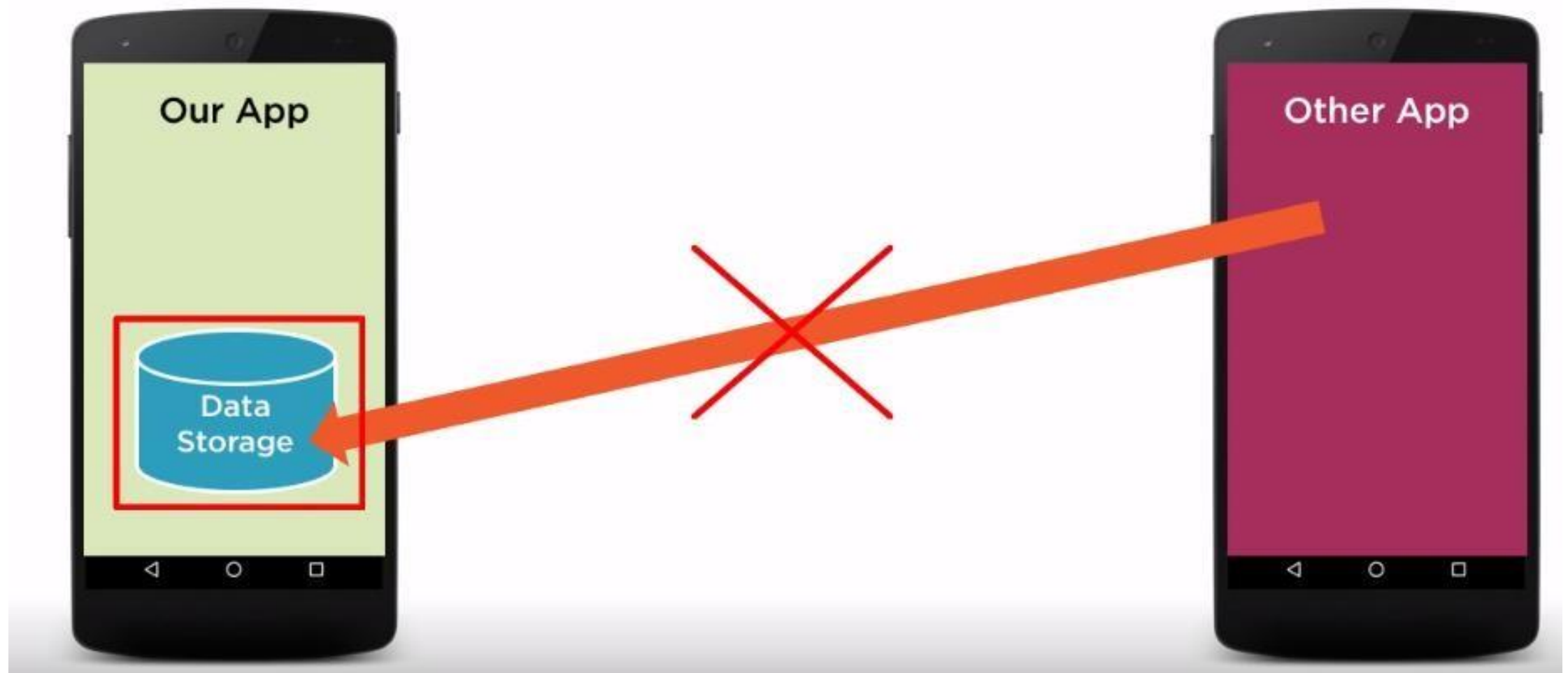
A *content provider* manages a shared set of app data that you can store in the file system, in a SQLite database, on the web, or on any other persistent storage location that your app can access.

A content provider component supplies data from one application to others on request. Such requests are handled by the methods of the *ContentResolver* class.

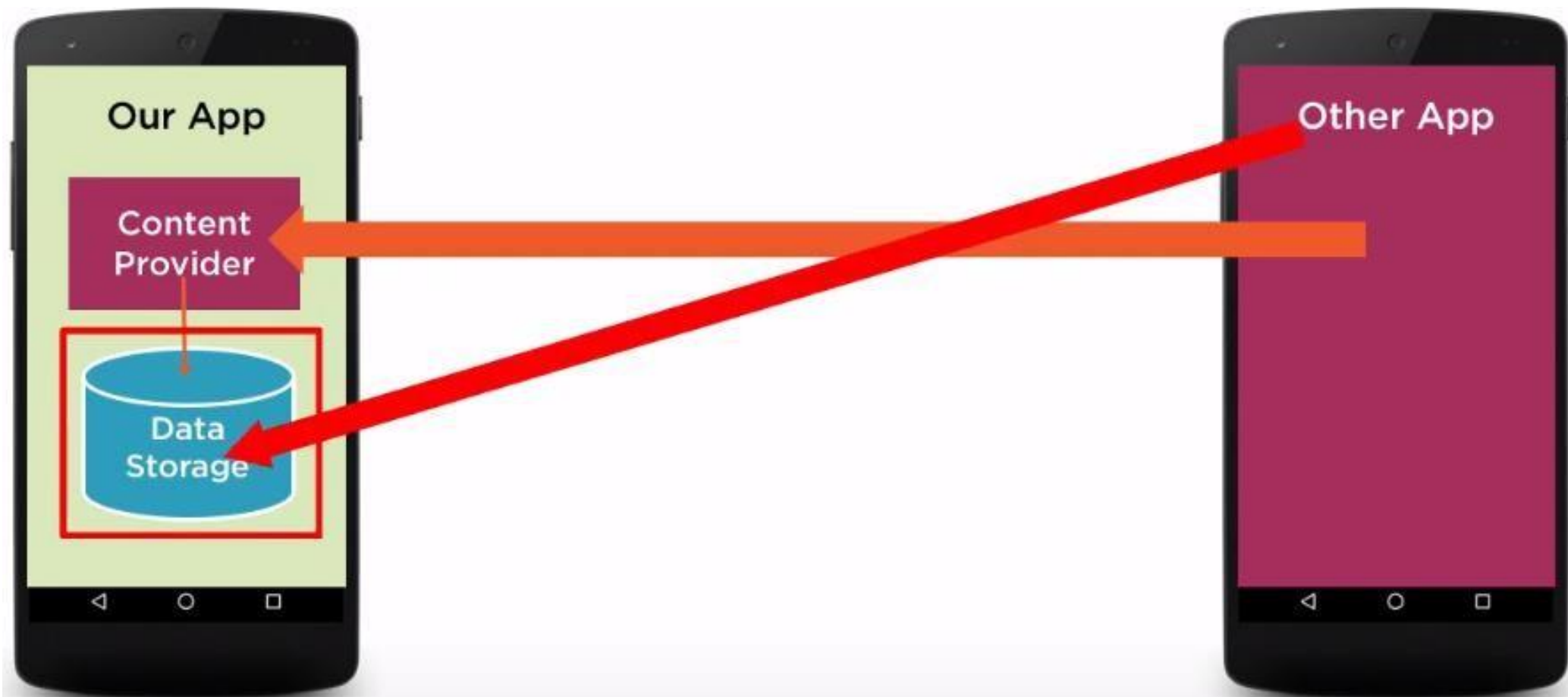


Content providers

- For example, the Android system provides a content provider that manages the user's contact information. Any app with the proper permissions can query the content provider, such as using Contacts. Data, to read and write information about a particular person. It is tempting to think of a content provider as an abstraction on a database.
- However, they have a different core purpose from a system-design perspective. To the system, a content provider is an entry point into an app for publishing named data items, identified by a URI scheme. Thus, an app can decide how it wants to map the data it contains to a URI namespace, handing out those URIs to other entities which can in turn use them to access the data.
- There are a few particular things this lets the system do in managing an app: Assigning a URI doesn't require that the app remain running, so URIs can persist after their owning apps exit. The system only needs to make sure that an owning app is still running when it retrieves the app's data from the corresponding URI. These URIs also provide an important fine-grained security model. For example, an app can place the URI for an image it has on the clipboard but leave its content provider locked up so that other apps cannot freely access it.







Content Provider : Data Storage options

- Content Providers data storage in two ways
 - File data
 - photo, video & audio files
 - Structured Data
 - SQLite database
 - Array

real-world example of a Content Provider in Android could be the Contacts app on your phone. When you open the Contacts app, it displays a list of all your contacts. These contacts are not stored within the Contacts app but are instead stored in a centralized location called the **Contacts Provider**.

The Contact Provider is a Content Provider that manages the contacts database on your phone. Other apps can interact with the Contacts Provider to read or modify the contacts data. For example, if you install a third-party email app and want to add a contact from within that app, it can use the Contacts Provider to add the new contact to the centralized contacts database.

By using a centralized Content Provider like the Contacts Provider, Android ensures that multiple apps can access and modify the same data without causing data inconsistencies or conflicts.

Content Providers in Android provide a level of security by controlling access to the data they manage.

When you define a Content Provider in your app, you can set permissions that control which apps or components can access the data.

There are two types of permissions for Content Providers:
read and write.

To access a Content Provider, an app must have the appropriate permission declared in its manifest file. When the app makes a request to the Content Provider, the Android system checks whether the app has the required permission.

Additionally, Content Providers can implement different levels of access control for different types of data.

For example, a Content Provider managing contacts may allow read access to the name and phone number fields but restrict read access to the email address field. This way, apps can access only the data they need and are authorized to access.

Content Providers can also implement other security measures, such as data encryption or authentication, to ensure that the data they manage is secure.

Content Provider : Maintain your app's Security

- Securely expose your private data to other apps.

```
<provider android:name="MyProviderName"  
          android:exported="false" />
```

Prevents any client apps to access content Provider.

```
<provider android:name="MyProviderName"  
          android:exported="true" />
```

Allows any untrusted apps to access ContentProvider.

Any app that has Provider's info can access it.

* using the PackageManager class it's possible.

Give rise to security issues.

Solution

Android:exported="true" with <permission>

- Allows only that client app that knows permission
- Make Provider app secured
- you may specify read write permission separately.

```
<provider android:name="MyProviderName"  
    android:exported="true"  
    android:readPermission="my.read.permission"  
    android:writePermission="my.write.permission"  
/>
```

```
// In Manifest.xml, declare the permissions  
<permission android:name="my.read.permission" />  
<permission android:name="my.write.permission" />
```



Provider
Application

Within the Client App

```
// In Manifest.xml, declare the permissions
<uses-permission
    android:name="my.read.permission"/>

<uses-permission
    android:name="my.write.permission"/>
```

<uses-permission />

Specifies a system permission that the user must grant in order for the app to operate correctly. Permissions are granted by the user when the application is installed or while the app is running (on devices running Android 6.0 and higher).

Android Manifest File

Every app project must have an `AndroidManifest.xml` file (with precisely that name) at the root of the project source set.

The manifest file describes essential information about your app to the Android build tools, the Android operating system, and Google Play.

the manifest file is required to declare the following:

- The app's package name.
- The components of the app.
- The permissions that the app needs to access protected parts of the system or other apps.
- The hardware and software features the app requires.


```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.birthdaycard">

    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="Bithday Card"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/AppTheme">
        <activity android:name=".MainActivity">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

Package name - The manifest file's root element requires an attribute for your app's package name (usually matching your project directory structure—the Java namespace).

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.birthdaycard">
```

App components - Activity

```
<activity android:name=".MainActivity">
    <intent-filter>
        <action android:name="android.intent.action.MAIN" />

        <category android:name="android.intent.category.LAUNCHER" />
    </intent-filter>
</activity>
```

Service

```
<service>
```

```
    <intent_filter>
```

```
    </intent_filter>
```

```
    <meta-data/>
```

```
</service>
```

Permissions

Android apps must request permission to access sensitive user data (such as contacts and SMS) or certain system features (such as the camera and internet access). Each permission is identified by a unique label. For example, an app that needs to send SMS messages must have the following line in the manifest:

```
<manifest ... >  
  <uses-permission android:name="android.permission.SEND_SMS"/>  
  ...  
</manifest>
```

Android security features

- The Android operating system is a multi-user Linux system in which each app is a different user.
- By default, the system assigns each app a unique Linux user ID, which is used only by the system and is unknown to the app. The system sets permissions for all the files in an app so that only the user ID assigned to that app can access them.
- Each process has its own virtual machine (VM), so an app's code runs in isolation from other apps.
- By default, every app runs in its own Linux process. The Android system starts the process when any of the app's components need to be executed, and then shuts down the process when it's no longer needed or when the system must recover memory for other apps.
- The Android system implements the principle of least privilege. That is, each app, by default, has access only to the components that it requires to do its work and no more. This creates a very secure environment in which an app can't access parts of the system it is not given permission for.

Common architectural principles 1

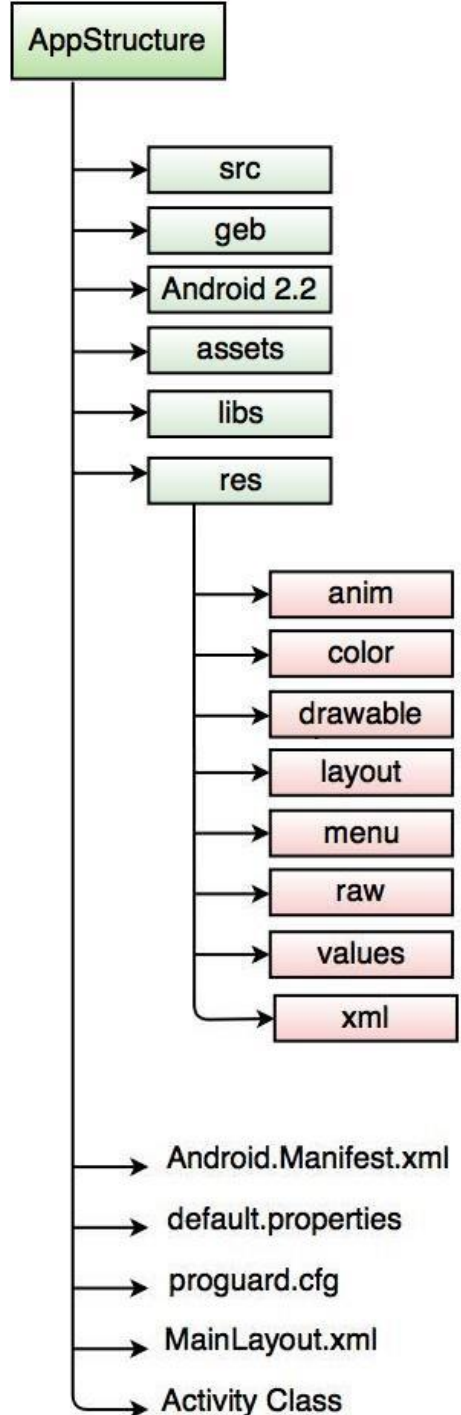
- Separation of concerns - The most important principle to follow. It's a common mistake to write all your code in an Activity or a Fragment. These UI-based classes should only contain logic that handles UI and operating system interactions. By keeping these classes as lean as possible, you can avoid many problems related to the component lifecycle, and improve the testability of these classes.
- Drive UI from data models - Another important principle is that you should drive your UI from data models, preferably persistent models. Data models represent the data of an app. They're independent from the UI elements and other components in your app. This means that they are not tied to the UI and app component lifecycle, but will still be destroyed when the OS decides to remove the app's process from memory.

Common architectural principles 2

- Single source of truth - When a new data type is defined in your app, you should assign a Single Source of Truth (SSOT) to it. The SSOT is the owner of that data, and only the SSOT can modify or mutate it. To achieve this, the SSOT exposes the data using an immutable type, and to modify the data, the SSOT exposes functions or receive events that other types can call.
- Unidirectional Data Flow - In UDF, state flows in only one direction. The events that modify the data flow in the opposite direction. In Android, state or data usually flow from the higher-scoped types of the hierarchy to the lower-scoped ones. Events are usually triggered from the lower-scoped types until they reach the SSOT for the corresponding data type. User events such as button presses flow from the UI to the SSOT where the application data is modified and exposed in an immutable type.

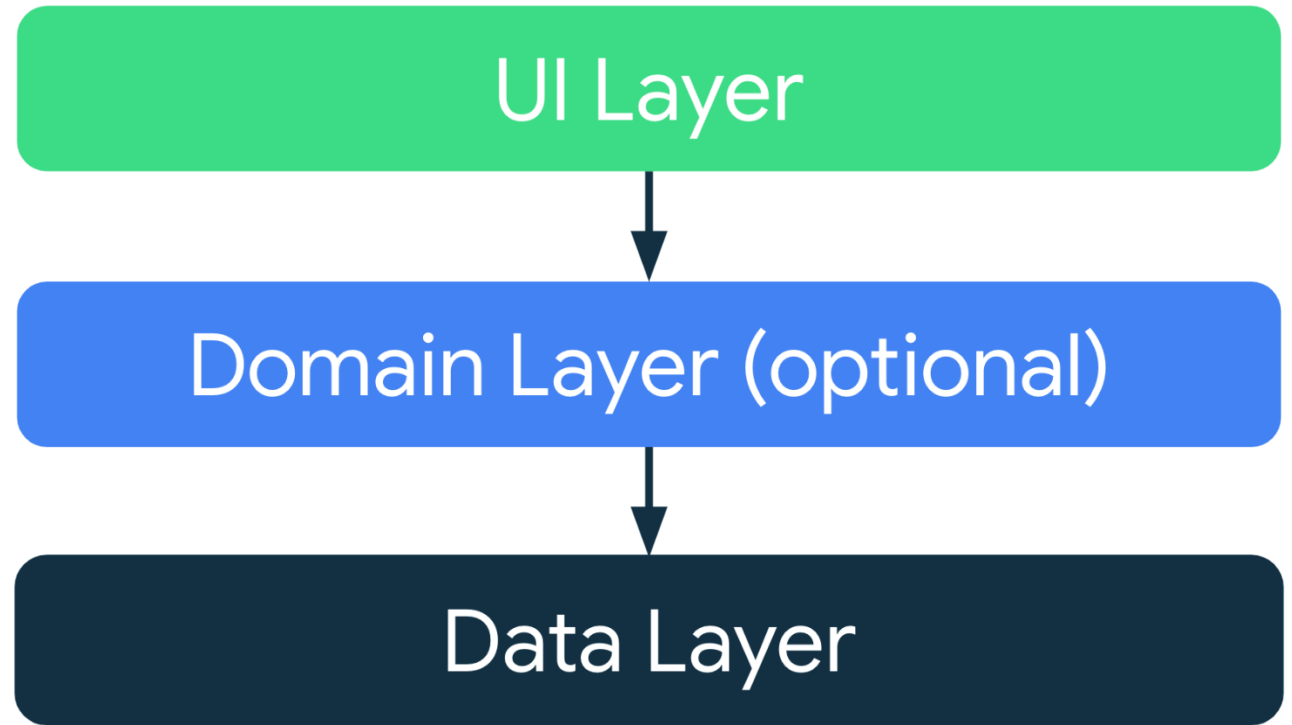
Android Application Package

- Android Package (APK) is the package file format used by the Android operating system for distribution and installation of mobile apps, mobile games and middleware.
- APK file is package which contains installation of mobile apps. When we finish developing our app and want make it public for our users, we compile project and pack into APK file. An APK file contains all of that program's code assets, resources, manifest file, certificates and can be installed on Android OS.



Name	Description
src	The 'src' stands for Source Code . It contains the Java Source files.
gen	The 'gen' stands for Generated Java Library . This library is for Android internal use only.
Android 2.2	The Android Framework Library is stored here.
assets	It is used to store raw asset files.
libs	It contains private libraries.
res	The 'res' stands for Resource file. It can store resource files such as pictures, XML files, etc. It contains some additional folders such as Drawable, Layout and Values. anim: It is used for XML files that are compiled into animation objects. color: It is used for XML files that describe colors. drawable: It is used to store various graphics files. In Android project structure, layout: It is used for placing the XML layout files, which defines how various Android objects such as textbox, buttons, etc. are organized on the screen. menu: It is used for defining the XML files in the application menu. raw: The 'raw' stands for Raw Asset Files. These files are referenced from the application using a resource identifier in the R class. For example, good place for media is MP3 or Ogg files. values: It is used for XML files which stores various string values, such as titles, labels, etc. xml: It is used for configuring the application components.
AndroidManifest.xml	This file indicates the Android definition file. This file contains the information about the Android application such as minimum Android version, permission to access Android device capabilities such as Internet access permission, phone permission etc.
default.properties	This file contains the project settings, such as build the target. Do not edit this file manually. It should be maintained in a Source Revision Control System.
Proguard.cfg	This file defines how ProGuard optimizes and makes your code unclear.
MainLayout.xml	This file describes the layout of the page. So all the components such as textboxes, labels, radio buttons, etc. are displaying on the application screen.
Activity class	The application occupies the entire device screen which needs at least one class inherits from the Activity class. OnCreate() method initiates the application and loads the layout page.

Recommended app architecture



Considering the common architectural principles mentioned in the previous section, each application should have at least two layers:

- The *UI layer* that displays application data on the screen.
- The *data layer* that contains the business logic of your app and exposes application data.

You can add an additional layer called the *domain layer* to simplify and reuse the interactions between the UI and data layers.

Modern App Architecture

This *Modern App Architecture* encourages using the following techniques, among others:

- A reactive and layered architecture.
- Unidirectional Data Flow (UDF) in all layers of the app.
- A UI layer with state holders to manage the complexity of the UI.
- Coroutines and flows.
- Dependency injection best practices.

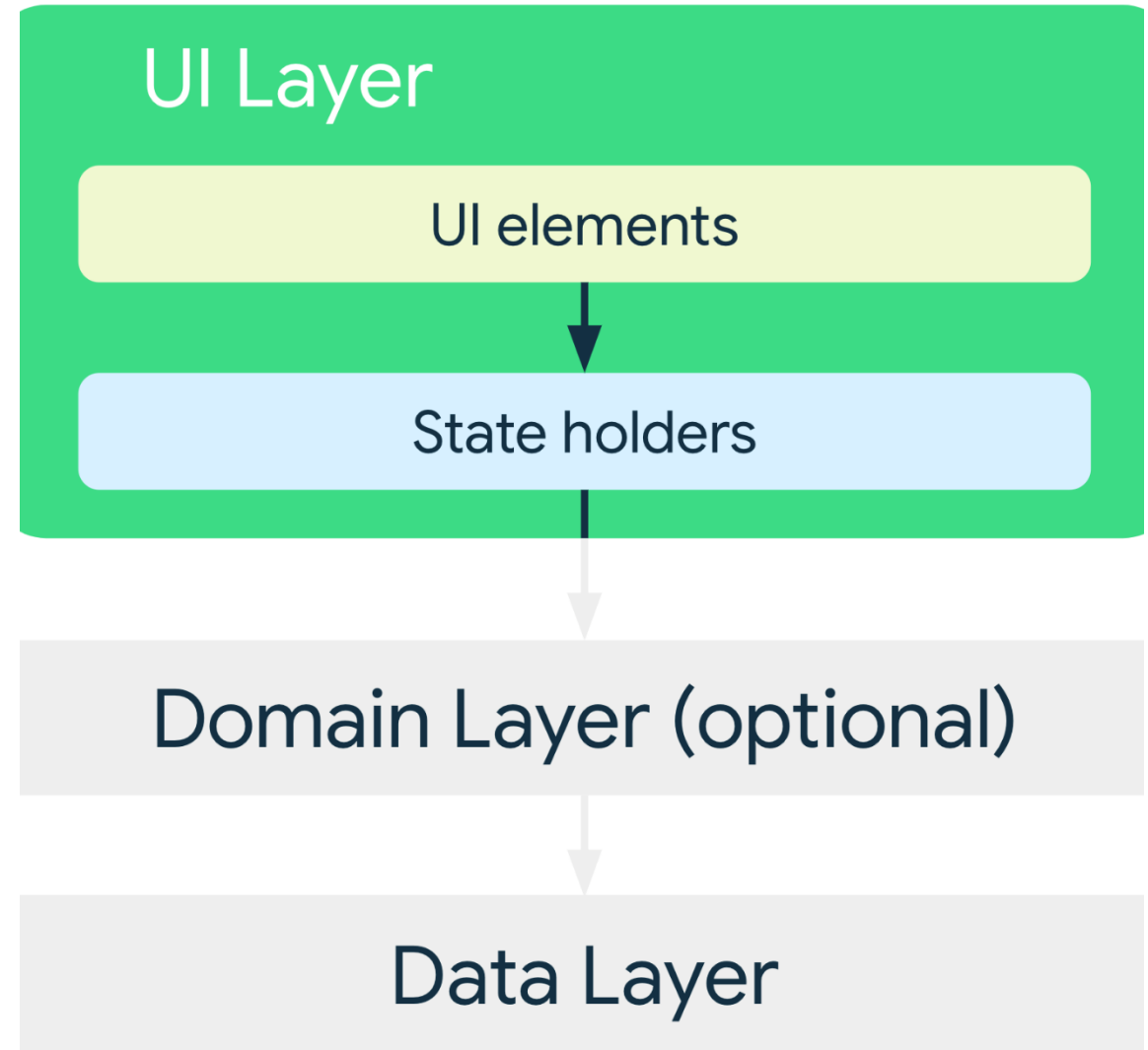
UI layer

The role of the UI layer (or *presentation layer*) is to display the application data on the screen.

Whenever the data changes, either due to user interaction (such as pressing a button) or external input (such as a network response), the UI should update to reflect the changes.

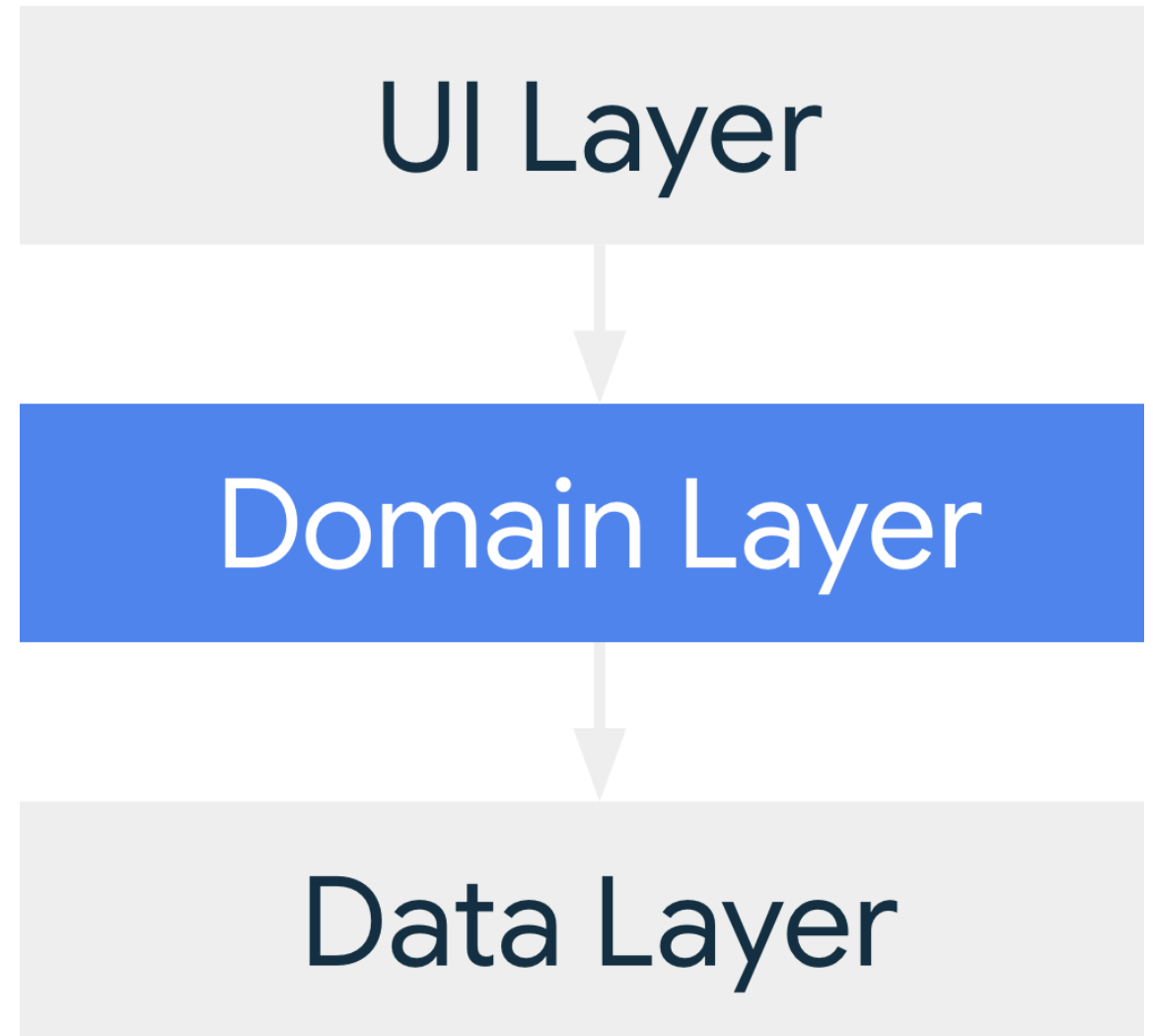
The UI layer is made up of two things:

- UI elements that render the data on the screen. You build these elements using Views or Jetpack Compose functions.
- State holders (such as ViewModel classes) that hold data, expose it to the UI, and handle logic.



Domain layer (optional)

The domain layer is an optional layer that sits between the UI and data layers. The domain layer is responsible for encapsulating complex business logic, or simple business logic that is reused by multiple ViewModels. This layer is optional because not all apps will have these requirements. You should use it only when needed—for example, to handle complexity or favor reusability.

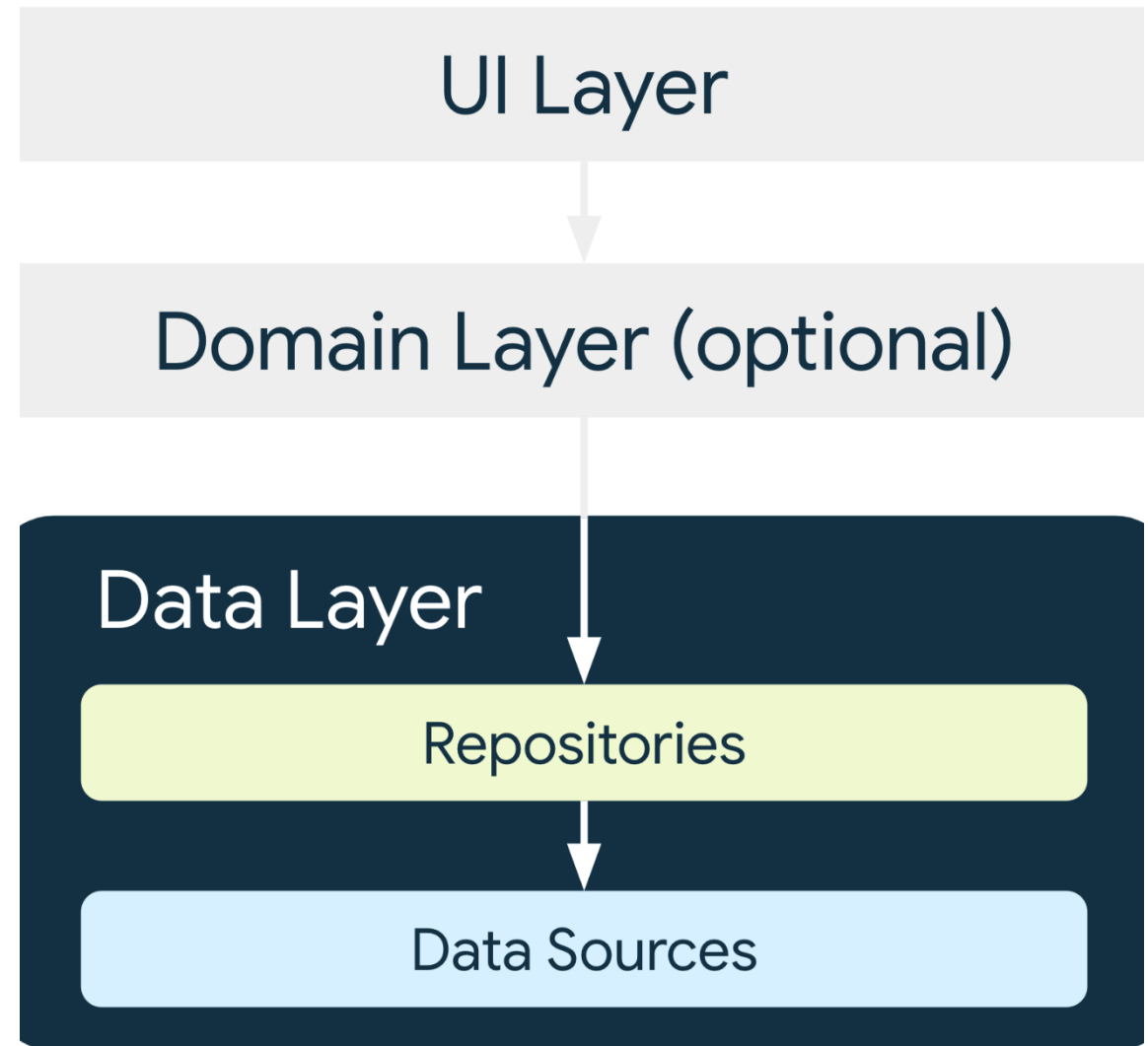


Data layer

The data layer of an app contains the business logic. The business logic is what gives value to your app—it's made of rules that determine how your app creates, stores, and changes data. The data layer is made of repositories that each can contain zero to many data sources. You should create a repository class for each different type of data you handle in your app.

Repository classes are responsible for the following tasks:
Exposing data to the rest of the app, Centralizing changes to the data, Resolving conflicts between multiple data sources, Abstracting sources of data from the rest of the app, Containing business logic.

Each data source class should have the responsibility of working with only one source of data, which can be a file, a network source, or a local database. Data source classes are the bridge between the application and the system for data operations.



Resources to Learn & Develop

<https://developer.android.com/courses/android-basics-compose/course>

Mobile Application Development

Mobile Platforms and Application Development fundamentals

Lecture Plan

- Introduction to App Development
- Mobile Platforms and Application Development fundamentals
- Mobile Interface Design Concepts and UI/UX Design
- Introduction to Android Operating System
- Main Components of Android Application
- Sensors and Media Handling in Android Applications
- Data Handling in Android Applications
- **Android Application Testing and security aspects**

Purpose of Software Testing

- What is a software test?
 - A piece of software which executes another piece of software.
 - Validates if the code results as expected.
 - Software unit tests help the developer to verify that the logic of piece of the program is correct.

Android App Testing

- Android app testing is a complex task due to the existence of multiple device manufacturers, device models, Android OS versions, screen sizes, and network conditions.
- ✓ Testing on Real Android Devices
 - Testing against a wide selection of devices from various manufacturers with different screen resolutions and Android OS versions.
- ✓ Immediate new Android version support
 - Supported for newly release devices and Android versions

Android App Testing

- ✓ Test complex scenarios and custom UI elements
 - Testing coverage integrations with device components, peripherals, and system apps such as camera, audio, GPS, Google now, Google Assistant or Google Maps.
 - Automate customized actions and UI elements such as sliders, pickers, tables, gestures, etc.
- ✓ Test performance to ensure a great user experience
 - Able to catch performance issues before deployment.

To-Do before Mobile Testing for first release

1. Research on OS and Devices
2. Test Bed
3. Test plan
4. Automation Tools
5. Testing techniques or methods

Best Practices in Android App Testing

- ✓ Device Selection
- ✓ Beta Testing of the Application
- ✓ Connectivity
- ✓ Manual or Automated Testing

Testing Types for Mobile Apps

1. Functional Testing

- ✓ Checks whether the application is working based on the requirements.
- ✓ The flow of use cases and various business rules are tested.

Eg:

To validate whether – all required mandatory fields are working as required.

- the device is able to perform required multitasking requirements.
- navigation between relevant modules in the app as per the requirement.
- the user receives appropriate error messages like “network error, try again after time”, etc..

some

Testing Types for Mobile Apps

2. Android UI Testing

- ✓ User-centric testing of the application is done under this.
- ✓ Normally performed by manual users.

Eg: testing

- visibility of text in various screens of the app
- interactive messages
- alignment of data
- look and feel of the app for different screens
- whether the buttons are in required size and suitable to big fingers.
- whether the icons are natural and consistent with the app.
- size of fields, etc..

Testing Types for Mobile Apps

3. Compatibility Testing

- ✓ Performed to ensure that the app is fit across all the devices because they have different size, resolution, screen, version and hardware.
- ✓ So, mostly done in form of two matrices
 1. OS vs. App
 2. Device model vs. App

Eg: To ensure – the UI of the app is as per the screen size of the device and no text/control is partially invisible or inaccessible.

- text is readable for all users.
- call/alarm functionality is enabled whenever the app is running.

Testing Types for Mobile Apps

4. Interface Testing / Integration Testing

- ✓ This is done after all the modules of the app are completely developed.
- ✓ Includes a complete end-to-end testing of the app, interaction with other apps like Maps and social apps, usage of microphone to enter text, usage of camera to scan a barcode or to take a picture, etc.

Testing Types for Mobile Apps

5. Network Testing

- ✓ Mainly done to verify the response time in which the activity is performed like refreshing data after sync or loading data after login.
- ✓ This is an in-house testing.
- ✓ Done for both strong WiFi connection and the mobile data network.
- ✓ Request/response to/from the service is tested for various conditions.
- ✓ App should talk to the immediate service to carry out the process.

Testing Types for Mobile Apps

6. Performance Testing

- ✓ Performance of the app under some conditions are checked.
- ✓ Tested from both application end and the app server end.
- ✓ Conditions- Low memory in the device
 - The battery is extremely at a low level.
 - Poor/Bad network reception.

Testing Types for Mobile Apps

7. Installation Testing

- ✓ This is done to ensure that the installation of the app is going smoothly without ending up in errors or partial installation etc.
- ✓ Upgrade and uninstallation testing are carried out as part of this testing.

Testing Types for Mobile Apps

8. Security Testing

- ✓ Testing for the data flow for encryption and decryption mechanism is tested under this.

Eg: -To validate whether the app is not permitting an attacker to access sensitive content or functionality without proper authentication.

- To validate the app has a strong password protection system.

- To prevent from insecure data storage in the keyboard cache of the applications.

Testing Types for Mobile Apps

9. Field Testing

- ✓ Done specifically for the mobile data network.
- ✓ Doing only after the whole app is developed.
- ✓ Verify the behavior of the app when the phone has 2G or 3G connection.
- ✓ This testing verifies if the app is crashing under slow network connection or if it is taking too long to load the information.

Testing Types for Mobile Apps

10. Interrupt Testing (Offline Scenario Verification)

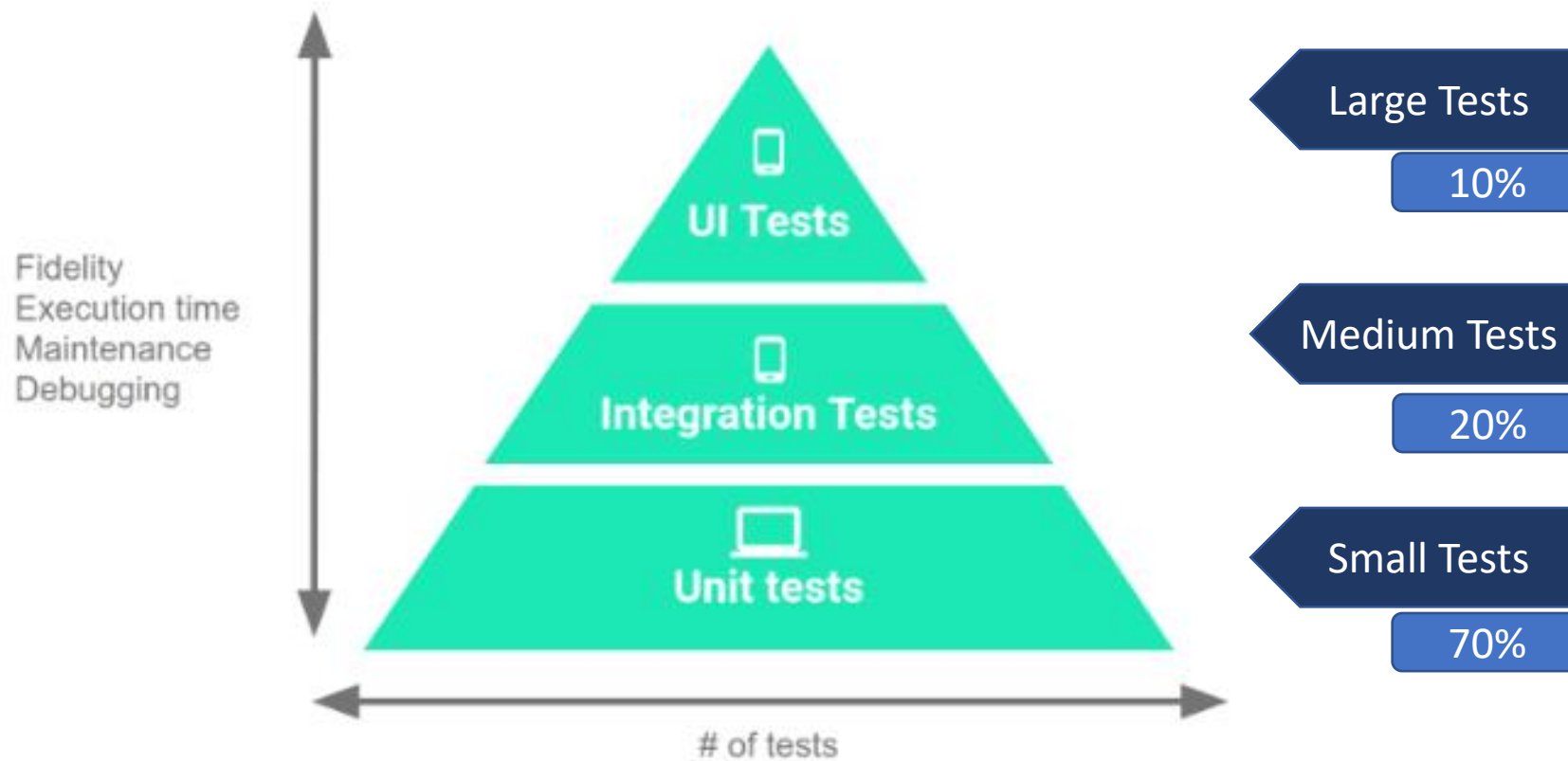
✓ Offline conditions – Condition where the communication breaks in the middle

- Eg:
- Data cable removal during data transfer process.
 - Network outage during the transaction posting phase.
 - Network recovery after an outage.
 - Battery removal or power ON/Off when it is in the transactional phase.

Mobile Testing tools

- Kobiton
 - TestProject
 - Squish By FroLogic
 - TestingBot
 - Apptim
 - Headspin
 - Appium (iOS/Android Testing tool)
 - Selendroid
 - MonkeyRunner
 - Calabash
- KIF
 - Testroid
 - Robotium - Android
 - Robo-electric - Android

Writing Tests



Testing Pyramid

Write Small Tests

- Highly focused to Unit tests.

Local Unit tests	Instrumented Unit tests
• Use AndroidX Test APIs	• Can be done on a physical device or emulator
• Can be used Roboelectric for tests that always run on a JVM-powered development machine	• AndroidX test makes use of following threads. - Main thread (UI thread/ activity thread) -> occur UI interactions and activity lifecycle events - Instrumentation thread -> most of the tests are run under this. When the test suit begins, the AndroidJUnitTest class starts this thread.
• Roboelectric supports: - Component lifecycles - Event loops - All resources	

Write Small Tests

- Highly focused to Unit tests.

Local Unit tests	Instrumented Unit tests
• Use AndroidX Test APIs	• Can be done on a physical device or emulator
• Can be used Roboelectric for tests that always run on a JVM-powered development machine	• AndroidX test makes use of following threads. - Main thread (UI thread/ activity thread) -> occur UI interactions and activity lifecycle events - Instrumentation thread -> most of the tests are run under this. When the test suit begins, the AndroidJUnitTest class starts this thread.
• Roboelectric supports: - Component lifecycles - Event loops - All resources	

Write Medium Tests

- Validate the collaboration and interaction of a group of units.

Eg:

- ✓ Interactions between a view and view model (testing a Fragment object, validating a layout XML, evaluation a data binding logic of a ViewModel object)
 - ✓ Testing of app's repository layer – verify different data sources and data access objects interact as expected.
- Use methods from the **Espresso Intents** library.

Write Large Tests

- Validate end-to-end workflows that guide users through multiple modules and features.

Configuring the environment in Android Studio

- ✓ Organize test directories based on execution environment

Android Studio contains two directories to locate tests.

1. androidTest – Contains the tests that run on **real or virtual devices**.

- Tests include **integration tests, end-to-end tests** and other tests where JVM alone cannot validate the app's functionality.

2. test – Contains the tests that run on the **local machine** such as **unit tests**.

Testing annotations in JUnit4

JUnit 4	Description
<code>import org.junit.*</code>	Import statement for using the following annotations.
<code>@Test</code>	Identifies a method as a test method.
<code>@Before</code>	Executed before each test. It is used to prepare the test environment (e.g., read input data, initialize the class).
<code>@After</code>	Executed after each test. It is used to cleanup the test environment (e.g., delete temporary data, restore defaults). It can also save memory by cleaning up expensive memory structures.
<code>@BeforeClass</code>	Executed once, before the start of all tests. It is used to perform time intensive activities, for example, to connect to a database. Methods marked with this annotation need to be defined as <code>static</code> to work with JUnit.
<code>@AfterClass</code>	Executed once, after all tests have been finished. It is used to perform clean-up activities, for example, to disconnect from a database. Methods annotated with this annotation need to be defined as <code>static</code> to work with JUnit.

Testing annotations in JUnit4

<code>@AfterClass</code>	Executed once, after all tests have been finished. It is used to perform clean-up activities, for example, to disconnect from a database. Methods annotated with this annotation need to be defined as <code>static</code> to work with JUnit.
<code>@Ignore</code> or <code>@Ignore("Why disabled")</code>	Marks that the test should be disabled. This is useful when the underlying code has been changed and the test case has not yet been adapted. Or if the execution time of this test is too long to be included. It is best practice to provide the optional description, why the test is disabled.
<code>@Test (expected = Exception.class)</code>	Fails if the method does not throw the named exception.
<code>@Test(timeout=100)</code>	Fails if the method takes longer than 100 milliseconds.

Methods to assert test results

Statement	Description
<code>fail([message])</code>	Let the method fail. Might be used to check that a certain part of the code is not reached or to have a failing test before the test code is implemented. The message parameter is optional.
<code>assertTrue([message,] boolean condition)</code>	Checks that the boolean condition is true.
<code>assertFalse([message,] boolean condition)</code>	Checks that the boolean condition is false.
<code>assertEquals([message,] expected, actual)</code>	Tests that two values are the same. Note: for arrays the reference is checked not the content of the arrays.
<code>assertEquals([message,] expected, actual, tolerance)</code>	Test that float or double values match. The tolerance is the number of decimals which must be the same.

Methods to assert test results

<code>assertNull([message,] object)</code>	Checks that the object is null.
<code>assertNotNull([message,] object)</code>	Checks that the object is not null.
<code>assertSame([message,] expected, actual)</code>	Checks that both variables refer to the same object.
<code>assertNotSame([message,] expected, actual)</code>	Checks that both variables refer to different objects.



That's all Folks!

Thank You!

Mobile Security – IE3112 SPYWARE and its architecture

SLIIT – Y3S1,S2

Tharaniyawarma.K – Y3S1,2 SLIIT

List of content

- What is Spyware
- Types of Spyware
- How it differ from other malware
- Behavior comparation
- Installation or Infection mechanisms
- Persistence methodologies
- Information gathering phases
- Communication phases
- Data Exfiltration Mechanism
- Payload delivery mechanisms
- Detection and removal

Types of Spyware

- Keyloggers or system monitors
- Adware and trojan ware
- Password stealers
- Banking trojan
- Infostealers
- Browser hijackers and web beacons
- Stalker ware
- USB spyware

What is Spyware

- Spyware refers to malicious software designed to infiltrate a computer system, gather information about the user or organization, and transmit that data to a third party without the user's consent. This information may include keystrokes, browsing history, personal data, and system information.
- It operates stealthily in the background, often without the user's knowledge or consent, and can compromise the privacy and security of the affected system.
- Spyware can take various forms and employ different techniques to achieve its objectives.

How it differ from other malware

- Spyware is a type of malware, but it differs from other types of malware in terms of its specific objectives, behaviors, and methods of operation.
- The primary objective of spyware is to gather sensitive information from the infected device without the user's knowledge or consent. Spyware focuses on stealthily monitoring and extracting data rather than causing immediate harm to the system.
- Other types of malware, such as viruses, worms, trojans, ransomware, and adware, may have different objectives. For example, viruses and worms aim to replicate and spread across systems, trojans disguise themselves as legitimate programs to trick users into executing them, ransomware encrypts files and demands payment for decryption, and adware displays unwanted advertisements to generate revenue.

Behavior comparison

- Spyware operates stealthily in the background, often without the user's awareness. It typically remains dormant until triggered by specific events or commands from a remote server. Spyware focuses on information gathering and communication with a remote attacker-controlled server to transmit the stolen data.
- Other types of malware may exhibit a wide range of behaviors depending on their objectives.

Installation or Infection mechanisms

These infection mechanisms highlight the diverse tactics and strategies employed by spyware authors to infiltrate devices and compromise user privacy and security.

Spyware can be installed on a device through various means, including:

- **Bundling:** Spyware often piggybacks on legitimate software downloads as bundled components. When users download and install software from untrustworthy sources or without paying attention to installation prompts, they may inadvertently install spyware along with the intended program.
- **Drive-By Downloads:** this occur when users visit compromised or malicious websites that exploit vulnerabilities in web browsers or plugins to automatically download and install spyware onto their devices without their consent.

Installation or Infection mechanisms (cont)

- **Social Engineering:** SE tactics manipulate users into installing spyware voluntarily. For example, attackers may create fake software updates, security alerts, or system warnings that prompt users to download and install malicious software under the guise of legitimate updates or tools.
- **Email Attachments:** Spyware can be distributed via email attachments, often disguised as legitimate files or documents. When users open these attachments, the spyware executes and infects the device.
- **Physical Access:** In some cases, spyware may be installed onto devices through physical access by attackers or individuals with malicious intent. For example, attackers may gain physical access to a device and manually install spyware onto it using removable media, such as USB drives, or by exploiting vulnerabilities in the device's security defenses.

Installation or Infection mechanisms (cont)

- Exploiting Software Vulnerabilities: Spyware authors exploit vulnerabilities in operating systems, applications, or plugins to remotely execute malicious code and install spyware onto vulnerable devices. These vulnerabilities may exist in outdated software versions or unpatched systems, allowing attackers to exploit security flaws and gain unauthorized access.
- Trojan Horses: Spyware may be bundled with trojan horse programs that masquerade as legitimate software or files to deceive users into executing them. Once executed, the trojan horse installs the spyware onto the device and may perform additional malicious activities, such as disabling security software or creating backdoors for remote access.

Persistence methodologies

Spyware is designed to remain undetected and maintain a presence on the infected device for as long as possible.

- Auto-Start Mechanisms: Spyware often configures itself to start automatically when the infected device boots up.
- Registry Entries: It may create or modify registry entries to ensure it runs each time the system starts.
- File System Modifications: Spyware may hide its files in system directories or disguise them as legitimate system files to evade detection and removal.

Information gathering phases

Once installed, spyware actively monitors the activities of the infected device and collects sensitive information.

- **Keystrokes:** Some spyware records keystrokes to capture passwords, usernames, credit card numbers, and other sensitive data.
- **Browsing History:** It may track websites visited by the user to gather information about their interests, habits, and online behavior.
- **Personal Information:** Spyware can harvest personal information such as email addresses, phone numbers, social security numbers, and more.
- **System Information:** It may collect details about the infected device, such as its hardware configuration, installed software, and network settings.

Communication phases

Spyware typically communicates with a remote server controlled by the attacker to transmit the stolen information and receive commands.

- HTTP/HTTPS: Spyware may use standard web protocols to send data to and receive instructions from a remote server.
- Domain Generation Algorithms (DGAs): Some sophisticated spyware variants use DGAs to dynamically generate domain names for command and control (C&C) servers, making it harder to block or disrupt communications.
- Peer-to-Peer (P2P): In some cases, spyware may use P2P networks to exchange information with other infected devices.

Data Exfiltration Mechanism

- Once the spyware has collected the desired information, it needs a way to transmit this data to the attacker's remote server or command-and-control (C2) infrastructure. Spyware typically employs various covert communication methods to evade detection, such as:
- Encrypted Communication: Encrypts the collected data before transmission to prevent interception and analysis by security tools.
- Command-and-Control Servers: Establishes communication with remote servers controlled by the attacker, allowing for the transfer of stolen data and receipt of commands or updates.
- Proxy Servers: Routes communication through intermediary servers to obfuscate the origin of the data and evade detection by network security measures.
- Steganography: Hides the transmitted data within seemingly innocuous files or network traffic to avoid detection by intrusion detection systems (IDS) and antivirus software.

Payload delivery mechanisms

In addition to gathering information, spyware may also deliver payloads designed to further compromise the infected device or perform additional malicious activities.

- **Additional Malware:** Spyware may download and install other types of malware, such as ransomware or banking trojans, to carry out specific tasks.
- **Advertising:** Some spyware displays unwanted advertisements or redirects the user's browser to malicious websites to generate revenue for the attacker.
- **System Modifications:** It may modify system settings or install browser extensions to manipulate search results, redirect web traffic, or perform other unauthorized actions.

Detection and removal

Anti-detection and Evasion Techniques:

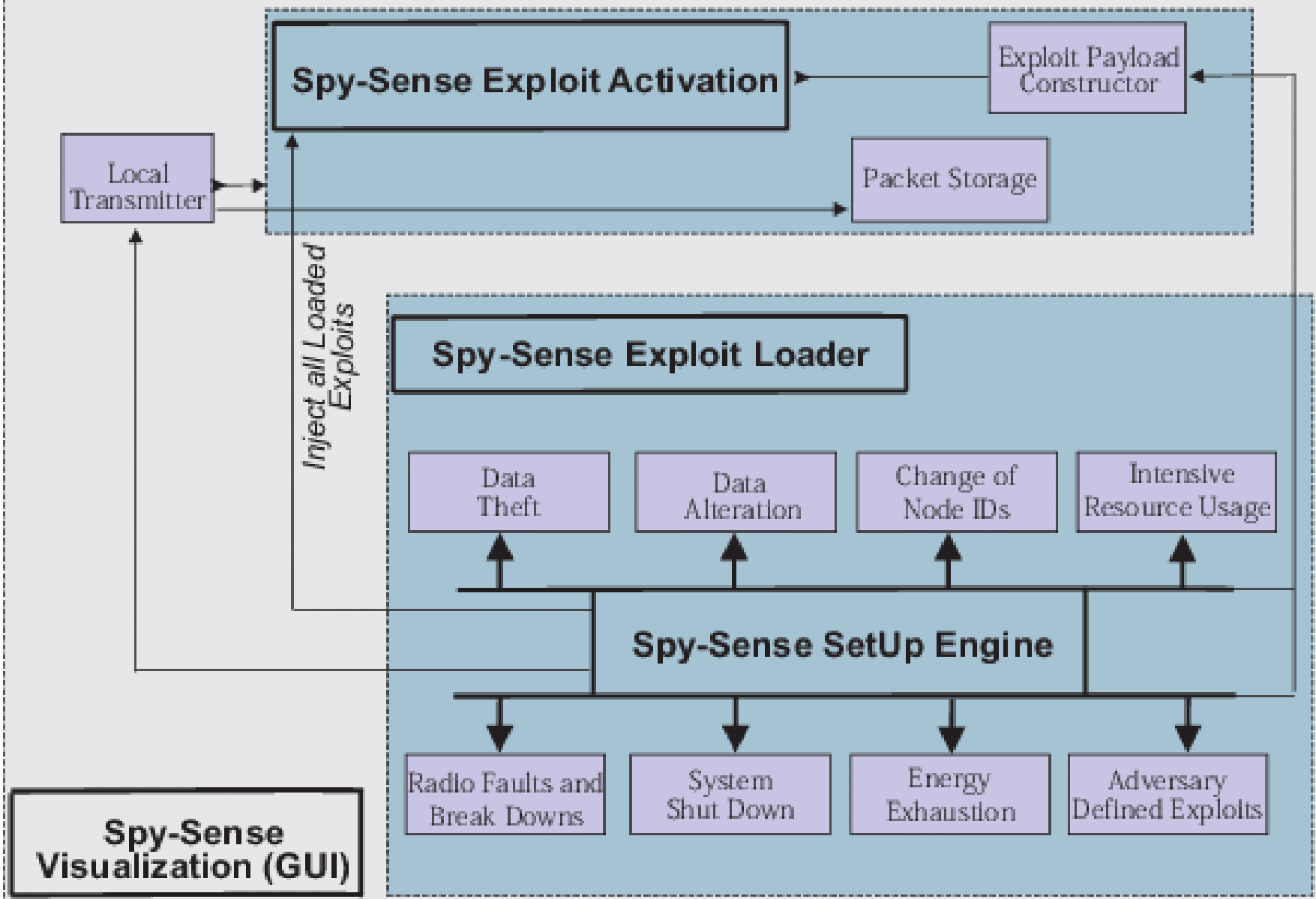
- Spyware developers constantly evolve their techniques to evade detection by security software. This may include code obfuscation, polymorphism, or encryption to obfuscate the spyware's behavior and evade signature-based detection.
- Fileless execution techniques allow spyware to operate entirely in memory without leaving traces on disk, making detection more challenging for traditional antivirus solutions.
- Spyware may employ rootkit capabilities to hide its presence from the operating system and security software, making it harder to detect and remove.

Detection and removal

- Antivirus/Anti-malware Software
- Firewalls
- Regular Software Updates
- Permission management
- Safe browsing practices
- Behavioral analysis
- Manual inspection and removal

Advanced technologies

- Remote Control and Update Mechanism
- API connection to master
- Remote screen
- Hardware access (camera, microphone)
- System permissions



Spyware examples - Computers

- CoolWebSearch uses security vulnerabilities found in Internet Explorer to take control, change settings and send browsing information to spyware authors.
- DarkHotel is a targeted spear phishing spyware that selectively attacks business hotel visitors through the hotel's Wi-Fi network.
- Emotet was one of the most prevalent threats in the 2010s. It acted as a Trojan that stole banking credentials from its victims.
- Gator is commonly found in file sharing software. It monitors a victim's web browsing habits to present the user with better-targeted ads.
- TIBS Dialer disconnects the user's computer from local phone lines and connects it to a toll number designed for accessing pornographic websites.
- Zlob downloads itself onto a computer to record keystrokes and search a user's browsing history.

Spyware examples - Computers

- FinSpy, or FinFisher, is an advanced suite of surveillance tools sold to law enforcement and intelligence agencies. FinSpy works on Windows, macOS, Linux, Android and iOS operating systems.
- GO Keyboard is a virtual Android keyboard app that masquerades as a legitimate mobile application. Once installed on an Android device, GO Keyboard transmits personal information to its remote servers without explicit consent from users.
- HawkEye, a keylogger virus, was dormant for years but returned during the early days of the COVID-19 pandemic. Infected machines track key logs and other inputs and share this information with a remote server.
- HuntBar is a Trojan application that hijacks web browser settings and downloads and installs adware without the user's knowledge.
- Look2Me is spyware that tracks user behavior, website logs and social media interactions and shares this information with a remote server.

Spyware examples - Smartphones

- PhoneSpy - researchers at Zimperium zLabs published a report about a spyware app they found in South Korea affecting Android devices in the wild.
- Android/SpyC23.A - One infamous APT group is ATP-C-23. ATP-C-23 use many types of attack, including Android/SpyC23. On Nov. 23, 2021, Sophos published a report stating that they'd discovered a new, powerful variant of spyware called Android/SpyC23.A
- Pegasus - It's safe to say that Israel-based NSO Group's Pegasus spyware disrupted the world of espionage, making headlines all over the world. Although the company claims that it helps nations fight terrorism and crime, evidence suggests that people are using Pegasus software for their personal agendas.
- Ghost RAT - Ghost RAT (also written as Gh0st RAT) is a trojan horse made for spying. RAT stands for "remote administration tool." This name is appropriate considering that Ghost RAT's operators, GhostNet System, use a C&C server to control victims' devices remotely.

Android App Bug Bounty and Auditing

A Practical Guide for Security Researchers

Contents

1. Introduction to Android App Security
2. Setting Up a Pen-testing Environment
3. Static and Dynamic Analysis Techniques
4. Common Vulnerabilities in Android Apps
5. Bug Bounty Platforms and Reporting
6. Case Studies and Hands-On Demo
7. Q&A

Introduction to Android App Security

- Android architecture overview
- APK structure - Manifest, classes.dex, res/, lib/, original/
- Security models - sandboxes, permissions

Why Bug Bounty on Android Apps

- Growing mobile user base.
- Apps handle sensitive data.
- Real world impact and rewards.

Android App Architecture

- Activities
- Services
- Broadcast Receivers
- Content Providers

Setting up your Environment

- Android Studio / AVD
- Genymotion
- Burp Suite
- Frida / Objection
- Jadx / apktool / MobSF

Mobile App Security Testing Methodology

- Information gathering
- Threat Modeling
- Static analysis
- Dynamic analysis
- Vulnerability identification
- Reporting

Static Analysis

- Decompiling APKs
- Reading Manifest files
- Identify hardcoded information
- Analyzing code logic with jadx

Dynamic Analysis

- Intercepting network traffic with Burp
- Instrumentation using Frida / Objection
- Runtime permission abuse

Common Android Vulnerabilities

- Insecure data storage
- Insecure communication
- Code obfuscation bypass
- Component exposure
- Intent manipulation
- Broken authentication
- WebView misuse
- Hardcoded information
- Outdated libraries

Android permissions & implications

- Dangerous permissions
- Permission escalation
- Implicit intents
- Over requesting permissions

More in Android / APK Auditing

- APK Reverse Engineering
- Frida and Objection for Runtime Analysis
- Intercepting Android App Traffic
- Finding Exported Components

Bug Bounty Platforms

And Writing a Good bug Report

- HackerOne
- Bugcrowd
- Intigriti
- Synack
- Self hosted programs
- Reproducibility is key
- Include PoC, screenshots
- Suggest impact and mitigation

Ethical Considerations

- Adherence to scope
- Disclosure process
- No Exploitation
- Data privacy

Case Study

Insecure data storage

- App stores tokens in plain text in shared_prefs
- Exploitable via root access or backup tools

Broken authentication

- Hardcoded API keys
- JWT token manipulation

Android App Hardening Techniques

- Proguard / R8
- Secure Data Storage
- SSL Pinning
- Root / Jailbreak Detection
- Code Obfuscation
- Secure WebViews

Best Practices & Tips

And Resources

- Always test with scope
- Respect user data and privacy
- Use automation (MobSF, Frida scripts)
- Join Security Communities
-
- OWASP MASVS & MSTG
- Android hackers handbook
- Frida and Objection docs
- APKMirror (for app version)

And common pitfalls

- Not respecting scope
- Ignoring rate limiting
- Submitting unverified findings
- Ignoring authentication and session management
- Lack of PoC
- Not reporting Confidential data exposure



OWASP Top 10 Mobile Risks

Jack Mannino, nVisium Security
Mike Zusman, Carve Systems
Zach Lanier, Intrepidus Group

OWASP Mobile Security Project

Agenda

- Introductions
- Mobile Security Project
- Mobile Threat Model
- Top 10 Risks
- Wrap Up/Q&A



Introductions

Mike Zusman

- Carve Systems
- Principal Consultant
- <http://www.carvesystems.com>



Jack Mannino

- nVisium Security
- CEO
- <https://www.nvisiumsecurity.com>



Zach Lanier

- Intrepidus Group
- Principal Consultant
- <https://intrepidusgroup.com>



Mobile Security Project

- Began Q3 2010
- ***Why*** Unique and different security risks
- ***Goal*** To build security into mobile dev. life cycle
- Interested? Contribute





Mobile Threat Model

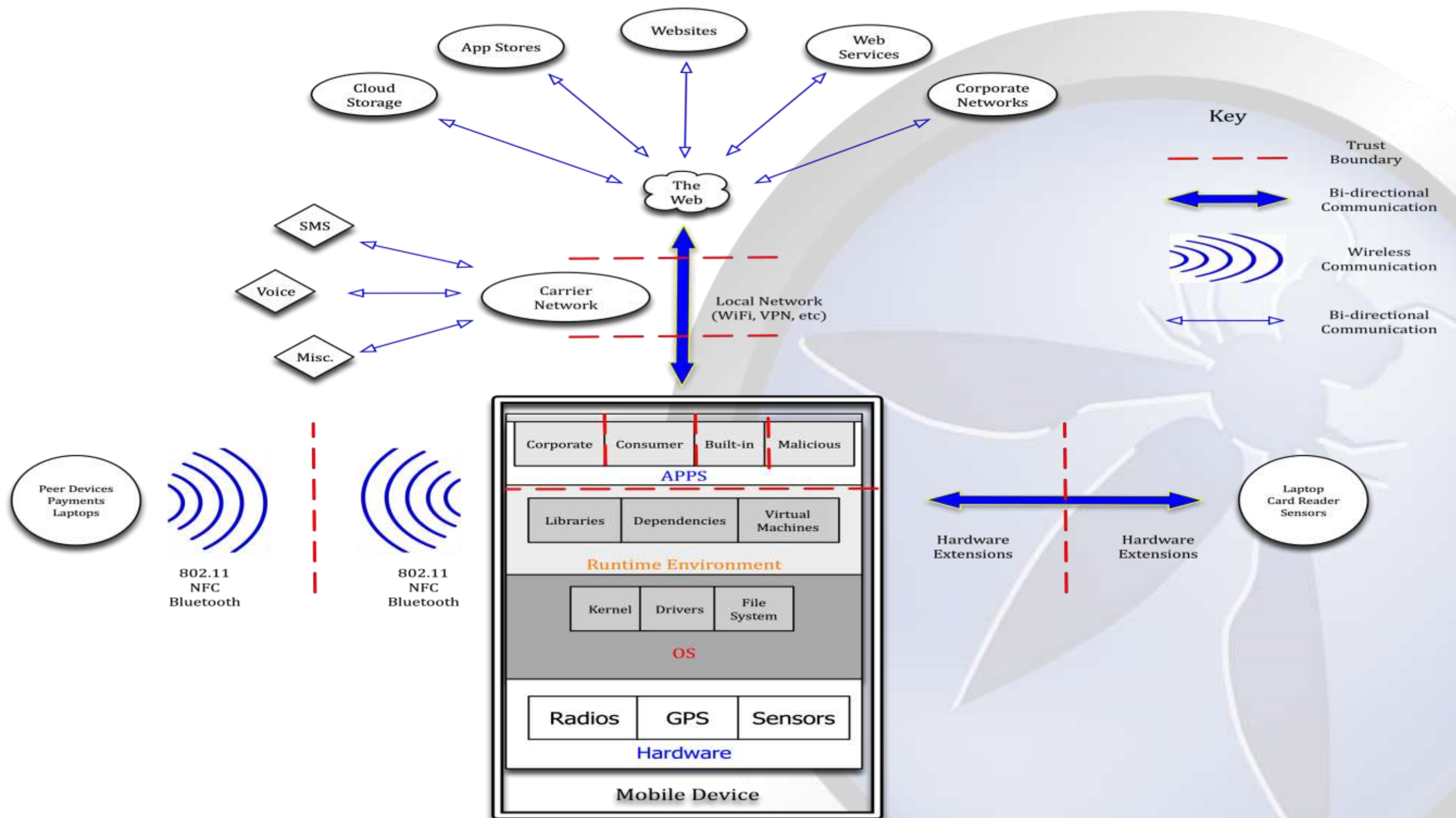




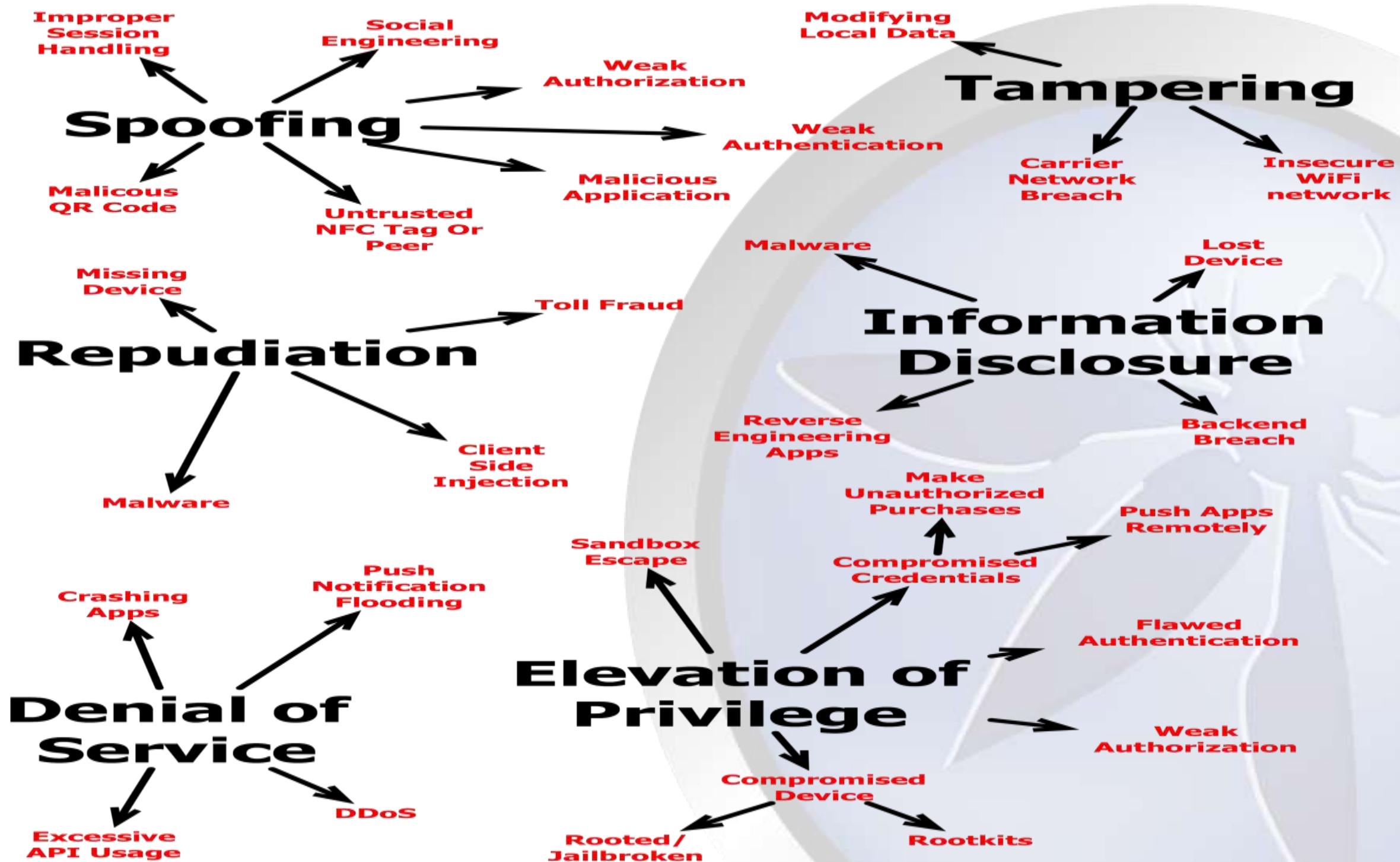
Mobile Threat Model

- Platforms vary with mileage
- Very different from traditional web app model due to wildly varying use cases and usage patterns
- Must consider more than the 'apps'
 - Remote web services
 - Platform integration (iCloud, C2DM)
 - Device (in)security considerations

Mobile Threat Model



Mobile Threat Model





Top 10 Risks



Top 10 Risks

- Intended to be platform-agnostic
- Focused on areas of risk rather than individual vulnerabilities
- Weighted utilizing the OWASP Risk Rating Methodology
 - https://www.owasp.org/index.php/OWASP_Risk_Rating_Methodology
- Thanks to everyone who participated

Top 10 Risks

OWASP Mobile Top 10 Risks

M1- Insecure Data Storage

M6- Improper Session Handling

M2- Weak Server Side Controls

M7- Security Decisions Via Untrusted Inputs

M3- Insufficient Transport Layer Protection

M8- Side Channel Data Leakage

M4- Client Side Injection

M9- Broken Cryptography

M5- Poor Authorization and Authentication

M10- Sensitive Information Disclosure

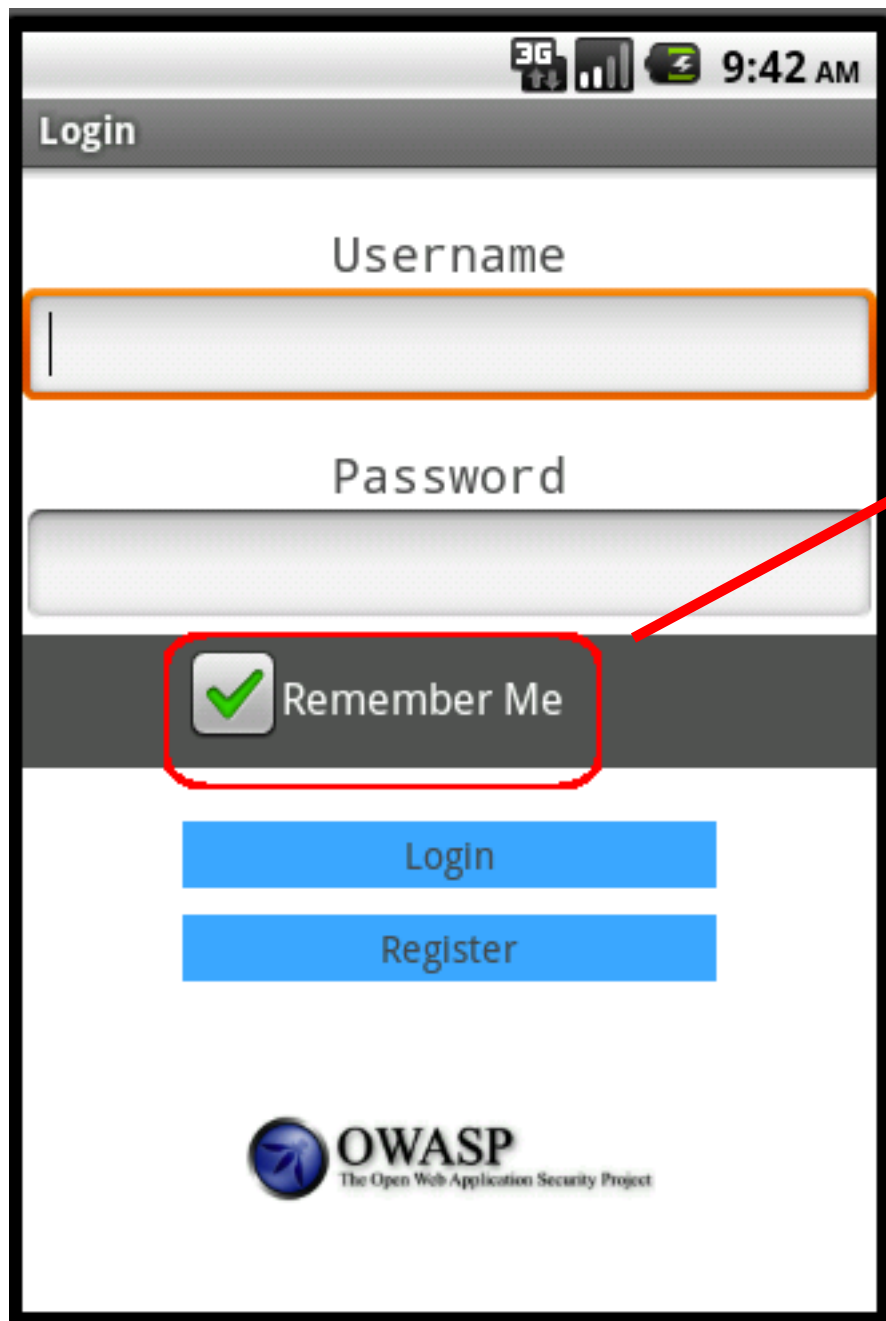
M1- Insecure Data Storage

- Sensitive data left unprotected
- Applies to locally stored data + cloud synced
- Generally a result of:
 - Not encrypting data
 - Caching data not intended for long-term storage
 - Weak or global permissions
 - Not leveraging platform best-practices

Impact

- Confidentiality of data lost
- Credentials disclosed
- Privacy violations
- Non-compliance

M1- Insecure Data Storage



A screenshot of a mobile application's login screen. At the top, the status bar shows '3G', signal strength, battery, and the time '9:42 AM'. The app title 'Login' is in a grey header. Below it are two text input fields: 'Username' and 'Password'. The 'Remember Me' checkbox, which is checked, is highlighted with a red box. Below the checkbox are two blue buttons: 'Login' and 'Register'. At the bottom is the OWASP logo with the text 'OWASP The Open Web Application Security Project'. A red arrow points from the 'Remember Me' checkbox to the code block on the right.

```
public void saveCredentials(String userName, String password) {  
    SharedPreferences credentials = this.getSharedPreferences(  
        "credentials", MODE_WORLD_READABLE); — Very Bad  
    SharedPreferences.Editor editor = credentials.edit();  
    editor.putString("username", userName); — Convenient!  
    editor.putString("password", password);  
    editor.putBoolean("remember", true);  
    editor.commit();  
}
```



M1- Insecure Data Storage

Prevention Tips

- Store ONLY what is absolutely required
- Never use public storage areas (ie- SD card)
- Leverage secure containers and platform provided file encryption APIs
- Do not grant files world readable or world writeable permissions

Control #	Description
1.1-1.14	Identify and protect sensitive data on the mobile device
2.1, 2.2, 2.5	Handle password credentials securely on the device

M2- Weak Server Side Controls

- Applies to the backend services
- Not mobile specific per se, but essential to get right
- We still can't trust the client
- Luckily, we understand these issues well
- Existing controls may need to be re-evaluated (ie- out of band comms)

Impact

- Confidentiality of data lost
- Integrity of data not trusted

M2- Weak Server Side Controls

OWASP Top 10



- https://www.owasp.org/index.php/Category:OWASP_Top_Ten_Project

OWASP Cloud Top 10



- <https://www.owasp.org/images/4/47/Cloud-Top10-Security-Risks.pdf>

M2- Weak Server Side Controls

Prevention Tips

- Understand the additional risks mobile apps introduce into existing architectures
- Leverage the wealth of knowledge that is already out there
- OWASP Web Top 10, Cloud Top 10, Web Services Top 10
- Cheat sheets, development guides, ESAPI

Control #	Description
5.1-5.8	Keep the backend APIs (services) and the platform (server) secure

M3- Insufficient Transport Layer Protection

- Complete lack of encryption for transmitted data
 - Yes, this unfortunately happens *often*
- Weakly encrypted data in transit
- Strong encryption, but ignoring security warnings
 - Ignoring certificate validation errors
 - Falling back to plain text after failures

Impact

- Man-in-the-middle attacks
- Tampering w/ data in transit
- Confidentiality of data lost

M3- Insufficient Transport Layer Protection

Real World Example: Google ClientLogin Authentication Protocol

- Authorization header sent over HTTP
- When users connected via wifi, apps automatically sent the token in an attempt to automatically synchronize data from server
- Sniff this value, impersonate the user

- <http://www.uni-ulm.de/in/mi/mitarbeiter/koenings/catching-authtokens.html>

M3- Insufficient Transport Layer Protection

Prevention Tips

- Ensure that all sensitive data leaving the device is encrypted
- This includes data over carrier networks, WiFi, and even NFC
- When security exceptions are thrown, it's generally for a reason...*DO NOT* ignore them!

Control #	Description
3.1.3.6	Ensure sensitive data is protected in transit

M4- Client Side Injection

- Apps using browser libraries

- Pure web apps
- Hybrid web/native apps

- Some familiar faces

- XSS and HTML Injection
- SQL Injection

- New and exciting twists

- Abusing phone dialer + SMS
- Abusing in-app payments

Impact

- Device compromise
- Toll fraud
- Privilege escalation

M4- Client Side Injection

Garden Variety XSS....

```
@Override
public void onCreate(Bundle savedInstanceState) {

    super.onCreate(savedInstanceState);
    setContentView(R.layout.demo);
    context = this.getApplicationContext();
    webView = (WebView) findViewById(R.id.demoWebView);
    webView.getSettings().setJavaScriptEnabled(true);
    webView.addJavascriptInterface(new SmsJSInterface(this),
        "smsJSInterface");
    GetSomeInfo getInfo = new GetSomeInfo();
    getInfo.execute(null, null);
}

public String generateHTML(String untrustedData) {

    return "<b>Check this out!</b><br>" + untrustedData;
}
```

With access to:

```
public class SmsJSInterface implements Cloneable {

    Context mContext;

    public SmsJSInterface(Context context) {

        mContext = context;
    }

    public void sendSMS(String phoneNumber, String message) {

        SmsManager sms = SmsManager.getDefault();
        sms.sendTextMessage(phoneNumber, null, message, null, null);
    }
}
```


M4- Client Side Injection

Prevention Tips

- Sanitize or escape untrusted data before rendering or executing it
- Use prepared statements for database calls...concatenation is still bad, and always will be bad
- Minimize the sensitive native capabilities tied to hybrid web functionality

Control #	Description
6.3	Pay particular attention to validating all data received from and sent to non-trusted third party apps before processing
10.1-10.5	Carefully check any runtime interpretation of code for errors

M5- Poor Authorization and Authentication

- Part mobile, part architecture
 - Some apps rely solely on immutable, potentially compromised values (IMEI, IMSI, UUID)
 - Hardware identifiers persist across data wipes and factory resets
 - Adding contextual information is useful, but not foolproof
- Impact**
- Privilege escalation
 - Unauthorized access

M5- Poor Authorization and Authentication

```
if (dao.isDevicePermanentlyAuthorized(deviceID)) {  
    int newSessionToken = LoginUtils.generateSessionToken();  
    dao.openConnection();  
    dao.updateAuthorizedDeviceSession(deviceID,  
        sessionToken, LoginUtils.getTimeMilliseconds());  
    bean.setSessionToken(newSessionToken);  
    bean.setUserName(dao.getUserName(sessionToken));  
    bean.setAccountNumber(dao.getAccountNumber(sessionToken));  
    bean.setSuccess(true);  
    return bean;  
}
```

M5- Poor Authorization and Authentication

Prevention Tips

- Contextual info can enhance things, but only as part of a multi-factor implementation
- Out-of-band doesn't work when it's all the same device
- Never use device ID or subscriber ID as sole authenticator

Control #	Description
4.1-4.6	Implement user authentication/authorization and session management correctly
8.4	Authenticate all API calls to paid resources

M6- Improper Session Handling

- Mobile app sessions are generally MUCH longer
- Why? Convenience and usability
- Apps maintain sessions via
 - HTTP cookies
 - OAuth tokens
 - SSO authentication services
- Bad idea= using a device identifier as a session token

Impact

- Privilege escalation
- Unauthorized access
- Circumvent licensing and payments

M6- Improper Session Handling

Prevention Tips

- Don't be afraid to make users re-authenticate every so often
- Ensure that tokens can be revoked quickly in the event of a lost/stolen device
- Utilize high entropy, tested token generation resources

Control #	Description
1.13	Use non-persistent identifiers
4.1-4.6	Implement user authentication/authorization and session management correctly

M7- Security Decisions Via Untrusted Inputs

- Can be leveraged to bypass permissions and security models
 - Similar but different depending on platform
 - iOS- Abusing URL Schemes
 - Android- Abusing Intents
 - Several attack vectors
 - Malicious apps
 - Client side injection
- Impact**
- Consuming paid resources
 - Data exfiltration
 - Privilege escalation

M7- Security Decisions Via Untrusted Inputs

Skype iOS URL Scheme Handling Issue



- <http://software-security.sans.org/blog/2010/11/08/insecure-handling-url-schemes-apples-ios/>

M7- Security Decisions Via Untrusted Inputs

Prevention Tips

- Check caller's permissions at input boundaries
- Prompt the user for additional authorization before allowing
- Where permission checks cannot be performed, ensure additional steps required to launch sensitive actions

Control #	Description
10.2	Run interpreters at minimal privilege levels

M8- Side Channel Data Leakage

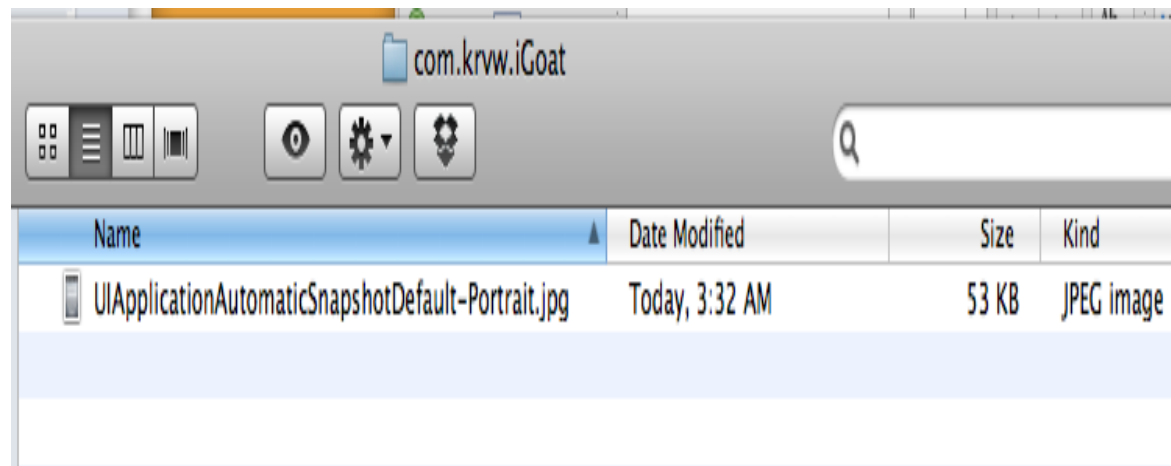
- Mix of not disabling platform features and programmatic flaws
- Sensitive data ends up in unintended places
 - Web caches
 - Keystroke logging
 - Screenshots (ie- iOS backgrounding)
 - Logs (system, crash)
 - Temp directories
- Understand what 3rd party libraries in your apps are doing with user data (ie- ad networks, analytics)

Impact

- Data retained indefinitely
- Privacy violations

M8- Side Channel Data Leakage

Screenshots



Logging

```
try {
    userInfo = client.validateCredentials(userName, password);
    if (userInfo.get("success").equals("true"))
        launchHome(v);
    else {
        Log.w("Failed login", userName + " " + password);
    }
} catch (Exception e) {
    Log.w("Failed login", userName + " " + password);
}
```

M8- Side Channel Data Leakage

Prevention Tips

- Never log credentials, PII, or other sensitive data to system logs
- Remove sensitive data before screenshots are taken, disable keystroke logging per field, and utilize anti-caching directives for web content
- Debug your apps before releasing them to observe files created, written to, or modified in any way
- Carefully review any third party libraries you introduce and the data they consume
- Test your applications across as many platform versions as possible

Control #	Description
7.3	Check whether you are collecting PII, it may not always be obvious
7.4	Audit communication mechanisms to check for unintended leaks (e.g. image metadata)

M9- Broken Cryptography

- Two primary categories
 - Broken implementations using strong crypto libraries
 - Custom, easily defeated crypto implementations
- Encoding != encryption
- Obfuscation != encryption
- Serialization != encryption

Impact

- Confidentiality of data lost
- Privilege escalation
- Circumvent business logic

M9- Broken Cryptography

```
ldc literal_876:"QIVtT0JoVmY2N2E="
invokestatic byte[] decode( java.lang.String ) // Base 64
invokespecial_lib java.lang.String.<init> // pc=2
astore 8
```

```
private final byte[]
com.picuploader.BizProcess.SendRequest.routine_12998
    (com.picuploader.BizProcess.SendRequest, byte[], byte[] );
{
    enter
    new_lib net.rim.device.api.crypto.TripleDESKey
```


M9- Broken Cryptography

Prevention Tips

- Storing the key with the encrypted data negates everything
- Leverage battle-tested crypto libraries vice writing your own
- Take advantage of what your platform already provides!

Control #	Description
1.3	Utilize file encryption API's
2.3	Leverage secure containers

M10- Sensitive Information Disclosure

- We differentiate by stored (M1) vs. embedded/hardcoded (M10)
 - Apps can be reverse engineered with relative ease
 - Code obfuscation raises the bar, but doesn't eliminate the risk
 - Commonly found "treasures":
 - API keys
 - Passwords
 - Sensitive business logic
- Impact
- Credentials disclosed
 - Intellectual property exposed

M10- Sensitive Information Disclosure

```
if (rememberMe)
    saveCredentials(userName, password);
//our secret backdoor account
if (userName.equals("all_powerful")
    && password.equals("iamsosmart"))
    launchAdminHome(v);
```

```
public static final double SECRET_SAUCE_FORMULA = (1.2344 * 4.35 - 4 + 1.442) * 2.221;
```

M10- Sensitive Information Disclosure

Prevention Tips

- Private API keys are called that for a reason...keep them off of the client
- Keep proprietary and sensitive business logic on the server
- Almost never a legitimate reason to hardcode a password (if there is, you have other problems)

Control #	Description
2.10	Do not store any passwords or secrets in the application binary



Wrap Up



Going Forward

- 60 day review period open to the public
- RC1 then becomes 'Final v1.0'
- 12 month revision cycle
 - Rapidly evolving platforms
 - Stale data = not as useful
- If you have suggestions or ideas, we want them!

Conclusion

- This is a good start, but we have a long way to go
- We've identified the issues...now we have to fix them
- Platforms must mature, frameworks must mature, apps must mature
- The OWASP Mobile body of knowledge must grow

Q&A

Thanks for listening!

- Jack Mannino jack@nvisiumsecurity.com
http://twitter.com/jack_mannino
- Zach Lanier zach.lanier@intrepidusgroup.com
<http://twitter.com/quine>
- Mike Zusman mike.zusman@carvesystems.com
<http://twitter.com/schmoilito>